
深度学习与计算物理

Deep Learning and Computational Physics
(Lecture Notes)

Deep Ray, Orazio Pinti and Assad A. Oberai

Department of Aerospace and Mechanical Engineering
University of Southern California, Los Angeles, California, USA

arXiv v1 · arXiv · 2023

Model: Codex (GPT-5)

审核: 曹溪能

Updated: 31 Aug 2026

译本说明

本译本以 arXiv:2301.00942v1 的 88 页 PDF 为可见内容与结构的最高依据，arXiv 源码仅用于恢复公式、标签、文献键与原始图件。译文保留原讲义的章节、公式、问题、图表、引用和编号关系；发现的源文疑点记录于项目的 issues.md，不作静默修订。

审核：曹溪能

31 Aug 2026

目录

译本说明	i
前言	1
第 1 章 引言	2
1.1 计算物理	2
1.2 机器学习	2
1.2.1 机器学习示例	3
1.2.2 按学习任务划分的机器学习算法类型	3
1.3 人工智能、机器学习与深度学习	4
1.4 机器学习与计算物理	5
第 2 章 深度神经网络导论	7
2.1 MLP 架构	7
2.2 激活函数	8
2.2.1 线性激活	8
2.2.2 修正线性单元 (ReLU)	9
2.2.3 Leaky ReLU	9
2.2.4 Logistic 函数	10
2.2.5 双曲正切	10
2.2.6 正弦	10
2.3 网络的表达能力	11
2.3.1 通用逼近结果	11
2.4 神经网络的训练、验证与测试	13
2.5 泛化能力	14
2.5.1 正则化	15
2.6 梯度下降	16
2.7 若干高级优化算法	17
2.7.1 动量方法	18
2.7.2 Adam	18
2.7.3 随机优化	19
2.8 使用反向传播计算梯度	21
2.9 回归与分类	22
第 3 章 残差神经网络	25
3.1 深度网络中的梯度消失	25
3.2 ResNet	26
3.3 与 ODE 的联系	27
3.4 神经 ODE	28
第 4 章 用 MLP 求解 PDE	31
4.1 有限差分法	32

4.2	谱配置法	34
4.3	物理信息神经网络 (PINN)	36
4.4	将 PINN 推广到更一般的 PDE	38
4.5	PINN 的误差分析	39
4.6	使用 PINN 进行数据同化	40
第 5 章	卷积神经网络	42
5.1	函数与图像	42
5.2	函数的卷积	42
5.2.1	例 1: Gaussian 核	43
5.2.2	例 2: 导数核	43
5.3	离散卷积	45
5.4	与有限差分的联系	46
5.5	卷积层	46
5.5.1	平均池化与最大池化	47
5.5.2	多通道输入的卷积	48
5.6	卷积神经网络 (CNN)	49
5.7	转置卷积层	50
5.8	图像到图像的变换	53
第 6 章	算子网络	54
6.1	PINN 的局限	54
6.2	参数化 PDE	54
6.3	算子	55
6.4	深度算子网络 (DeepONet) 架构	56
6.5	训练 DeepONet	57
6.6	DeepONet 的误差分析	59
6.7	物理信息 DeepONet	59
6.8	Fourier 神经算子——架构	60
6.9	Fourier 神经算子的离散化	62
6.10	Fourier 变换的使用	63
第 7 章	概率深度学习	65
7.1	概率论的基本要素	65
7.2	随机变量	67
7.2.1	累积分布函数	67
7.2.2	概率密度函数	69
7.2.3	重要随机变量示例	69
7.2.4	随机变量的期望与方差	70
7.2.5	随机变量对	71
7.3	无监督概率深度学习算法	73
7.3.1	生成对抗网络	74

7.4	监督概率深度学习算法	77
7.4.1	条件生成对抗网络	77
参考文献		79
术语索引		82

前言

这份讲义整理自南加州大学开发并讲授的一门课程. 对于具有扎实应用数学基础的一般工科研究生而言, 其内容应当是可以理解的.

本讲义的主要目标, 是向熟悉线性代数和偏微分方程概念的学生介绍若干精选的深度学习主题. 为此, 讲义利用深度学习算法与较为传统的计算物理技术之间的紧密联系来实现两个目标. 第一, 借助计算物理中的概念来理解深度学习算法. 许多深度学习概念都能与计算物理中的相似概念联系起来, 这一点并不令人意外; 利用这种联系, 可以更好地理解这些算法. 第二, 若干新颖的深度学习算法可用于求解计算物理中的困难问题. 因而, 对于有兴趣为物理现象建模的人来说, 这些算法提供了一套互补工具.

引言

本课程讨论位于计算物理 (computational physics) 与机器学习 (machine learning) 交界处的主题. 在理解为何需要结合这两个重要概念之前, 我们需要先分别理解它们各自的含义.

1.1 计算物理

计算物理在解决许多科学与工程问题时发挥着基础性作用. 为理解这一概念, 我们简要列出求解物理问题所涉及的关键步骤:

1. 考察一个物理现象, 并收集某个感兴趣可观测量的测量数据. 例如, 研究海浪时, 可收集海洋浮标测得的水位与波向数据.
2. 根据观测提出一条物理定律 (physical law). 例如, 观察到封闭系统中的流体质量始终守恒.
3. 写出该定律的数学描述. 其中可以使用常微分方程 (ODE)、偏微分方程 (PDE)、积分方程等.
4. 数学模型建立之后, 求出系统的解. 有两种途径:
 - (a) 在某些情形下, 可以得到解的精确解析形式. 例如, 可以使用分离变量法、拉普拉斯变换、傅里叶变换或积分因子求解 ODE/PDE.
 - (b) 在大多数情形下, 无法得到解的精确表达式, 必须用数值算法作适当近似. 例如, 求解 ODE 系统时可以使用向前或向后欧拉法、中点法或龙格-库塔格式; 求解 PDE 时则可使用有限差分法、有限体积法或有限元法.
5. 设计出精确或近似计算解的算法后, 用它验证数学模型, 即检查模型预测是否与收集的数据相符.

以上步骤概括地说明了计算物理所包含的内容.

1.2 机器学习

与计算物理不同, 机器学习 (ML) 并不要求先提出物理定律. 一般步骤如下:

1. 通过观测物理现象来收集数据, 可以实时测量某个可观测量, 也可以使用近似该现象的数值求解器生成数据.

2. 用收集的数据训练合适的算法，目的是发现不同样本之间的模式或关系。一些具体例子见节 1.2.1.
3. 训练完成后，使用 ML 算法作出未来预测，并用额外收集的数据验证它。

1.2.1 机器学习示例

1. **回归算法：**给定成对数据集 $\{(x_i, y_i) : 1 \leq i \leq N\}$ ，它对应于某个未知函数 $y = f(x)$ 。为了近似 f ，可以对该数据集拟合一个多项式（或任意其他基函数展开）。例如，求线性拟合 $\tilde{f}(x; a, b) = ax + b$ 的系数 a, b ，使误差

$$\mathcal{E}(a, b) = \sum_{i=1}^N |y_i - \tilde{f}(x_i)|^2$$

最小。若 $(a^*, b^*) = \arg \min_{a, b} \mathcal{E}(a, b)$ ，则可将 $\tilde{f}^*(x) := \tilde{f}(x; a^*, b^*)$ 视为 $f(x)$ 的近似（见图 1.1(a)）。

2. **决策树：**给定来自一个样本人群的数据集，其中包含年龄、收入等特征。数据又分为两组：A 组中的个人拥有住房，而 B 组中的个人没有住房。现在给定一个新数据点的特征，我们希望预测这个新人拥有住房的概率。决策树可用于求解这一分类（classification）问题。典型决策树的工作方式，是选择能使数据集中不同组样本的分离程度最大的切分（见图 1.1(b)）。随后，算法根据这些切分确定新点属于某个类别或组的概率。
3. **聚类算法：**给定一组数据，每个样本具有若干特征，寻找数据中的簇或模式（见图 1.1(c)）。

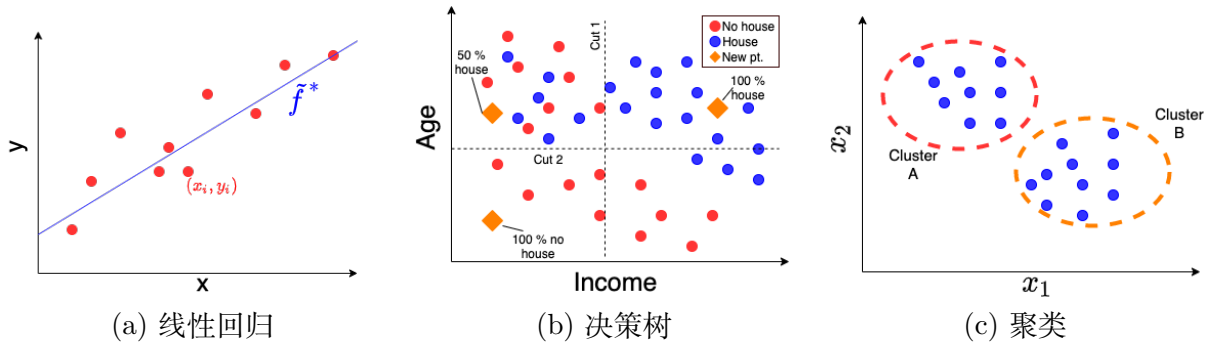


图 1.1: 机器学习示例.

1.2.2 按学习任务划分的机器学习算法类型

概括而言，机器学习算法有四种类型：

1. **监督学习：**给定数据 $\mathcal{S} = \{(x_i, y_i) : 1 \leq i \leq N\}$ ，对某个新的 $\hat{x}$ 预测 $\hat{y}$ ，且 $(\hat{x}, \hat{y}) \notin \mathcal{S}$ 。例如，给定一组图像及其标签（狗、猫、牛等），训练分类 ML 算法来学习图像与标签之间的关系，再用它预测新图像的标签。

2. **无监督学习**：给定数据 $\mathcal{S} = \{\mathbf{x}_i \in \Omega_x : 1 \leq i \leq N\}$ ，寻找 Ω_x 不同区域之间的关系。例如，在数据集中寻找簇，或者求出控制这些数据分布的概率分布 $p_x(\mathbf{x})$ 的表达式，并从中生成新样本。
3. **半监督学习**：这类方法介于监督学习与无监督学习之间。它们通常结合有标签与无标签数据进行训练。例如，假设有 10,000 张无标签图像，却只有 50 张有标签图像，我们能否利用这个数据集开发图像分类算法？
4. **强化学习**：这类方法由决策所获得的奖励或惩罚驱动学习。因而，它们会学习一条合适的路径或策略，以使奖励最大化。这类方法可用于训练下国际象棋或围棋的算法。

本课程将主要关注前两类 ML 算法。

1.3 人工智能、机器学习与深度学习

人工智能（AI）、ML 与深度学习（DL）这些术语有时会混用。实际上，它们三个彼此相关却不不同的概念。图 1.2 的维恩图说明了这一点。

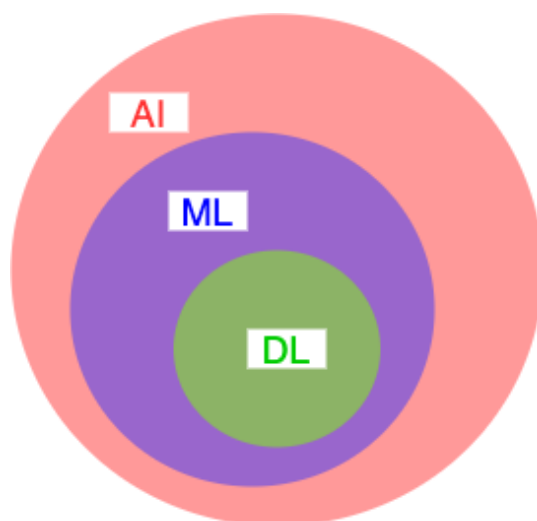


图 1.2: AI、ML 与 DL 之间的关系。

人工智能（artificial intelligence）指具有类人智能的系统。ML 是 AI 系统的关键组成部分，但系统还包含其他要素。自动驾驶汽车是 AI 的典型例子。下面仔细考察这种系统的设计（见图 1.3）。汽车上安装的摄像机会拍摄前方道路的实时图像或视频。随后，这些帧被送入一个 ML 算法作语义分割（semantic segmentation）：算法划分图像帧的不同区域，并对每个区域中的对象类型（汽车、树木、道路、天空等）进行分类。分割结果再传给决策系统，由它根据这幅分割图像决定汽车的下一步动作。信息随后经过控制模块，由该模块实际控制汽车的机械动作。整个过程模拟了真实驾驶员的行为，因此属于人工智能。

另一方面，机器学习（ML）指这个系统中使用数据训练的组成部分，也就是说，它们通过数据学习。在上例中，语义分割器就是这样的系统。许多 ML 算法都能使用数据

完成这一任务，本课程将学习其中一些。决策系统也可以是 ML 组件，即从以往数据中学习应当作出的适当决策；不过它也可以不是 ML 组件，例如可以是基于规则的专家系统。

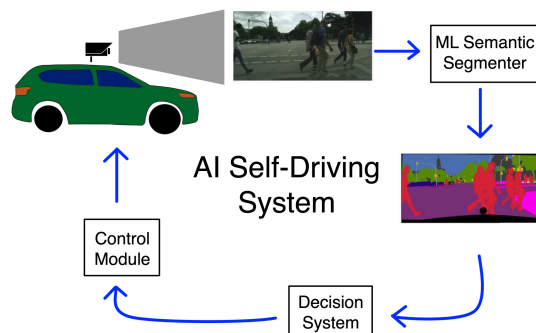


图 1.3: 自动驾驶汽车 AI 系统示意图. 部分插图取自 [32].

深度学习 (deep learning) 是 ML 算法的一个子集. 最简单的 DL 架构称为前馈网络 (feed-forward network). 它由多层非线性变换组成; 每个非线性变换逐分量作用于某个中间输出的仿射变换. 这种架构粗略地受到生物体中枢神经系统信号传递方式的启发. 章 2 将更详细地研究 DL 架构.

1.4 机器学习与计算物理

现在我们对计算物理和 ML 已有了较好的理解, 下一个自然的问题是: “为什么需要把二者结合起来考察?” 以下列出几个动机:

- 对于具有复杂模式的“物理”数据, ML 为表示数学定律提供了另一条途径. 考虑一个包含两个重要组成部分的物理过程: 其中一部分已得到充分理解并有可信的数学模型, 另一部分却理解不足且没有数学描述. 在这种情形下, 可对前一部分使用计算物理, 对后一部分使用 ML. 一个具体例子是由能量守恒和复杂本构模型共同控制的系统: 前者可能已有充分理解的数学模型, 后者则可能不得不依靠 ML 建模.
- ML 通常需要大量数据. 但物理知识有助于限制输入与解或预测所在的流形. 有了这种约束, 就能减少训练 ML 算法所需的数据量.
- 分析计算物理的工具 (泛函分析、数值分析、收敛到精确解的概念、概率框架) 也可迁移到 ML. 将这些工具应用于 ML, 有助于我们更好地理解并设计更好的 ML 算法.

本课程将覆盖的主题简要总结如下:

- 深度神经网络 (MLP) 及其收敛性;
- ResNet 及其与非线性 ODE (神经 ODE) 的联系;

- 循环网络及其与非线性 ODE 的联系；
- 卷积神经网络及其与 PDE 的联系；
- 随机梯度下降及其与 ODE 的关系；
- 求解 PDE 的深度学习算法；
- 逼近算子的深度学习算法；
- 生成对抗算法及其与计算物理的联系.

深度神经网络导论

本章将更仔细地考察现有网络架构中最简单的一种，即多层感知机 (multilayer perceptron, MLP)。

2.1 MLP 架构

我们的目标是使用一个 MLP 近似函数 $f: \mathbf{x} \in \mathbb{R}^d \mapsto \mathbf{y} \in \mathbb{R}^D$ ，并将该 MLP 记为 $\mathcal{F}$ 。MLP 的计算单元称为人工神经元 (artificial neuron)，它们依次堆叠成若干层。 $\mathcal{F}$ 的第零层称为源层 (source layer)；它不是计算层，只负责向网络提供一个 d 维输入。最后一层称为输出层 (output layer)，输出网络的 D 维预测。位于二者之间的其他各层均称为隐藏层 (hidden layer)。一层中的神经元数目定义了该层的宽度。具有两个隐藏层的 MLP 示意图图 2.1。

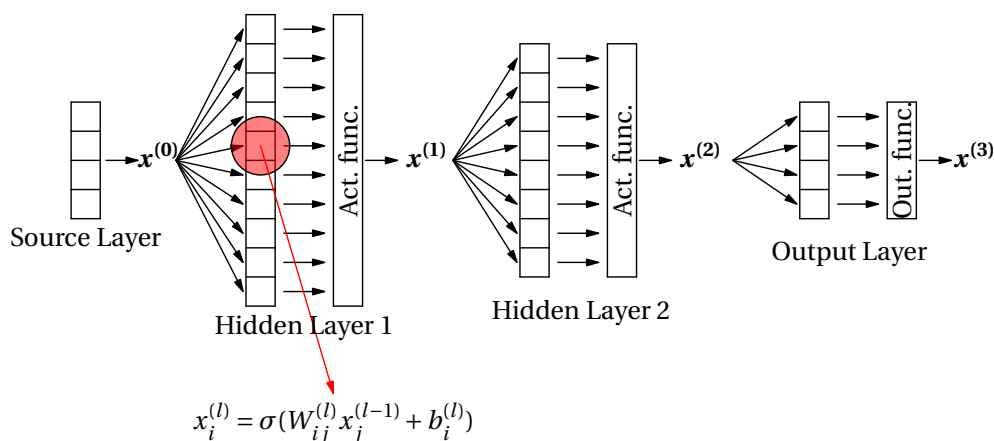


图 2.1: 具有两个隐藏层的 MLP.

为理解 MLP 内部发生的运算，先定义一些记号。考虑一个具有 L 个隐藏层的网络，对 $l = 0, 1, \dots, L+1$ ，以 H_l 表示第 l 层的宽度。为了与所要近似的函数 f 一致，必须有 $H_0 = d$ 和 $H_{L+1} = D$ 。记第 l 层的输出向量为 $\mathbf{x}^{(l)} \in \mathbb{R}^{H_l}$ ，它将作为下一层的输入。令 $\mathbf{x}^{(0)} = \mathbf{x} \in \mathbb{R}^d$ ，即输入层提供的输入信号。在每一层 l ($1 \leq l \leq L+1$) 中，第 i 个神经元先对该层的输入 $\mathbf{x}^{(l-1)}$ 作仿射变换，再作非线性变换：

$$x_i^{(l)} = \sigma \left(\underbrace{W_{ij}^{(l)} x_j^{(l-1)}}_{\text{爱因斯坦求和}} + b_i^{(l)} \right), \quad 1 \leq i \leq H_l, \quad 1 \leq j \leq H_{l-1}. \quad (2.1)$$

其中， $W_{ij}^{(l)}$ 和 $b_i^{(l)}$ 分别称为第 l 层第 i 个神经元的权重 (weight) 和偏置 (bias)。函数 $\sigma(\cdot)$ 称为激活函数 (activation function)，对于帮助网络表示复杂的非线性函数至关重要。

要. 令 $\mathbf{W}^{(l)} \in \mathbb{R}^{H_{l-1} \times H_l}$ 为第 l 层的权重矩阵, $\mathbf{b}^{(l)} \in \mathbb{R}^{H_l}$ 为该层的偏置向量, 则整层的作用可重写为

$$\mathbf{x}^{(l)} = \sigma(\mathcal{A}^{(l)}(\mathbf{x}^{(l-1)})), \quad \mathcal{A}^{(l)}(\mathbf{x}^{(l-1)}) = \mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}. \quad (2.2)$$

这里激活函数逐分量作用. 因而, 从数学上看, 整个网络 $\mathcal{F}: \mathbb{R}^d \mapsto \mathbb{R}^D$ 的作用是交替出现的仿射变换和逐分量激活的复合:

$$\mathcal{F}(\mathbf{x}) = \mathcal{A}^{(L+1)} \circ \sigma \circ \mathcal{A}^{(L)} \circ \sigma \circ \mathcal{A}^{(L-1)} \circ \dots \circ \sigma \circ \mathcal{A}^{(1)}(\mathbf{x}). \quad (2.3)$$

这里作几点说明:

1. 为使表示简单, 我们假设网络所有层都使用同一个激活函数, 但这并非严格规则. 事实上, 近期证据表明, 逐层交替使用激活函数能够得到更好的神经网络 [33].
2. 输出层末端有时会使用输出函数 O 而不是激活函数, 通常用来把输出改写成适当形式. 本课程后面会见到这类函数的例子.
3. 我们用网络的深度 (depth) 表示 MLP 中计算层的数目, 即隐藏层与输出层的总数; 按照上述记号, 它等于 $L + 1$.

网络的参数由所有权重和偏置组成, 记为

$$\boldsymbol{\theta} = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L+1} \in \mathbb{R}^{N_\theta}.$$

其中 N_θ 表示网络参数总数. 网络 $\mathcal{F}(\mathbf{x}; \boldsymbol{\theta})$ 表示一族参数化函数; 需要适当地选择 $\boldsymbol{\theta}$, 使网络在输入 $\mathbf{x}$ 处近似目标函数 $\mathbf{f}(\mathbf{x})$.

问题 2.1.1: 证明

$$N_\theta = \sum_{l=1}^{L+1} (H_{l-1} + 1) H_l.$$

2.2 激活函数

激活函数或许是 MLP 最重要的组成部分. 文献中有大量可用的激活函数, 各有优缺点. 下面考察其中几种 (另见图 2.2).

2.2.1 线性激活

最简单的激活函数是 $\sigma(\xi) = \xi$. 它具有以下特征:

- 函数无穷光滑, 但二阶以上的所有导数均为零.
- 函数值域为 $(-\infty, \infty)$.
- 如果所有层都使用线性激活函数, 整个网络就会退化为输入 $\mathbf{x}$ 的单个仿射变换. 换言之, 网络不过是目标函数 $\mathbf{f}$ 的线性近似; 当 $\mathbf{f}$ 高度非线性时, 这种近似没有用处.

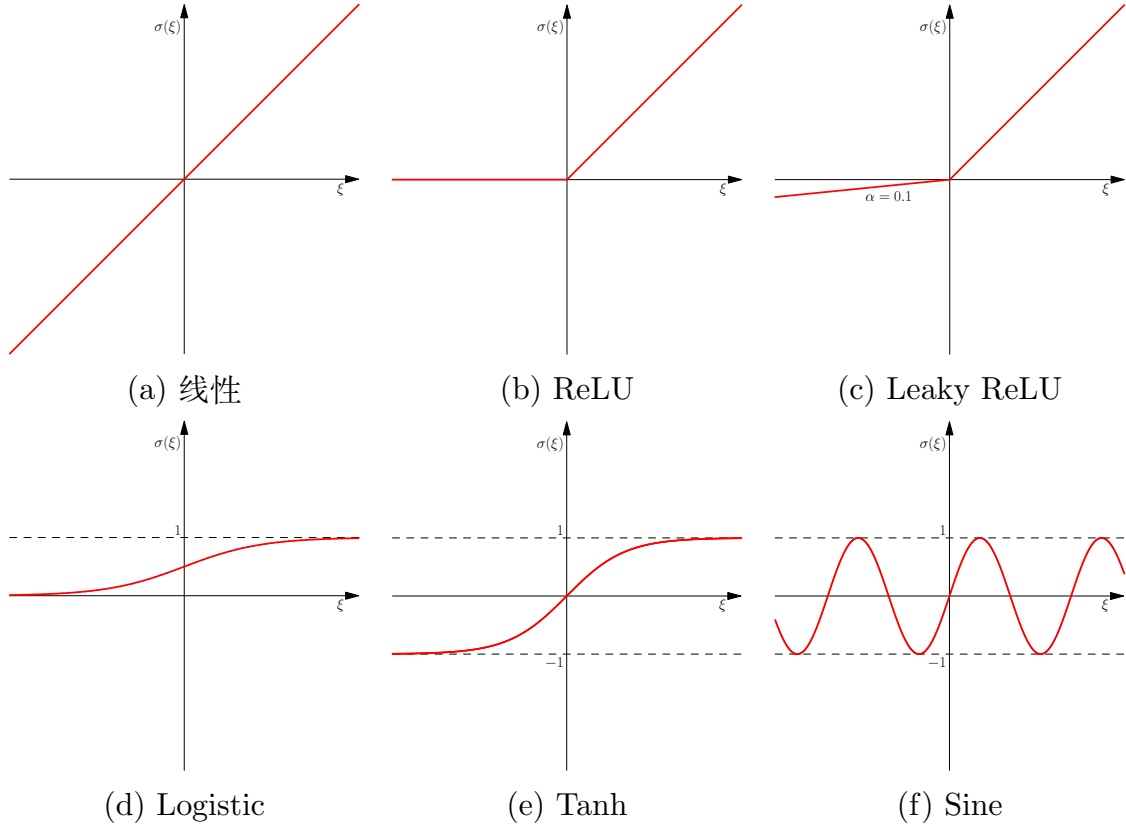


图 2.2: 激活函数示例.

2.2.2 修正线性单元 (ReLU)

该函数是分段线性的，定义为

$$\sigma(\xi) = \max\{0, \xi\} = \begin{cases} \xi, & \xi \geq 0, \\ 0, & \xi < 0. \end{cases} \quad (2.4)$$

它是实践中最常用的激活函数之一，具有以下特征：

- 函数连续，其导数为分段常数，并在 $\xi = 0$ 处发生跳跃. 二阶导数是在 $\xi = 0$ 处集中的狄拉克函数. 换言之，一阶以上的高阶导数没有良好定义.
- 函数值域为 $[0, \infty)$.

2.2.3 Leaky ReLU

若神经元的仿射变换为负，ReLU 激活会使该神经元输出为零. 训练神经网络时，这可能导致神经元死亡 (dying neuron) 现象 [15]：神经元完全退出网络，不再对最终预测作贡献. 为克服这一困难，人们设计了 ReLU 的泄漏版本：

$$\sigma(\xi; \alpha) = \begin{cases} \xi, & \xi \geq 0, \\ \alpha\xi, & \xi < 0, \end{cases} \quad (2.5)$$

其中 α 成为网络的超参数 (hyper-parameter). 该函数具有以下特征：

- Leaky ReLU 的导数性质与 ReLU 相同.
- 函数值域为 $(-\infty, \infty)$.

2.2.4 Logistic 函数

Logistic 或 Sigmoid 激活函数为

$$\sigma(\xi) = \frac{1}{1 + e^{-\xi}}, \quad (2.6)$$

并具有以下性质:

- 函数无穷光滑且单调.
- 函数值域为 $(0, 1)$, 即函数有界. 这类函数适合表示概率.
- 由于导数离开 $\xi = 0$ 后迅速衰减至零, 该激活函数可能导致网络训练收敛缓慢.

2.2.5 双曲正切

双曲正切函数可视为 Logistic 函数的对称延拓:

$$\sigma(\xi) = \frac{e^{\xi} - e^{-\xi}}{e^{\xi} + e^{-\xi}}, \quad (2.7)$$

并具有以下性质:

- 函数无穷光滑且单调.
- 函数值域为 $(-1, 1)$, 即函数有界. 它把零输入映射为零, 同时把正 (负) 输入推向 $+1$ (-1).
- 与 Logistic 函数类似, 双曲正切函数的导数离开 $\xi = 0$ 后迅速衰减至零, 因此也可能导致网络训练收敛缓慢.

2.2.6 正弦

近来, 正弦函数 $\sigma(\xi) = \sin(\xi)$ 被提出作为一种高效激活函数 [27]. 它兼具以上激活函数的良好特征:

- 函数无穷光滑.
- 函数值域为 $(-1, 1)$, 即函数有界.
- 该函数的各阶导数都不会衰减至零.

问题 2.2.1: 能否构造一种使用正弦激活函数的 MLP 架构, 使其给出的近似与傅里叶级数展开非常相似?

2.3 网络的表达能力

下面尝试理解增大 N_θ 的影响. 考虑一个使用 ReLU 激活函数 $\sigma(\xi) = \max\{\xi, 0\}$ 的简单例子. 令 $d = D = 1$ 、 $L = 1$, 并取参数

$$\mathbf{W}^{(1)} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} -2 \\ 0 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad b^{(2)} = 0.$$

如图 2.3(a) 所示, 各层输出为

$$\begin{aligned} x_1^{(1)} &= \max\{2x_1^{(0)} - 2, 0\}, & x_2^{(1)} &= \max\{x_1^{(0)}, 0\}, \\ x_1^{(2)} &= \max\{2x_1^{(0)} - 2, 0\} + \max\{x_1^{(0)}, 0\}. \end{aligned}$$

注意, 隐藏层输出 $\mathbf{x}^{(1)}$ 只有一个折角 (见图 2.3(b)、(c)), 而最终输出有两个折角 (见图 2.3(d)).

把上述构造推广到一个具有 L 个隐藏层、每层宽度均为 H 的更大网络. 可以预期, 每个 $x_i^{(1)}$ ($1 \leq i \leq H$) 都有一个折角, 其位置和夹角由隐藏层相应神经元的权重与偏置决定. 向量 $\mathbf{x}^{(1)}$ 传入下一隐藏层后, 每个神经元会组合这些单折角函数, 得到可能具有 H 个折角的输出. 第二隐藏层每个神经元输出中的 H 个折角, 其位置和夹角仍会不同. 位置之所以不同, 是因为每个神经元可具有不同偏置, 从而产生不同平移. 沿此思路继续, 当信号通过宽度为 H 的各隐藏层时, 折角数可依次按 H, H^2, H^3 增长; 一般而言, 折角总数可按 H^L 增长. 换言之, 随着网络深度 (和宽度) 增加, 网络能够具有更强的表达能力.

2.3.1 通用逼近结果

为以数学上严格的方式量化网络的表达能力, 下面考察 MLP 的若干逼近性质. 在这些结果中, 假设 $K \subset \mathbb{R}^d$ 是闭有界集.

定理 2.3.1 (Pinkus, 1999 [23]): 设 $f: K \rightarrow \mathbb{R}$ (即 $D = 1$) 为连续函数. 则给定任意 $\epsilon > 0$, 存在一个只有单隐藏层 ($L = 1$)、宽度 H 任意且采用非多项式连续激活函数 σ 的 MLP, 使得

$$\max_{\mathbf{x} \in K} |\mathcal{F}(\mathbf{x}; \boldsymbol{\theta}) - f(\mathbf{x})| \leq \epsilon.$$

定理 2.3.2 (Kidger, 2020 [9]): 设 $\mathbf{f}: K \rightarrow \mathbb{R}^D$ 为连续向量值函数. 则给定任意 $\epsilon > 0$, 存在一个具有任意隐藏层数 L 、每层宽度 $H \geq d + D + 2$, 并采用满足某些额外温和条件的连续激活函数 σ 的 MLP, 使得

$$\max_{\mathbf{x} \in K} \|\mathcal{F}(\mathbf{x}; \boldsymbol{\theta}) - \mathbf{f}(\mathbf{x})\| \leq \epsilon.$$

定理 2.3.3 (Yarotsky, 2021 [33]): 设 $f: K \rightarrow \mathbb{R}$ 具有二阶连续导数, 即 $f \in C^2(K)$. 考虑一个采用 ReLU 激活且 $H \geq 2d + 10$ 的 MLP. 则存在具有这一配置的网络, 使其误差按下式收敛:

$$\max_{\mathbf{x} \in K} |\mathcal{F}(\mathbf{x}; \boldsymbol{\theta}) - f(\mathbf{x})| \leq C(N_\theta)^{-4},$$

其中 C 是依赖于网络参数数目的常数.

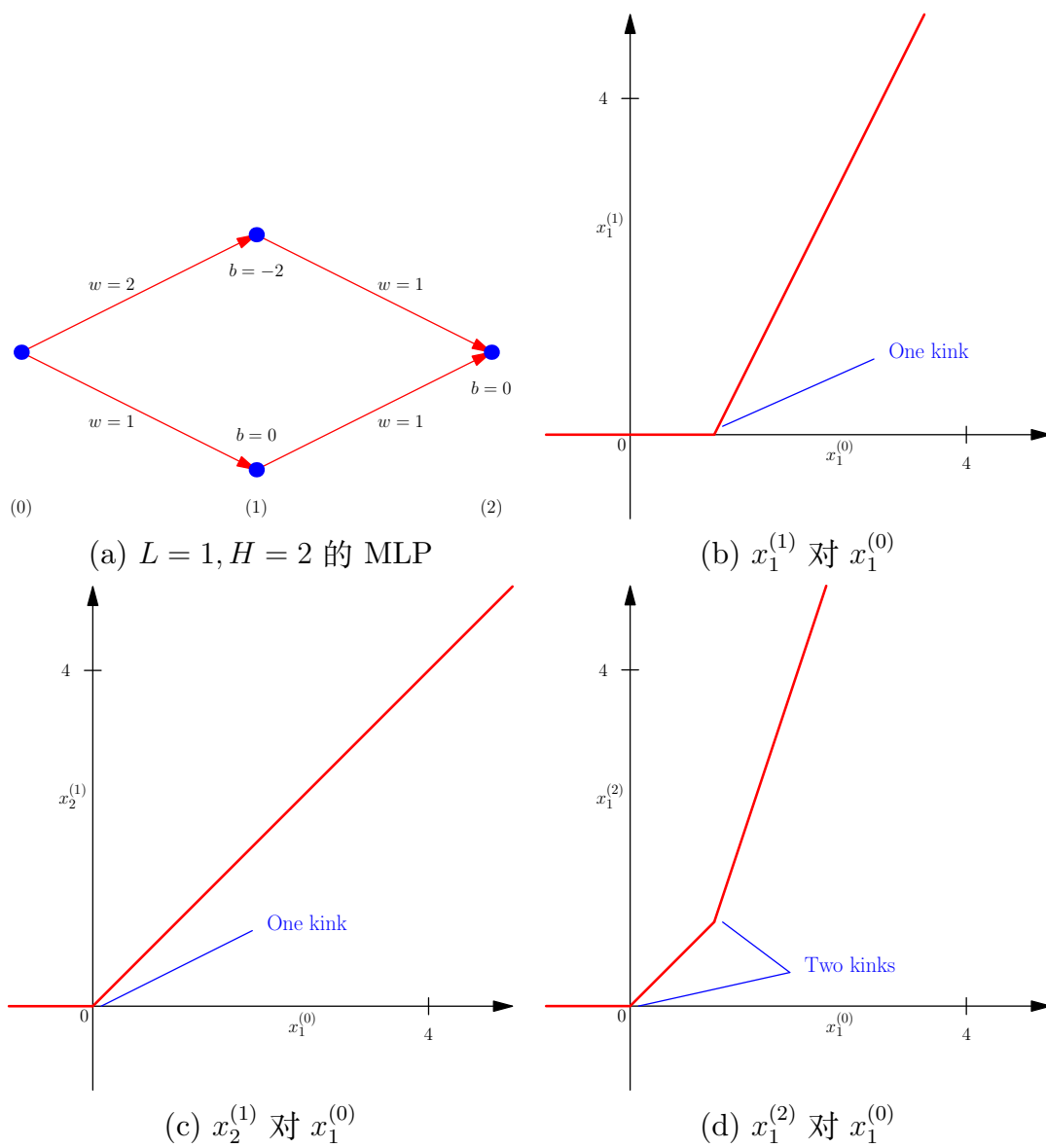


图 2.3: 用于理解神经网络表达能力的示例.

上述数值结果有助于消解神经网络的“黑箱”性质，并可在设计网络架构时提供有用的实践准则。

2.4 神经网络的训练、验证与测试

现在我们已较好地理解了 MLP 的架构，接下来讨论如何设定网络参数以近似某个目标函数。这里将讨论限制在监督学习框架内。

假设给定与目标函数 $f: \mathbf{x} \mapsto \mathbf{y}$ 对应的成对样本数据集 $\mathcal{S} = \{(\mathbf{x}_i, \mathbf{y}_i) : 1 \leq i \leq N\}$ 。我们希望使用神经网络

$$\mathcal{F}(\mathbf{x}; \boldsymbol{\theta}, \mathcal{H})$$

近似该函数，其中 $\boldsymbol{\theta}$ 是前面定义的网络参数，而 $\mathcal{H}$ 表示网络的超参数，例如深度 $L+1$ 、宽度 H 、激活函数 σ 的类型等。设计一个稳健网络的策略包含三个步骤：

1. 在训练 (training) 阶段，对固定的 $\mathcal{H}$ 求 $\boldsymbol{\theta}$ 的最优值。
2. 在验证 (validation) 阶段，求 $\mathcal{H}$ 的最优值。
3. 在测试 (testing) 阶段，用未见过的数据检验网络性能。

为完成这三项任务，通常先把数据集 $\mathcal{S}$ 分成互不相同的三部分：含 N_{train} 个样本的训练集 (training set)、含 N_{val} 个样本的验证集 (validation set) 和含 N_{test} 个样本的测试集 (test set)，其中 $N = N_{\text{train}} + N_{\text{val}} + N_{\text{test}}$ 。一般将约 60% 的样本用于训练，20% 用于验证，其余 20% 用于测试。

之所以必须划分数据集，是因为神经网络是高度过参数化的函数。用于建模数据的大量自由度可能导致对数据的过拟合 (over-fitting)。当数据中的误差或噪声比潜在的输入-输出关系本身更强烈地支配网络行为时，就会发生过拟合。因此，一部分数据用于确定 $\boldsymbol{\theta}$ ，另一部分用于确定超参数 $\mathcal{H}$ 。剩余数据保留给测试，以评估训练后的网络在未见数据上的性能，即网络良好泛化 (generalize) 的能力。

下面更详细地讨论这三个阶段如何使用上述数据划分。

网络训练使用训练集 $\mathcal{S}_{\text{train}}$ 求解如下优化问题：对固定的 $\mathcal{H}$ ，求

$$\begin{aligned} \boldsymbol{\theta}^* &= \arg \min_{\boldsymbol{\theta}} \Pi_{\text{train}}(\boldsymbol{\theta}), \\ \Pi_{\text{train}}(\boldsymbol{\theta}) &= \frac{1}{N_{\text{train}}} \sum_{\substack{i=1 \\ (\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{S}_{\text{train}}}}^{N_{\text{train}}} \|\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta}, \mathcal{H})\|^2. \end{aligned}$$

最优参数 $\boldsymbol{\theta}^*$ 用合适的基于梯度的算法求得 (后文讨论)。函数 Π_{train} 称为损失函数 (loss function)。上例使用的是均方损失函数，后面还会考虑其他类型的损失函数。

网络验证使用验证集 $\mathcal{S}_{\text{val}}$ 求解如下优化问题：求

$$\mathcal{H}^* = \arg \min_{\mathcal{H}} \Pi_{\text{val}}(\mathcal{H}),$$

$$\Pi_{\text{val}}(\mathcal{H}) = \frac{1}{N_{\text{val}}} \sum_{\substack{i=1 \\ (\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{S}_{\text{val}}}}^{N_{\text{val}}} \|\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta}^*, \mathcal{H})\|^2.$$

最优的 $\mathcal{H}^*$ 可用随机网格搜索或张量网格搜索等技术得到。

得到由 $\boldsymbol{\theta}^*$ 和 $\mathcal{H}^*$ 表征的“最佳”网络后，在测试集 $\mathcal{S}_{\text{test}}$ 上评估它，以估计网络在前两个阶段未使用的数据上的性能：

$$\Pi_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{\substack{i=1 \\ (\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{S}_{\text{test}}}}^{N_{\text{test}}} \|\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta}^*, \mathcal{H}^*)\|^2.$$

这一测试误差也称为网络的（近似）泛化误差（generalizing error）。

下面用一个例子进一步说明如何得到这样的网络。

例 2.4.1： 考虑一个 MLP，除以下选择外，其他所有超参数均已固定：

$$\sigma \in \{\text{ReLU}, \tanh\}, \quad L \in \{10, 20\}.$$

采用如下算法：

1. 对每一个可能的 (σ, L) 组合：
 - (a) 求 $\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \Pi_{\text{train}}(\boldsymbol{\theta})$ ；
 - (b) 使用该 $\boldsymbol{\theta}^*$ 计算 $\Pi_{\text{val}}(\mathcal{H})$ 。
2. 选择使 $\Pi_{\text{val}}(\mathcal{H})$ 最小的 $\mathcal{H}$ 作为 $\mathcal{H}^*$ 。
3. 最后，报告该 $\mathcal{H}^*$ 及相应 $\boldsymbol{\theta}^*$ 下的 Π_{test} 。

◇

2.5 泛化能力

如果训练所得网络的 Π_{train} 和 Π_{val} 都很小，能否保证 Π_{test} 也很小？这个问题通过研究已训练网络的泛化能力（generalizability）来回答，即网络在训练和验证期间未见数据上表现良好的能力。如果把网络训练得对训练数据过拟合，网络通常会在测试数据上给出较差预测。一般而言，如果 $\mathcal{S}_{\text{train}}$ 、 $\mathcal{S}_{\text{val}}$ 和 $\mathcal{S}_{\text{test}}$ 取自同一数据分布，那么较小的 Π_{train} 和 Π_{val} 可以带来较小的 Π_{test} 。下面考察一种常用的防止数据过拟合的技术，称为正则化（regularization）。

2.5.1 正则化

神经网络，尤其是 MLP，几乎总是过参数化（over-parametrized）的，即 $N_\theta \gg N$ ，其中 N 是训练样本数。这会产生高度非线性的网络模型，其损失函数 $\Pi(\theta)$ （为简洁起见省略下标“train”）的地形可能含有许多局部极小值（见图 2.4(a)）。那么，应如何确定哪个极小值具有更好的泛化性能？可以采用正则化技术，把 θ^* 的选择推向更有利的方向。

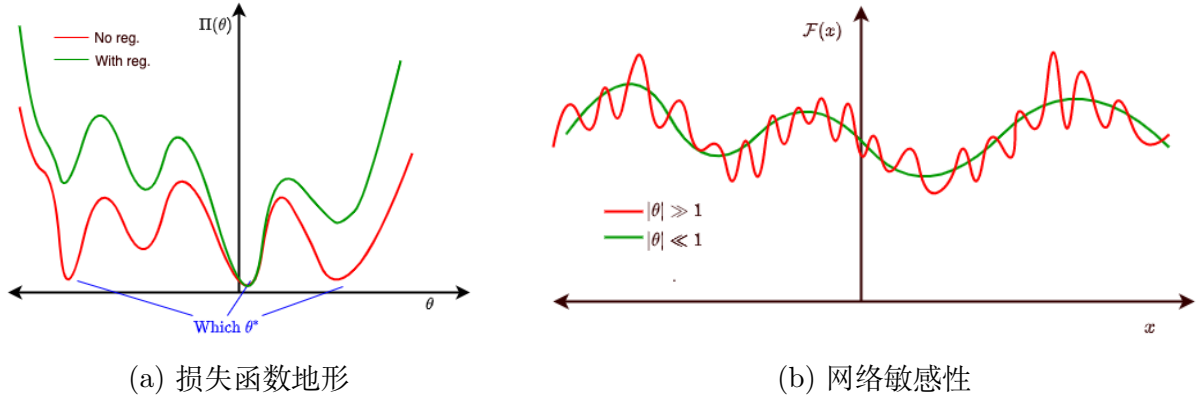


图 2.4: 正则化对损失函数的影响. 为便于说明，假设 θ 为标量。

最简单的正则化方法是在损失函数上增加一个惩罚项：

$$\Pi(\theta) \longrightarrow \Pi(\theta) + \alpha \|\theta\|, \quad \alpha \geq 0,$$

其中 α 是正则化超参数， $\|\theta\|$ 是网络参数 θ 的某个合适范数。如图 2.4(a) 所示，这一增广会改变 $\Pi(\theta)$ 的地形。换言之，这种正则化鼓励选择参数 θ 较小的极小值。

参数 θ 较小为何更好并不显然。为说明这一点，考虑中间网络输出

$$x_1^{(1)} = \sigma\left(W_{1j}^{(1)}x_j^{(0)} + b_1^{(1)}\right),$$

由此得到

$$\frac{\partial x_1^{(1)}}{\partial x_1^{(0)}} = \sigma'\left(W_{1j}^{(1)}x_j^{(0)} + b_1^{(1)}\right) W_{11}^{(1)} \propto W_{11}^{(1)}.$$

由于该导数随 $W_{11}^{(1)}$ 缩放， $\left|\partial \mathcal{F}(x)/\partial x_1^{(0)}\right|$ 也随 $W_{11}^{(1)}$ 缩放。若 $\left|W_{11}^{(1)}\right| \gg 1$ ，网络会对输入 $x_1^{(0)}$ 的微小变化极其敏感，即网络会成为不适定的。如图 2.4(b) 所示，采用适当正则化有助于避免过拟合。

常见的正则化类型包括：

- **ℓ_2 正则化**：在正则项中使用 ℓ_2 范数，

$$\|\theta\| = \|\theta\|_2 = \left(\sum_{i=1}^{N_\theta} \theta_i^2\right)^{1/2}.$$

- **ℓ_1 正则化**: 在正则项中使用 ℓ_1 范数,

$$\|\boldsymbol{\theta}\| = \|\boldsymbol{\theta}\|_1 = \sum_{i=1}^{N_\theta} |\theta_i|,$$

它会促进 $\boldsymbol{\theta}$ 的稀疏性.

2.6 梯度下降

回顾训练阶段所要求解的最小化问题 $\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \Pi(\boldsymbol{\theta})$. 该问题可用梯度下降 (gradient descent, GD) (又称最速下降) 求解. 考虑 $\boldsymbol{\theta}_0$ 附近的泰勒展开

$$\Pi(\boldsymbol{\theta}_0 + \Delta\boldsymbol{\theta}) = \Pi(\boldsymbol{\theta}_0) + \frac{\partial \Pi}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}_0) \cdot \Delta\boldsymbol{\theta} + \frac{\partial^2 \Pi}{\partial \theta_i \partial \theta_j}(\hat{\boldsymbol{\theta}}) \Delta\theta_i \Delta\theta_j,$$

其中 $\hat{\boldsymbol{\theta}}$ 位于 $\boldsymbol{\theta}_0$ 的一个小邻域内. 当 $|\Delta\boldsymbol{\theta}|$ 很小, 并假设二阶导数有界时, 可忽略二阶项, 得到近似

$$\Pi(\boldsymbol{\theta}_0 + \Delta\boldsymbol{\theta}) \approx \Pi(\boldsymbol{\theta}_0) + \frac{\partial \Pi}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}_0) \cdot \Delta\boldsymbol{\theta}.$$

为使损失函数相对于 $\boldsymbol{\theta}_0$ 处的值下降得尽可能多, 即最小化 $\Delta\Pi = \Pi(\boldsymbol{\theta}_0 + \Delta\boldsymbol{\theta}) - \Pi(\boldsymbol{\theta}_0)$, 应令步长方向与梯度相反:

$$\Delta\boldsymbol{\theta} = -\eta \frac{\partial \Pi}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}_0),$$

其中 $\eta \geq 0$ 是步长, 也称学习率 (learning rate). 它是另一个需要在验证阶段调节的超参数. 这正是 GD 算法的核心, 可总结如下:

1. 初始化 $k = 0$ 和 $\boldsymbol{\theta}_0$.
2. 当 $|\Pi(\boldsymbol{\theta}_k)| > \epsilon_1$ 时, 重复:
 - (a) 计算 $\partial \Pi / \partial \boldsymbol{\theta}(\boldsymbol{\theta}_k)$;
 - (b) 更新 $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \partial \Pi / \partial \boldsymbol{\theta}(\boldsymbol{\theta}_k)$;
 - (c) 令 $k = k + 1$.

收敛性: 假设 $\Pi(\boldsymbol{\theta})$ 凸且可微, 其梯度以 $\mathcal{K}$ 为 Lipschitz 常数连续. 则当 $\eta \leq 1/\mathcal{K}$ 时, GD 迭代按

$$\|\boldsymbol{\theta}^* - \boldsymbol{\theta}_k\|_2 \leq \frac{C}{k}$$

收敛.

然而, 在多数情形下 $\Pi(\boldsymbol{\theta})$ 并不凸. 若存在多个极小值, GD 倾向于选择哪一种? 考虑图 2.5 中标量 θ 的损失函数, 它有两个谷地. 假设每个谷地中的 $\Pi(\theta)$ 都可用中心化抛物线

$$\Pi(\theta) \approx \frac{1}{2}a\theta^2$$

近似, 其中 $a > 0$ 是谷地曲率. 左侧谷地的曲率远小于右侧. 固定学习率 η , 并在任一谷地中选择初值 θ_0 . 此时

$$\frac{\partial \Pi}{\partial \theta}(\theta_0) = a\theta_0,$$

一次 GD 更新后得到 $\theta_1 = \theta_0(1 - a\eta)$, 类似地, 后续迭代满足 $\theta_{k+1} = \theta_k(1 - a\eta)$. 收敛要求

$$\left| \frac{\theta_{k+1}}{\theta_k} \right| < 1 \implies |1 - a\eta| < 1,$$

又因谷地中 $a > 0$, 学习率必须满足

$$-1 < 1 - a\eta \implies a\eta < 2.$$

固定 η 后, 局部曲率需满足 $a < 2/\eta$. 换言之, GD 更倾向于收敛到平坦、曲率小的极小值, 即图中的左侧极小值. 若初值位于右侧谷地, 迭代可能不断越过右侧极小值, 在两侧谷壁间反弹, 直至把 θ_k 甩出该谷地; 之后它进入曲率较小的左侧谷地, 并逐渐趋近其极小值.

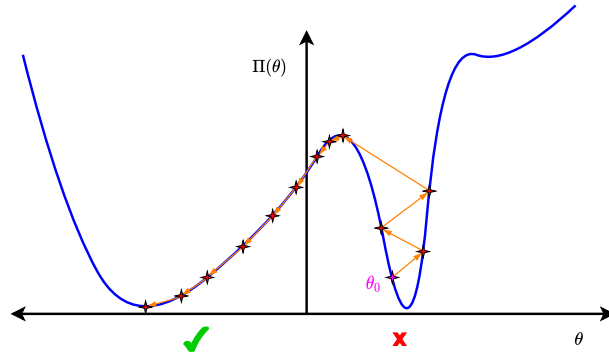


图 2.5: GD 偏好较平坦的极小值.

虽然 GD 偏好平坦极小值这一点很清楚, 但平坦极小值为何更好并不明显. 经验证据表明, 在平坦极小值处得到的参数往往具有更好的泛化性能, 因而更值得选择.

2.7 若干高级优化算法

前面讨论了如何使用 GD 求解神经网络训练中的优化问题. 下面考察几种由 GD 启发的常用高级优化技术. 一般而言, 多数优化算法使用如下更新公式:

$$[\theta_{k+1}]_i = [\theta_k]_i - [\eta_k]_i [g_k]_i, \quad 1 \leq i \leq N_\theta, \quad (2.8)$$

其中 $[\eta_k]_i$ 是逐分量学习率, 向量值函数 g_k 依赖于或近似梯度. 记号 $[\cdot]_i$ 表示向量的第 i 个分量; 学习率也允许依赖迭代次数 k . GD 使用

$$[\eta_k]_i = \eta, \quad g_k = \frac{\partial \Pi}{\partial \theta}(\theta_k).$$

若 η 选择不当, GD 向极小值的收敛可能很慢. 例如, 图 2.6 的目标函数地形沿 $[\theta]_2$ 方向的梯度比沿 $[\theta]_1$ 方向更陡. 从图示点出发, 若 η 过大 (但仍在稳定范围内), 更新将以之字形路径趋向极小值. 理想情况下, 应沿 $[\theta]_1$ 方向迈更大的步子、沿 $[\theta]_2$ 方向迈更小的步子, 从而更快到达极小值.

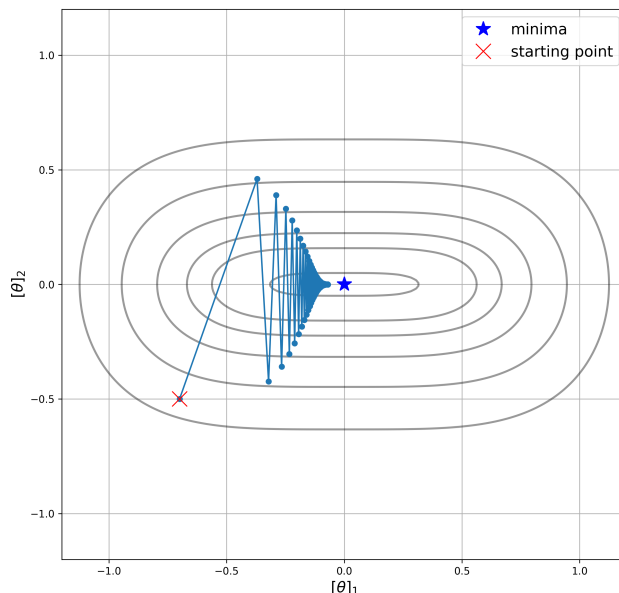


图 2.6: GD 的之字形更新.

下面考察两种能克服 GD 部分问题的常用方法.

2.7.1 动量方法

动量方法 (momentum method) 不仅使用前一步梯度, 还利用梯度的历史. 更新公式为

$$[\eta_k]_i = \eta, \quad \mathbf{g}_k = \beta_1 \mathbf{g}_{k-1} + (1 - \beta_1) \frac{\partial \Pi}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}_k), \quad \mathbf{g}_{-1} = 0,$$

其中 $\mathbf{g}_k$ 是梯度的加权移动平均. 这种加权可抵消梯度沿 $[\theta]_2$ 方向的分量, 从而平滑图 2.6 中的之字形运动, 更顺畅地趋向极小值. β_1 的常用取值为 0.9.

2.7.2 Adam

Kingma 和 Ba 提出的 Adam 优化算法 (Adam optimization) [10] 同时利用梯度历史和梯度的二阶矩 (衡量梯度大小). 对初始学习率 η , 更新为

$$\begin{aligned} \mathbf{g}_k &= \beta_1 \mathbf{g}_{k-1} + (1 - \beta_1) \frac{\partial \Pi}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}_k), \\ [\mathbf{G}_k]_i &= \beta_2 [\mathbf{G}_{k-1}]_i + (1 - \beta_2) \left(\frac{\partial \Pi}{\partial \theta_i}(\boldsymbol{\theta}_k) \right)^2, \\ [\eta_k]_i &= \frac{\eta}{\sqrt{[\mathbf{G}_k]_i} + \epsilon}. \end{aligned} \tag{2.9}$$

这里， $\mathbf{g}_k$ 和 $\mathbf{G}_k$ 分别为梯度与梯度平方的加权滑动平均。推荐超参数为 $\beta_1 = 0.9$ 、 $\beta_2 = 0.999$ 和 $\epsilon = 10^{-8}$ 。注意，各分量的学习率不同；某个分量的梯度越大，其学习率越小。回到图 2.6 的例子，这意味着 θ_2 的学习率小于 θ_1 ，从而缓解优化路径的之字形运动。

注 2.7.1: Adam 还包含对 $\mathbf{g}_k$ 和 $\mathbf{G}_k$ 的额外校正步骤，以提高算法效率，详见 [10]。◇

2.7.3 随机优化

训练损失可重写为

$$\Pi(\boldsymbol{\theta}) = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \Pi_i(\boldsymbol{\theta}), \quad \Pi_i(\boldsymbol{\theta}) = \|\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta}, \mathcal{H})\|^2,$$

故损失函数的梯度为

$$\frac{\partial \Pi}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}) = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \frac{\partial \Pi_i}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}).$$

但由于 N_{train} 通常很大 ($N_{\text{train}} \sim 10^6$)，求梯度之和可能非常昂贵。一种简单的规避方法是使用如下更新（这里以 GD 为例）：

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \frac{\partial \Pi_i}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}_k), \quad (2.10)$$

其中每个更新步 k 都随机选择 i 。这称为随机梯度下降 (stochastic gradient descent, SGD)。值得注意的是，若 $\Pi_i(\boldsymbol{\theta})$ 凸且可微，并且 $\eta_k \sim 1/\sqrt{k}$ ，则这一修改后的算法确实收敛 [19]。

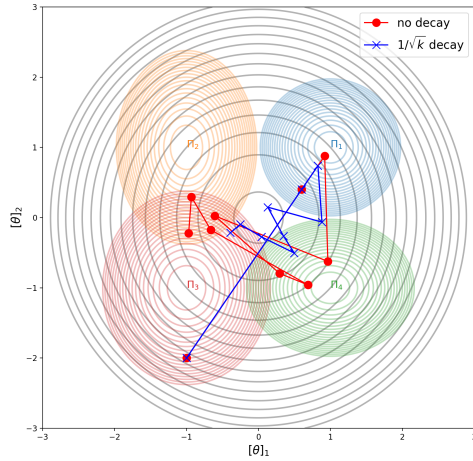
为说明 η_k 为何需要衰减，考虑 $\boldsymbol{\theta} \in \mathbb{R}^2$ 上的玩具函数：

$$\begin{aligned} \Pi_1(\boldsymbol{\theta}) &= ([\theta]_1 - 1)^2 + ([\theta]_2 - 1)^2, & \Pi_2(\boldsymbol{\theta}) &= ([\theta]_1 + 1)^2 + 0.5([\theta]_2 - 1)^2, \\ \Pi_3(\boldsymbol{\theta}) &= 0.7([\theta]_1 + 1)^2 + 0.5([\theta]_2 + 1)^2, & \Pi_4(\boldsymbol{\theta}) &= 0.7([\theta]_1 - 1)^2 + \frac{1}{2}([\theta]_2 + 1)^2, \\ \Pi(\boldsymbol{\theta}) &= \frac{1}{4}(\Pi_1(\boldsymbol{\theta}) + \Pi_2(\boldsymbol{\theta}) + \Pi_3(\boldsymbol{\theta}) + \Pi_4(\boldsymbol{\theta})). \end{aligned} \quad (2.11)$$

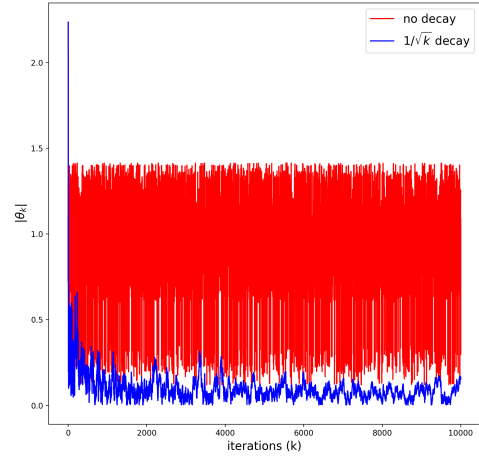
这些函数的等高线见图 2.7(a)，黑色等高线对应 $\Pi(\boldsymbol{\theta})$ 。 $\boldsymbol{\theta}^* = (0, 0)$ 是 Π 的唯一极小值。分别取常学习率 $\eta_k = 0.4$ 和衰减学习率 $\eta_k = 0.4/\sqrt{k}$ ，从 $\boldsymbol{\theta}_0 = (-1.0, 2.0)$ 出发，每一步随机选择 $i \in \{1, 2, 3, 4\}$ ，运行 10,000 次迭代。两种学习率下的前 10 步绘于图 2.7(a)。不衰减时，SGD 会不断越过极小值；如图 2.7(b) 所示，这种行为在之后的迭代中仍会持续，更新范数不衰减（预期它应趋于 $|\boldsymbol{\theta}^*| = 0$ ）。学习率按 $1/\sqrt{k}$ 衰减时，则会很快靠近 $\boldsymbol{\theta}^*$ 。

接近极小值时需要减小步长，是因为远离 Π 的极小值时， Π 的梯度向量与各 Π_i 的梯度向量大致对齐；靠近极小值后则不再如此，因此必须缩小步长，以免被抛到远离极小值的区域。

实践中通常不逐个样本使用随机优化算法，原因如下：



(a) 函数等高线与路径



(b) 更新范数

图 2.7: 学习率衰减与不衰减时的 SGD 算法.

1. 尽管损失随迭代次数下降，但它在极小值附近混乱波动，始终无法到达极小值.
2. 一次处理所有样本可能计算昂贵，而一次只处理一个样本又会严重浪费计算与存储资源.

可采用小批量优化 (mini-batch optimization) 折中. 将含 N_{train} 个样本的数据集划分为 N_{batch} 个互不相交的子集，称为小批量. 每个小批量含 $\bar{N}_{\text{train}} = N_{\text{train}}/N_{\text{batch}}$ 个样本，这一数目也称批量大小. 损失梯度可近似为

$$\frac{\partial \Pi}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}) = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \frac{\partial \Pi_i}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}) \approx \frac{1}{\bar{N}_{\text{train}}} \sum_{i \in \text{batch}(j)} \frac{\partial \Pi_i}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}). \quad (2.12)$$

取 $N_{\text{batch}} = 1$ 得到原始优化算法，取 $N_{\text{batch}} = N_{\text{train}}$ 则得到 SGD. 通常选择能使一次装入内存的数据量最大的批量大小. 定义一个轮次 (epoch) 为对完整训练集中所有样本 (或小批量) 的一次完整遍历. 小批量随机优化算法如下:

1. 对 $\text{epoch} = 1, \dots, J$:
 - (a) 随机打乱完整训练集;
 - (b) 创建 N_{batch} 个小批量;
 - (c) 对 $i = 1, \dots, N_{\text{batch}}$:
 - i. 使用式 (2.12) 计算批量梯度;
 - ii. 使用该梯度和所选优化算法 (梯度下降、动量法或 Adam) 更新 $\boldsymbol{\theta}$.

注 2.7.2: 一项研究 [31] 表明，SGD 或许确实有助于选择泛化性能更好的极小值. 作者证明，SGD 偏好曲率更均匀的极小值，即损失函数各分量的曲率分布尖锐，并集中在一个小值附近；与之相对，有些极小值的总体曲率虽小，各损失分量的曲率分布却更

分散. 他们进一步以经验结果表明, 较均匀的极小值往往比非均匀极小值具有更好的泛化性能. $\diamond$

2.8 使用反向传播计算梯度

训练算法最后需要理解的一环, 是训练网络时如何实际计算梯度. 第 $l+1$ 层的输出 $\mathbf{x}^{(l+1)}$ 为

$$\text{仿射变换: } \xi_i^{(l+1)} = W_{ij}^{(l+1)} x_j^{(l)} + b_i^{(l+1)}, \quad 1 \leq i \leq H_{l+1}, \quad (2.13)$$

$$\text{非线性变换: } x_i^{(l+1)} = \sigma(\xi_i^{(l+1)}), \quad 1 \leq i \leq H_{l+1}. \quad (2.14)$$

给定训练样本 $(\mathbf{x}, \mathbf{y})$, 令 $\mathbf{x}^{(0)} = \mathbf{x}$. 对该样本, 损失或目标函数可通过前向传播 (forward pass) 计算:

1. 对 $l = 1, \dots, L+1$:

(a) 用式 (2.13) 计算 $\boldsymbol{\xi}^{(l)}$;

(b) 用式 (2.14) 计算 $\mathbf{x}^{(l)}$.

2. 计算给定样本的损失函数

$$\Pi(\boldsymbol{\theta}) = \|\mathbf{y} - \mathcal{F}(\mathbf{x}; \boldsymbol{\theta}, \mathcal{H})\|^2.$$

这一运算可简洁地写成图 2.8 所示的计算图; 图的下半部分表示损失函数 Π 的计算. 对训练集中的所有样本都需要重复这一步 (随机优化时则对一个小批量重复). 为简洁起见, 以下只讨论单个样本的损失与梯度.

更新网络参数需要 $\partial\Pi/\partial\boldsymbol{\theta}$, 更具体地说, 需要对每个 $1 \leq l \leq L+1$ 计算 $\partial\Pi/\partial\mathbf{W}^{(l)}$ 和 $\partial\Pi/\partial\mathbf{b}^{(l)}$. 先推导 $\partial\Pi/\partial\boldsymbol{\xi}^{(l)}$ 和 $\partial\Pi/\partial\mathbf{x}^{(l)}$, 再得到这些导数.

从计算图容易看出网络中每个隐藏变量如何变换到下一个变量. 反复应用链式法则得到

$$\frac{\partial\Pi}{\partial\boldsymbol{\xi}^{(l)}} = \frac{\partial\Pi}{\partial\mathbf{x}^{(L+1)}} \frac{\partial\mathbf{x}^{(L+1)}}{\partial\boldsymbol{\xi}^{(L+1)}} \frac{\partial\boldsymbol{\xi}^{(L+1)}}{\partial\mathbf{x}^{(L)}} \cdots \frac{\partial\mathbf{x}^{(l+1)}}{\partial\boldsymbol{\xi}^{(l+1)}} \frac{\partial\boldsymbol{\xi}^{(l+1)}}{\partial\mathbf{x}^{(l)}} \frac{\partial\mathbf{x}^{(l)}}{\partial\boldsymbol{\xi}^{(l)}}. \quad (2.15)$$

计算上式需要以下各项:

$$\frac{\partial\Pi}{\partial\mathbf{x}^{(L+1)}} = -2(\mathbf{y} - \mathbf{x}^{(L+1)})^T, \quad (2.16)$$

$$\frac{\partial\boldsymbol{\xi}^{(l+1)}}{\partial\mathbf{x}^{(l)}} = \mathbf{W}^{(l+1)}, \quad (2.17)$$

$$\frac{\partial\mathbf{x}^{(l)}}{\partial\boldsymbol{\xi}^{(l)}} = \mathbf{S}^{(l)} \equiv \text{diag}[\sigma'(\xi_1^{(l)}), \dots, \sigma'(\xi_{H_l}^{(l)})]. \quad (2.18)$$

后两式对任意网络层 l 成立, H_l 是该层宽度, σ' 表示激活函数关于其自变量的导数. 将这些关系代入式 (2.15), 得到

$$\frac{\partial\Pi}{\partial\boldsymbol{\xi}^{(l)}} = \frac{\partial\Pi}{\partial\mathbf{x}^{(L+1)}} \mathbf{S}^{(L+1)} \mathbf{W}^{(L+1)} \cdots \mathbf{S}^{(l+1)} \mathbf{W}^{(l+1)} \mathbf{S}^{(l)}. \quad (2.19)$$

取转置，并注意 $\mathbf{S}^{(l)}$ 为对角矩阵，因而是对称的，最终得到

$$\frac{\partial \Pi}{\partial \xi^{(l)}} = \mathbf{S}^{(l)} \mathbf{W}^{(l+1)T} \mathbf{S}^{(l+1)} \dots \mathbf{W}^{(L+1)T} \mathbf{S}^{(L+1)} [-2(\mathbf{y} - \mathbf{x}^{(L+1)})]. \quad (2.20)$$

这一计算同样可表示成计算图. 如图 2.8 所示，可将它接到原计算图上，对应图的上半行. 此时沿网络反向遍历，因此称为反向传播 (back-propagation) .

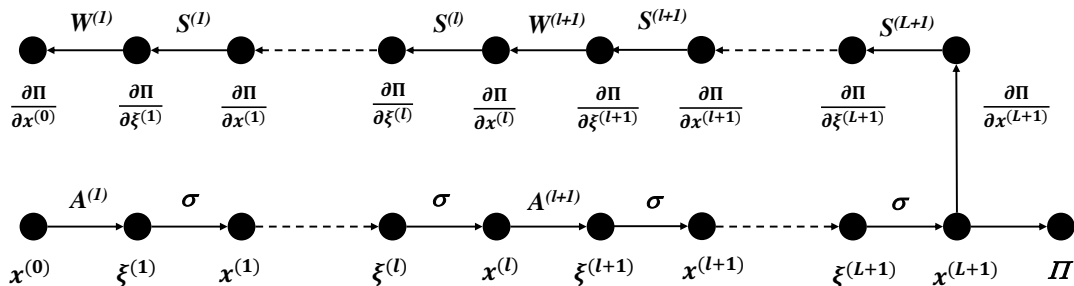


图 2.8: 计算损失函数及其关于隐藏或潜在向量之导数的计算图.

最后一步是得到 $\partial \Pi / \partial \mathbf{W}^{(l)}$ 的显式表达式:

$$\frac{\partial \Pi}{\partial \mathbf{W}^{(l)}} = \frac{\partial \Pi}{\partial \xi^{(l)}} \frac{\partial \xi^{(l)}}{\partial \mathbf{W}^{(l)}} = \frac{\partial \Pi}{\partial \xi^{(l)}} \otimes \mathbf{x}^{(l-1)}, \quad (2.21)$$

其中 $[\mathbf{x} \otimes \mathbf{y}]_{ij} = x_i y_j$ 为外积. 因而，计算 $\partial \Pi / \partial \mathbf{W}^{(l)}$ 需要前向阶段得到的 $\mathbf{x}^{(l-1)}$ ，以及反向传播阶段得到的 $\partial \Pi / \partial \xi^{(l)}$.

问题 2.8.1: 能否推导一组类似的表达式以及相应算法，用于计算 $\partial \Pi / \partial \mathbf{b}^{(l)}$?

问题 2.8.2: 能否推导 $\partial \mathbf{x}^{(L+1)} / \partial \mathbf{x}^{(0)}$ 的显式表达式，即网络输出关于输入的导数？这是一个很有用的量，会用于物理信息神经网络和 Wasserstein 生成对抗网络等算法.

2.9 回归与分类

到目前为止，对于带标签数据集 $\mathcal{S} = \{(\mathbf{x}_i, \mathbf{y}_i) : 1 \leq i \leq N\}$ ，我们考虑了两种损失：

- 均方误差 (MSE),

$$\Pi(\theta) = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \|\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \theta, \mathcal{H})\|^2.$$

- 平均绝对误差 (MAE),

$$\Pi(\theta) = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \|\mathbf{y}_i - \mathcal{F}(\mathbf{x}_i; \theta, \mathcal{H})\|.$$

采用这些损失的神经网络可求解多种回归问题，其中潜在函数高度非线性，输入和输出均可多维.

例 2.9.1: 给定房屋或公寓的特征, 例如邮政编码、卧室和卫生间数目、室内面积、建造年限等, 预测市场售价或在市场上挂牌的天数等结果. $\diamond$

下面考虑分类问题的若干例子, 其中网络输出通常属于一个有限离散集.

例 2.9.2: 给定 COVID-19 患者的症状与血液指标, 预测其是否需要收入 ICU. 该问题的输入和输出为

$$\begin{aligned}\mathbf{x} &= [\text{脉率}, \text{体温}, \text{SPO}_2, \text{降钙素原}, \dots], \\ \mathbf{y} &= [p_1, p_2],\end{aligned}$$

其中 p_1 是收入 ICU 的概率, p_2 是不收入 ICU 的概率. 注意 $0 \leq p_1, p_2 \leq 1$ 且 $p_1 + p_2 = 1$. $\diamond$

例 2.9.3: 给定一组动物图像, 预测图中动物是狗、猫还是鸟. 此时输入和输出应为

$$\begin{aligned}\mathbf{x} &= \text{图像}, \\ \mathbf{y} &= [p_1, p_2, p_3],\end{aligned}$$

其中 p_1, p_2, p_3 分别是狗、猫或鸟的概率. $\diamond$

分类问题的输出对应概率, 因此需要对网络作如下修改. 其中将使用 softmax 函数 (softmax function) .

1. 在输出层末端使用输出函数, 把输出向量适当地变换为概率向量. 通常使用 softmax 函数:

$$x_i^{(L+1)} = \frac{\exp(\xi_i^{(L+1)})}{\sum_{j=1}^C \exp(\xi_j^{(L+1)})},$$

其中 C 是类别数, 也是输出维数. 验证这一变换使 $\mathbf{x}^{(L+1)}$ 的分量构成凸组合, 即 $x_i^{(L+1)} \in [0, 1]$ 且 $\sum_{i=1}^C x_i^{(L+1)} = 1$.

2. 各样本的输出标签需要作独热编码 (one-hot encoding) . 对样本 $(\mathbf{x}, \mathbf{y})$, 输出标签 $\mathbf{y}$ 的维数应为 $D = C$; 仅表示 $\mathbf{x}$ 所属类别的那个分量为 1, 其余为 0. 例如, 在例 2.9.3 中, 源文给出

$$\mathbf{y} = \begin{cases} [1, 0, 0]^T, & \mathbf{x} \text{ 是狗}, \\ [0, 1, 0]^T, & \mathbf{x} \text{ 是猫}, \\ [0, 0, 1]^T, & \mathbf{x} \text{ 是猪}. \end{cases}$$

3. 尽管仍可使用 MSE 或源文所称的 MSA 作为损失函数, 但更适合采用交叉熵 (cross-entropy) 损失:

$$\Pi(\boldsymbol{\theta}) = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \sum_{c=1}^C -y_{ci} \log(\mathcal{F}_c(\mathbf{x}_i; \boldsymbol{\theta})), \quad (2.22)$$

其中 y_{ci} 是第 i 个样本真实标签的第 c 个分量. 式 (2.22) 把 y_c 和 $\mathcal{F}_c$ 视为概率分布, 并衡量二者差异. 可以证明, 它与两个分布之间的 Kullback-Leibler 散度有关. 与 MSE 相比, 该损失会严厉惩罚置信度很高却错误的预测, 见例 2.9.4.

例 2.9.4: 考虑二分类问题, 即 $C = 2$. 对给定的 $\mathbf{x}$, 令 $\mathbf{y} = [0, 1]$, 预测为 $\mathcal{F} = [p, 1-p]$. 显然, 较小的 p 更好, 因此任何合理代价函数都应惩罚较大的 p . 两种损失给出的误差为:

- MSE 损失 $= (0 - p)^2 + (1 - 1 + p)^2 = 2p^2$;
- 交叉熵损失 $= -(0 \log p + 1 \log(1 - p)) = -\log(1 - p)$.

两种损失都会惩罚较大的 p , 且 $p = 0$ 时均为零. 然而, 当 $p \rightarrow 1$ (导致错误预测) 时, MSE 损失趋于 2, 而交叉熵损失趋于 ∞ . 因而, 交叉熵会强烈惩罚置信度高的错误预测. $\diamond$

残差神经网络

He 等人于 2015 年提出了残差神经网络 (ResNet) (residual neural network, ResNet) [8]. 本章将讨论这类网络是什么、为何提出它们, 以及它们与常微分方程 (ODE) 的关系.

3.1 深度网络中的梯度消失

训练神经网络时, 梯度 $\partial\Pi/\partial\mathbf{W}^{(l)}$ 和 $\partial\Pi/\partial\mathbf{b}^{(l)}$ 可能变得非常小. 例如, 考虑一个很深的网络, 设 $L \geq 20$. 如果对 $l \leq \bar{l}$ 有 $|\partial\Pi/\partial\mathbf{W}^{(l)}| \ll 1$, 那么网络前 $\bar{l}$ 层的贡献将可以忽略, 因为这些层的权重对损失函数影响很小. 这种深度截断会使深度网络在表达能力上的优势丧失.

为什么会出现这种情况? 回顾节 2.8, 有

$$\frac{\partial\Pi}{\partial\mathbf{W}^{(l)}} = \frac{\partial\Pi}{\partial\boldsymbol{\xi}^{(l)}} \otimes \mathbf{x}^{(l-1)},$$

以及

$$\frac{\partial\Pi}{\partial\boldsymbol{\xi}^{(l)}} = \boldsymbol{\Sigma}^{(l)} \prod_{m=l+1}^{L+1} \left(\mathbf{W}^{(m)T} \boldsymbol{\Sigma}^{(m)} \right) \frac{\partial\Pi}{\partial\boldsymbol{\xi}^{(L+1)}}. \quad (3.1)$$

对任意矩阵 $\mathbf{A}$, 以 $\tau(\mathbf{A})$ 表示其最大奇异值 (largest singular value). 因而可将 $|\partial\Pi/\partial\boldsymbol{\xi}^{(l)}|$ 估计为

$$\left| \frac{\partial\Pi}{\partial\boldsymbol{\xi}^{(l)}} \right| \leq \tau(\boldsymbol{\Sigma}^{(l)}) \prod_{m=l+1}^{L+1} \left(\tau(\mathbf{W}^{(m)}) \tau(\boldsymbol{\Sigma}^{(m)}) \right) \left| \frac{\partial\Pi}{\partial\boldsymbol{\xi}^{(L+1)}} \right|. \quad (3.2)$$

回顾 $\boldsymbol{\Sigma}^{(m)} \equiv \text{diag}[\sigma'(\xi_1^{(m)}), \dots, \sigma'(\xi_{H_l}^{(m)})]$, 其中 σ' 表示 σ 对其自变量的导数. 对 ReLU 而言, 其值为 0 或 1, 因此源文取 $\tau(\boldsymbol{\Sigma}^{(m)}) = 1$.

此外, 为了稳定性, 我们希望 $\tau(\mathbf{W}^{(m)}) < 1$, 否则网络输出可能变得无界. 实践中, 这一要求由正则化项施加. 将其代入上式可得

$$\left| \frac{\partial\Pi}{\partial\boldsymbol{\xi}^{(l)}} \right| \leq \prod_{m=l+1}^{L+1} \tau(\mathbf{W}^{(m)}) \left| \frac{\partial\Pi}{\partial\boldsymbol{\xi}^{(L+1)}} \right|, \quad (3.3)$$

其中乘积中的每一项都是小于 1 的标量. 当因子数增加, 即 $L - l \gg 1$ 时, 该乘积可能而且确实会变得非常小. 这种情况通常在 $L - l \approx 20$ 时发生, 此时 $|\partial\Pi/\partial\boldsymbol{\xi}^{(l)}|$, 进而 $|\partial\Pi/\partial\mathbf{W}^{(l)}|$, 都会变得非常小. 这一问题称为梯度消失 (vanishing gradient) 问题. 它出现在深度网络中, 使内部各层 (例如 $L - l > 20$) 的权重不再对网络作出贡献.

He 等人 [8] 展示了采用更深的网络实际上可能导致训练误差和验证误差增大（见图 3.1）。因而，超过某一点后，继续增加网络深度可能适得其反。根据前面对梯度消失的讨论，我们已经知道其中原因。因此，希望设计一种网络架构，通过确保 $|\partial\Pi/\partial\boldsymbol{\xi}^{(L+1)}| \approx |\partial\Pi/\partial\boldsymbol{\xi}^{(1)}|$ 来处理梯度消失问题。这意味着，当网络权重趋于较小值时，网络应趋于恒等映射（identity mapping）而非零映射（null mapping）。这正是 ResNet 架构的核心思想。

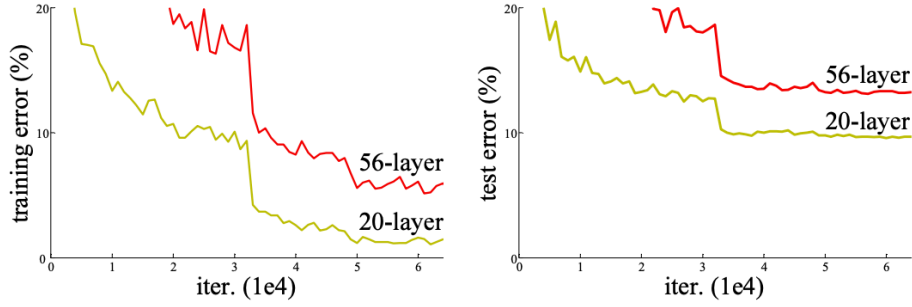


图 3.1: CIFAR-10 数据上“普通”深度网络的训练误差（左）和测试误差（右），引自文献 [8]。

3.2 ResNet

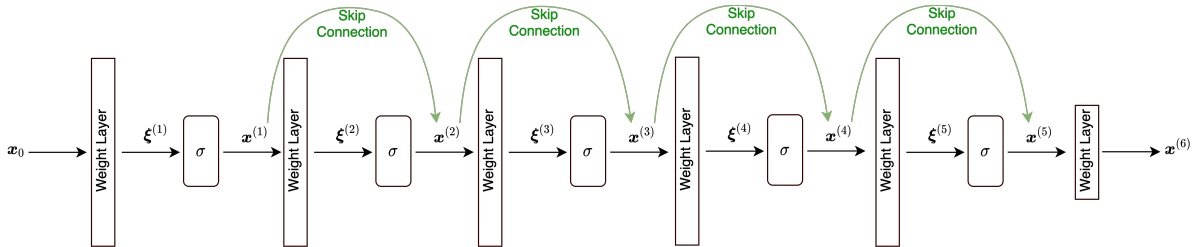


图 3.2: 具有跳跃连接的深度为 6 的 ResNet。

考虑一个深度为 6、每个隐藏层宽度固定为 H 的 MLP（见图 3.2）。按照下式在隐藏层之间加入跳跃连接（skip connection）：

$$x_i^{(l)} = \sigma\left(W_{ij}^{(l)} x_j^{(l-1)} + b_i^{(l)}\right) + x_i^{(l-1)}, \quad 2 \leq l \leq L. \quad (3.4)$$

可以作如下观察：

1. 如果所有权重（和偏置）均为零，则 $\mathbf{x}^{(5)} = \mathbf{x}^{(1)}$ ，进而有

$$\frac{\partial\Pi}{\partial\mathbf{x}^{(1)}} = \frac{\partial\Pi}{\partial\mathbf{x}^{(5)}}.$$

换言之，此时不会出现梯度消失问题。

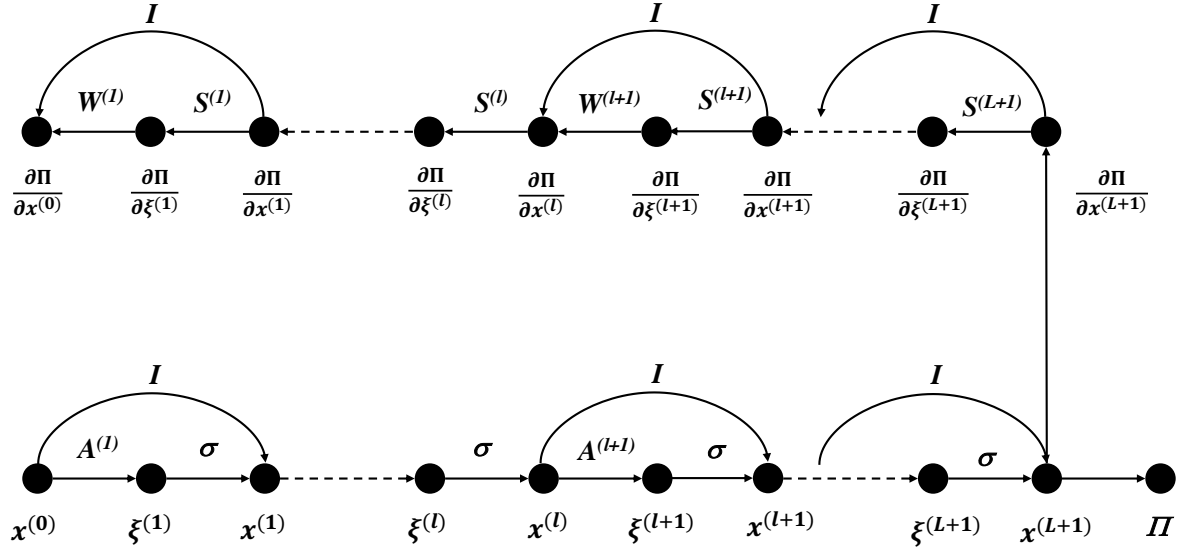


图 3.3: ResNet 前向传播和反向传播的计算图.

2. ResNet 前向传播和反向传播的计算图见图 3.3. 从该图可清楚看出, $\partial \mathbf{x}^{(l+1)} / \partial \mathbf{x}^{(l)}$ 的表达式现在涉及沿两条分支传播并将结果相加. 因此有

$$\frac{\partial \Pi}{\partial \xi^{(l)}} = \Sigma^{(l)} \prod_{m=l+1}^{L+1} \left(I + \mathbf{W}^{(m)T} \Sigma^{(m)} \right) \frac{\partial \Pi}{\partial \xi^{(L+1)}}. \quad (3.5)$$

如果进一步假设正则化使 $|\mathbf{W}^{(m)}| \ll 1$, 则

$$\frac{\partial \Pi}{\partial \xi^{(l)}} = \Sigma^{(l)} \left(I + \sum_{m=l+1}^{L+1} \mathbf{W}^{(m)T} \Sigma^{(m)} + \text{高阶项} \right) \frac{\partial \Pi}{\partial \xi^{(L+1)}}. \quad (3.6)$$

在上式中, 即使各矩阵的元素都很小, 它们的和也未必趋于零矩阵. 这意味着, 即使仍通过正则化要求权重较小, 也可以使靠近输入层与输出层的梯度之间产生有限且显著的变化.

注 3.2.1: 上述分析可以推广到 H 不固定的情形, 但分析不再这样简洁. 关于具体做法, 见文献 [8]. $\diamond$

3.3 与 ODE 的联系

先考虑 $d = D = H$ 的 ResNet 特例. 回顾关系式 (3.4), 可将其改写为

$$\frac{\mathbf{x}^{(l)} - \mathbf{x}^{(l-1)}}{\Delta t} = \frac{1}{\Delta t} \sigma \left(\mathbf{W}^{(l)} \mathbf{x}^{(l-1)} + \mathbf{b}^{(l)} \right) = \frac{1}{\Delta t} \sigma \left(\xi^{(l)} \right), \quad (3.7)$$

其中 Δt 为某个标量, 并注意 $\xi^{(l)}$ 是 $\mathbf{x}^{(l-1)}$ 的函数, 由 $\theta^{(l)} = [\mathbf{W}^{(l)}, \mathbf{b}^{(l)}]$ 参数化. 因而, 式 (3.7) 还可写成

$$\frac{\mathbf{x}^{(l)} - \mathbf{x}^{(l-1)}}{\Delta t} = V \left(\mathbf{x}^{(l-1)}; \theta^{(l)} \right). \quad (3.8)$$

现在考虑一个可能非线性的一阶 ODE 系统. 已知 $\mathbf{x}(0)$ 和

$$\dot{\mathbf{x}} \equiv \frac{d\mathbf{x}}{dt} = \mathbf{V}(\mathbf{x}, t), \quad (3.9)$$

我们希望求出 $\mathbf{x}(T)$. 为进行数值求解, 可用时间步长 Δt 和时间节点 $t^{(l)} = l\Delta t$ ($0 \leq l \leq L+1$) 均匀划分时间区间, 其中 $(L+1)\Delta t = T$. 将离散解定义为 $\mathbf{x}^{(l)} = \mathbf{x}(l\Delta t)$. 给定 $\mathbf{x}^{(l-1)}$ 后, 可以用时间积分器 (time integrator) 近似解 $\mathbf{x}^{(l)}$. 考虑一种受向前欧拉法 (forward Euler method) 启发的方法, 其中式 (3.9) 的左端近似为

$$\text{左端} \approx \frac{\mathbf{x}^{(l)} - \mathbf{x}^{(l-1)}}{\Delta t},$$

而右端利用参数 $\boldsymbol{\theta}^{(l)}$ 近似为

$$\text{右端} \approx \mathbf{V}(\mathbf{x}^{(l-1)}; t^{(l)}) = \mathbf{V}(\mathbf{x}^{(l-1)}; \boldsymbol{\theta}^{(l)}),$$

这里允许每个时间步采用不同参数. 将两式合并, 恰好得到式 (3.8) 所给的 ResNet 关系. 换言之, ResNet 无非是非线性 ODE 系统的一种离散化. 下面作几点说明, 以进一步强化这种联系:

- 对一个训练完毕的 ResNet, 给定 $\mathbf{x}^{(0)}$ 和网络权重, 预测 $\mathbf{x}^{(L+1)}$.
- 对一个 ODE 系统, 给定 $\mathbf{x}(0)$ 和 $\mathbf{V}(\mathbf{x}, t)$, 预测 $\mathbf{x}(T)$.
- 训练 ResNet 是指确定网络参数 $\boldsymbol{\theta}$, 使得对 $i = 1, \dots, N_{\text{train}}$, 当 $\mathbf{x}^{(0)} = \mathbf{x}_i$ 时, $\mathbf{x}^{(L+1)}$ 尽可能接近 $\mathbf{y}_i$.
- 从类比的 ODE 观点看, 训练是指确定右端 $\mathbf{V}(\mathbf{x}, t)$, 要求对 $i = 1, \dots, N_{\text{train}}$, 当 $\mathbf{x}(0) = \mathbf{x}_i$ 时, $\mathbf{x}(T)$ 尽可能接近 $\mathbf{y}_i$.
- 在 ResNet 中, 我们寻找“同一个” $\mathbf{V}(\mathbf{x}, t)$, 使它对所有 $1 \leq i \leq N_{\text{train}}$ 都能将 $\mathbf{x}_i$ 映射到 $\mathbf{y}_i$.

3.4 神经 ODE

受 ResNet 与 ODE 之间联系的启发, 文献 [4] 提出了神经 ODE (neural ODE). 考虑 ODE 系统

$$\frac{d\mathbf{x}}{dt} = \mathbf{V}(\mathbf{x}, t). \quad (3.10)$$

给定 $\mathbf{x}(0)$, 希望求出 $\mathbf{x}(T)$. 在文献 [4] 中, 右端 $\mathbf{V}(\mathbf{x}, t)$ 由参数为 $\boldsymbol{\theta}$ 的前馈神经网络定义 (见图 3.4). 网络输入为 $(\mathbf{x}, t)$, 输出为与 $\mathbf{x}$ 维数相同的 $\mathbf{V}(\mathbf{x}, t)$. 按照这一描述, 可使用适当的时间推进格式 (time-marching scheme) 求解系统 (3.10), 例如向前欧拉法、Runge-Kutta 方法等.

那么, 如何使用该网络求解回归问题? 假设给定带标签的训练数据 $\mathcal{S} = \{(\mathbf{x}_i, \mathbf{y}_i) : 1 \leq i \leq N_{\text{train}}\}$, 并假设 $\mathbf{x}_i$ 和 $\mathbf{y}_i$ 都具有相同的维数 $d-1$. 核心思想是把 $\mathbf{x}_i$ 看作

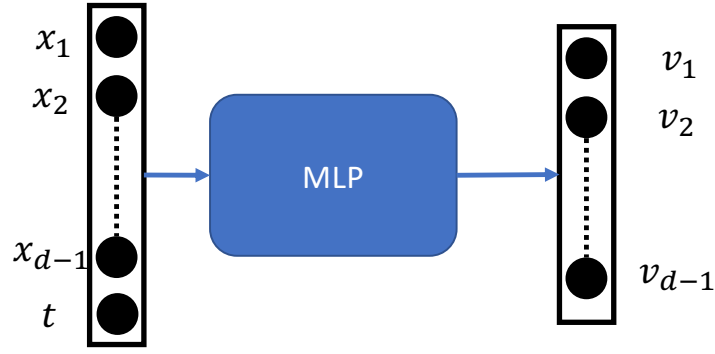


图 3.4: 用于表示神经 ODE 右端的前馈神经网络. 因变量数目为 $d - 1$.

$(d - 1)$ 维空间中表示系统初态的点, 把 $\mathbf{y}_i$ 看作表示终态的点. 回归问题于是转化为寻找式 (3.10) 的右端, 使初态点以最小误差映射到终态点. 换言之, 寻找参数 $\boldsymbol{\theta}$, 使

$$\Pi(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N |\mathbf{x}_i(T; \boldsymbol{\theta}) - \mathbf{y}_i|^2$$

达到最小. 这里, $\mathbf{x}_i(T; \boldsymbol{\theta})$ 表示式 (3.10) 在 $t = T$ 时的解, 其初值为 $\mathbf{x}(0) = \mathbf{x}_i$, 右端由前馈神经网络 $\mathbf{V}(\mathbf{x}, t; \boldsymbol{\theta})$ 表示. 注意, $\mathbf{y}_i$ 是测得的输出值. 还可以用一种较为直接的方法将该思路推广到 $\mathbf{x}_i$ 与 $\mathbf{y}_i$ 维数不同的情形. 总之, 神经 ODE 将回归问题转化为寻找 ODE 系统非线性、含时右端的问题.

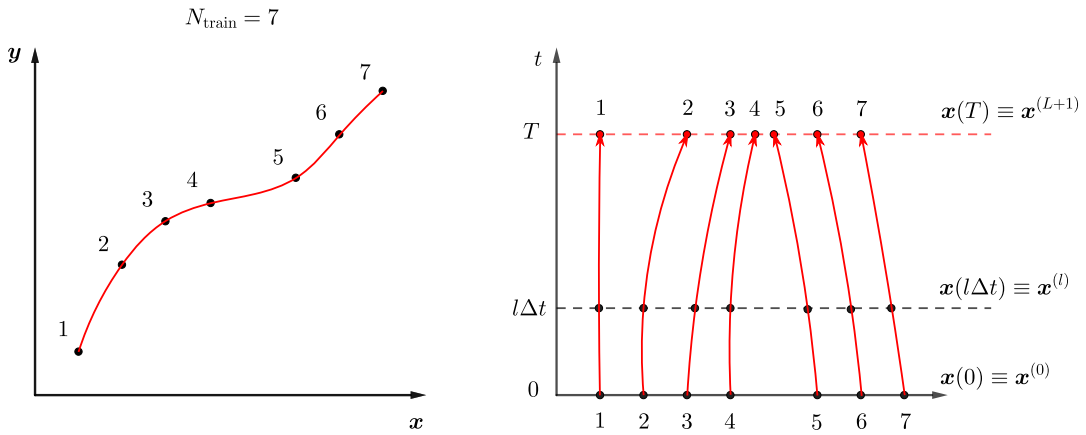


图 3.5: 回归问题与神经 ODE 之间的类比.

下面列出神经 ODE 相对于 ResNet 的优点和差异:

- 如果把神经 ODE 的时间步数解释为 ResNet 的隐藏层数 L , 则两种方法的计算成本均为 $\mathcal{O}(L)$. 这是执行一次前向传播和一次反向传播的成本. 但两者的内存成本, 即存储各层权重的成本, 并不相同. 对神经 ODE 而言, 全部权重都属于用来表示函数 $\mathbf{V}(\mathbf{x}, t; \boldsymbol{\theta})$ 的前馈网络, 因此权重数与求解 ODE 所用的时间步数无关. 而在 ResNet 中, 权重数随层数线性增长, 故存储成本按 $\mathcal{O}(L)$ 标度.

- 在神经 ODE 中，可以取极限 $\Delta t \rightarrow 0$ 并研究收敛性，因为这不会改变用于表示右端的网络大小. 对 ResNet 而言，这在计算上不可行，因为 $\Delta t \rightarrow 0$ 对应于网络深度 $L \rightarrow \infty$.
- ResNet 使用向前欧拉型方法，而神经 ODE 可以使用任意时间积分器，尤其可以使用其他高阶显式时间积分器，例如以更快速率收敛到“精确”解的 Runge–Kutta 方法.

用 MLP 求解 PDE

求解偏微分方程 (PDE) 的数值方法很多, 其中包括:

- 有限差分法.
- 有限体积法.
- 有限元法.
- 谱 Galerkin 法与谱配置法.
- 深度神经网络.

为了更好地理解其中一些方法, 尤其是深度神经网络方法, 考虑一个描述一维标量对流-扩散问题的简单模型: 在区间 $x \in (0, \ell)$ 上求 $u(x)$, 使得

$$\begin{aligned} a \frac{du}{dx} - \kappa \frac{d^2u}{dx^2} &= f(x), & x \in (0, \ell), \\ u(0) &= 0, \\ u(\ell) &= 1. \end{aligned} \tag{4.1}$$

其中, a 表示对流速度 (advective velocity), κ 是扩散系数, $f(x)$ 是源项. 这类方程用于模拟许多物理现象, 例如流体对污染物的输运, 以及半导体中电子的流动. 该问题的多维版本具有形式

$$\begin{aligned} \mathbf{a} \cdot \nabla \mathbf{u}(\mathbf{s}) - \kappa \Delta \mathbf{u}(\mathbf{s}) &= \mathbf{f}(\mathbf{s}), & \mathbf{s} \in \Omega, \\ \mathbf{u}(\mathbf{s}) &= \mathbf{g}(\mathbf{s}), & \mathbf{s} \in \partial\Omega. \end{aligned} \tag{4.2}$$

注意, 该模型问题是线性 PDE (在一维情形下是 ODE). 若以 u 替换速度 a , 便得到黏性 Burgers 方程.

当 $f \equiv 0$ 时, 式 (4.1) 的解可解析地写成

$$u(x) = \frac{1 - \exp(ax/\kappa)}{1 - \exp(a\ell/\kappa)}.$$

其中, $a\ell/\kappa$ 称为 Péclet 数 (Pe) (Péclet number, Pe), 它衡量对流强度与扩散强度之比. 不同 a 和 κ 下的解见图 4.1. 当 Pe 很小时, 解基本是一条直线; 随着 Pe 增大, 解开始弯曲, 并在右边界附近形成更陡的边界层. 源文给出的边界层厚度为 $\delta \approx \text{Pe} \times \ell$.

下面考察几种数值求解该示例问题的方法.

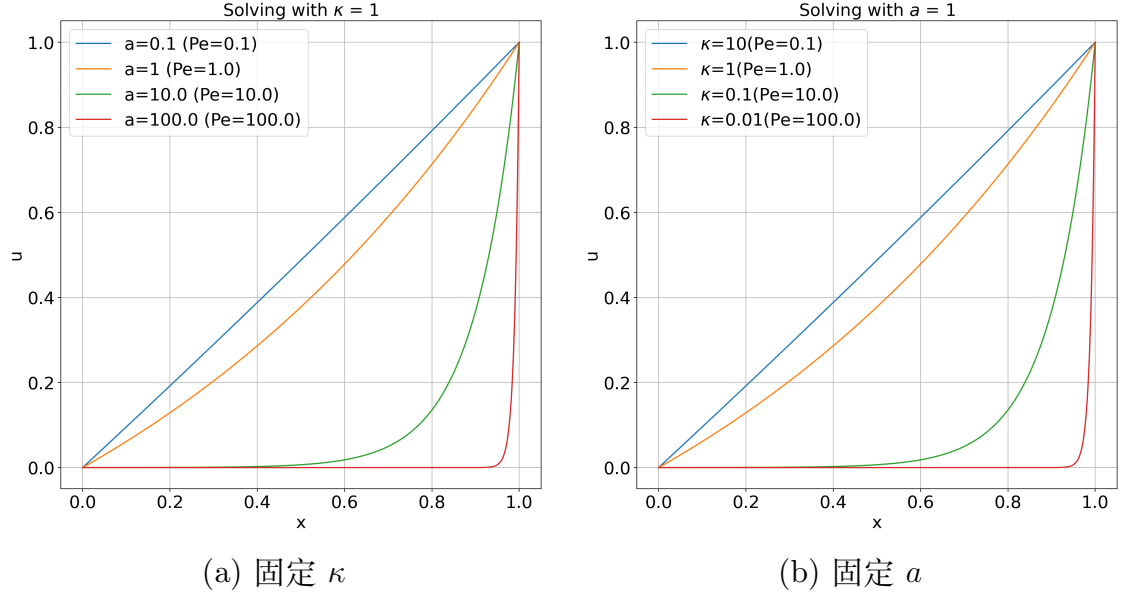


图 4.1: 取 $\ell = 1$ 时式 (4.1) 的精确解.

4.1 有限差分法

有限差分格式 (finite difference scheme) 的关键步骤如下:

1. 将区域离散成点网格, 目标是在这些点上求解.
2. 在这些点上用有限差分近似导数, 从而得到一个线性或非线性代数方程组.
3. 使用适当算法求解该方程组.

将这些步骤用于式 (4.1):

1. 将区域离散为 $N + 1$ 个点, 令 $x_i = ih$ 、 $0 \leq i \leq N$, 其中 $h = \ell/N$. 我们要求 $u(x_i) = u_i$. 由边界条件还已知 $u_0 = 0$ 和 $u_N = 1$.
2. 使用近似

$$\begin{aligned}\frac{du}{dx}(x_i) &= \frac{u_{i+1} - u_{i-1}}{2h} + \mathcal{O}(h^2), \\ \frac{d^2u}{dx^2}(x_i) &= \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + \mathcal{O}(h^2).\end{aligned}$$

上述两种近似都具有二阶精度. 它们是中心差分 (central difference) 近似, 因为它们对第 i 个点两侧的点赋予大小相同的权重. 值得注意的是, 在 Péclet 数很大的极限下, 对流项的中心差分近似并不理想, 因为它会导致数值不稳定; 此时更适合采用迎风近似 (upwind approximation) .

在 x_i ($1 \leq i \leq N-1$) 处把这些近似用于 PDE:

$$\begin{aligned} & a \frac{u_{i+1} - u_{i-1}}{2h} - \kappa \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} = f_i, \\ \iff & u_{i+1} \underbrace{\left(\frac{a}{2h} - \frac{\kappa}{h^2}\right)}_{\gamma} + u_i \underbrace{\left(\frac{2\kappa}{h^2}\right)}_{\beta} + u_{i-1} \underbrace{\left(-\frac{a}{2h} - \frac{\kappa}{h^2}\right)}_{\alpha} = f_i. \end{aligned}$$

对解未知的每个节点分别考察 (回顾 $u_0 = 0$ 和 $u_N = 1$ 已知), 得到

$$\begin{aligned} & \beta u_1 + \gamma u_2 = -\alpha u_0 + f_1, \\ & \alpha u_{i-1} + \beta u_i + \gamma u_{i+1} = f_i, \quad 2 \leq i \leq N-2, \\ & \alpha u_{N-2} + \beta u_{N-1} = -\gamma u_N + f_{N-1}. \end{aligned} \tag{4.3}$$

合并式 (4.3) 中的全部 $N-1$ 个方程, 可得线性系统

$$\mathbf{K} \mathbf{u} = \mathbf{f}. \tag{4.4}$$

其中, 三对角矩阵 $\mathbf{K}$ 及式 (4.4) 中的其他向量定义为

$$\begin{aligned} \mathbf{K} &= \begin{bmatrix} \beta & \gamma & & 0 \\ \alpha & \ddots & \ddots & \\ & \ddots & \ddots & \gamma \\ 0 & & \alpha & \beta \end{bmatrix} \in \mathbb{R}^{(N-1) \times (N-1)}, \\ \mathbf{u} &= \begin{bmatrix} u_1 & u_2 & \cdots & u_{N-2} & u_{N-1} \end{bmatrix}^T \in \mathbb{R}^{N-1}, \\ \mathbf{f} &= \begin{bmatrix} -\alpha u_0 + f_1 & f_2 & f_3 & \cdots & f_{N-2} & f_{N-1} & -\gamma u_N + f_{N-1} \end{bmatrix}^T \in \mathbb{R}^{N-1}. \end{aligned}$$

最后一行按源 PDF 保留其可见条目.

3. 求解 $\mathbf{u} = \mathbf{K}^{-1} \mathbf{f}$.

需要注意:

- 实践中从不真正求 $\mathbf{K}$ 的逆, 因为这样做计算代价很高. 我们会使用高效的数值算法求解式 (4.4). 对这个特定系统, 例如可采用 Thomas 三对角算法, 它是高斯消元法的简化版本.
- 我们只得到近似 $u_i \approx u(x_i)$. 为减小近似误差, 可以减小网格尺寸 h ; 也可以采用高阶有限差分近似, 但这会形成更“宽”的模板 (stencil) 来近似每一点的导数.
- 可以把每个“施加”PDE 的点视为配置点 (collocation point). 在配置点上施加 PDE 的思想也为下一种方法以及用 MLP 求解 PDE 的方法所共有.

4.2 谱配置法

谱配置法 (spectral collocation method) 寻求用一组光滑全局基函数展开的解. 基函数事先选定, 展开系数则是未知量; 通过要求 PDE 数值解在一组所谓的配置点上精确成立来计算这些系数. 更具体地说, 该方法包括以下步骤.

1. 选择一组满足以下性质的全局基函数:

- (a) 在所考虑的函数空间中构成完备基.
- (b) 足够光滑, 因而能够计算导数.
- (c) 容易求值.
- (d) 其导数容易求值.

例如, 可使用定义在 $\xi \in (-1, 1)$ 上的 Chebyshev 多项式, 其递推关系为

$$T_0(\xi) = 1, \quad T_1(\xi) = \xi, \quad T_{n+1}(\xi) = 2\xi T_n(\xi) - T_{n-1}(\xi).$$

前几个 Chebyshev 多项式见图 4.2. 该基满足以上全部要求. 利用递推关系以及前两个低阶多项式的值, 可以在任意点容易地求出下一阶 Chebyshev 多项式. 对递推关系求导, 还可以得到各阶导数的递推关系.

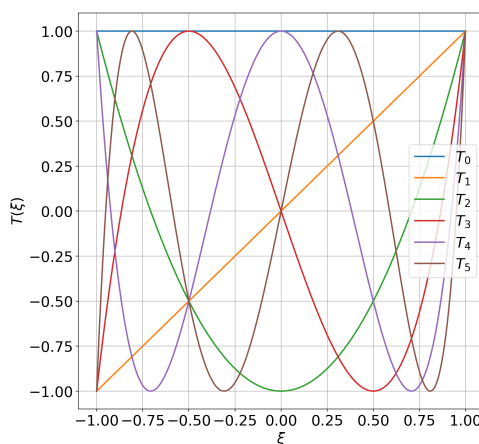


图 4.2: 前几个 Chebyshev 多项式.

2. 将解写成基函数 $\{\phi_n(x)\}_{n=0}^N$ 的线性组合:

$$u(x) = \sum_{n=0}^N u_n \phi_n(x). \quad (4.5)$$

其中 u_n 是基函数系数. 对示例问题 (4.1), 假定 $\ell = 1$, 使用 Chebyshev 多项式 $\phi_n(x) = T_n(2x - 1)$; 这里对自变量作了变换, 以便在区间 $(0, 1)$ 上使用这些函数.

3. 计算 PDE 所需的导数. 对示例问题, 有

$$\begin{aligned}\frac{du}{dx}(x) &= \sum_{n=0}^N u_n \phi'_n(x) = \sum_{n=0}^N 2u_n T'_n(2x-1), \\ \frac{d^2u}{dx^2}(x) &= \sum_{n=0}^N u_n \phi''_n(x) = \sum_{n=0}^N 4u_n T''_n(2x-1).\end{aligned}\quad (4.6)$$

4. 使用 PDE 的边界条件. 对式 (4.1) 的特定情形,

$$\begin{aligned}u(0) = 0 &\implies \sum_{n=0}^N u_n \phi_n(0) = \sum_{n=0}^N u_n T_n(-1) = 0, \\ u(1) = 1 &\implies \sum_{n=0}^N u_n \phi_n(1) = \sum_{n=0}^N u_n T_n(1) = 1.\end{aligned}\quad (4.7)$$

这给出关于 $N+1$ 个系数的两个线性方程. 然后, 在区域内部选择一组适当节点 x_i ($1 \leq i \leq N-1$) 作为配置点, 并在这 $N-1$ 个节点处把第 3 步得到的导数代入 PDE:

$$\begin{aligned}&a \sum_{n=0}^N u_n \phi'_n(x_i) - \kappa \sum_{n=0}^N u_n \phi''_n(x_i) = f(x_i), \\ \implies &\sum_{n=0}^N u_n [2aT'_n(2x_i-1) - 4\kappa T''_n(2x_i-1)] = f(x_i).\end{aligned}\quad (4.8)$$

这又给出关于 $N+1$ 个系数的 $N-1$ 个方程. 合并式 (4.7) 与式 (4.8), 得到线性系统

$$\mathbf{K}\mathbf{u} = \mathbf{f}. \quad (4.9)$$

其中

$$\begin{aligned}\mathbf{K} &\in \mathbb{R}^{(N+1) \times (N+1)}, \\ \mathbf{u} &= \begin{bmatrix} u_0 & u_2 & \cdots & u_{N-1} & u_N \end{bmatrix}^T \in \mathbb{R}^{N+1}, \\ \mathbf{f} &= \begin{bmatrix} 0 & f(x_1) & f(x_1) & \cdots & f(x_{N-2}) & f(x_{N-1}) & 1 \end{bmatrix}^T \in \mathbb{R}^{N+1}.\end{aligned}$$

上述两个向量按源 PDF 的可见条目保留.

5. 求解 $\mathbf{u} = \mathbf{K}^{-1}\mathbf{f}$.

需要适当选择配置点或求积点 x_i , 使 $\mathbf{K}$ 具有便于求解线性系统 (4.9) 的良好性质, 包括可逆性、正定性和稀疏性等.

注 4.2.1: 该方法之所以称为配置法, 是因为 PDE 在特定的配置点或求积点 x_i 上求值. $\diamond$

注 4.2.2: 处理非线性 PDE 时, 将得到关于系数 $u_0, \dots, u_N$ 的非线性代数方程组, 通常使用 Newton 法求解. $\diamond$

下面考察一种用最小二乘法求取谱展开系数的变体. 与前面一样, 仍用式 (4.5) 表示解并计算其导数, 然后通过最小化损失函数

$$\begin{aligned}\Pi(\mathbf{u}) &= \Pi_{\text{int}}(\mathbf{u}) + \lambda \Pi_{\text{bc}}(\mathbf{u}), \\ \Pi_{\text{bc}}(\mathbf{u}) &= \left| \sum_{n=0}^N u_n \phi_n(0) - 0 \right|^2 + \left| \sum_{n=0}^N u_n \phi_n(1) - 1 \right|^2, \\ \Pi_{\text{int}}(\mathbf{u}) &= \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \left| a \sum_{n=0}^N u_n \phi'_n(x_i) - \kappa \sum_{n=0}^N u_n \phi''_n(x_i) - f(x_i) \right|^2\end{aligned}\quad (4.10)$$

来求系数 $\mathbf{u}$. 可以使用第 2 章介绍的任意梯度方法求解. 这种方法对非线性 PDE 尤其有用, 因为在这些情形下一般无法写出类似式 (4.9) 的系数线性系统. 需要注意:

- 参数 λ 用于以不同尺度加权内部损失和边界损失.
- 内部点 x_i 的数目可以独立于基函数数目选择; 换言之, N_{train} 不必等于 N .
- 下一节将看到, 谱方法的这一变体与使用深度神经网络求解 PDE 的方式非常相似.

4.3 物理信息神经网络 (PINN)

Lagaris 等人在 20 世纪 90 年代至 21 世纪初提出了使用神经网络求解偏微分方程的思想 [11]. 随着人们重新关注利用机器学习工具求解 PDE, 该思想于 2019 年由 Raissi 等人再次提出 [24], 并被命名为物理信息神经网络 (PINN) (physics-informed neural network, PINN). PINN 的基本思想与回归相似, 区别在于损失函数 $\Pi(\boldsymbol{\theta})$ 包含所考虑 PDE 中出现的微分算子. 下面针对一维标量 PDE 概述主要步骤; 它们很容易推广到多维 PDE 系统. 建议读者思考该方法与上一节谱配置法之间的异同.

1. 选择神经网络作为 PDE 解的函数表示:

$$u = \mathcal{F}(x; \boldsymbol{\theta}). \quad (4.11)$$

这种表示需要具备若干关键性质:

- (a) 该表示是否完备, 即能否用它准确逼近所需函数类? 答案是肯定的, 因为神经网络具有通用逼近定理 (见节 2.3.1).
- (b) 该表示是否光滑? 如果激活函数光滑, 例如 $\tanh$ 、 $\sin$ 等, 答案是肯定的. ReLU 不具备足够多阶的光滑导数, 因此不能使用.
- (c) 该表示是否容易求值? 答案是肯定的, 因为只需快速执行一次前向传播.
- (d) 其导数是否容易求值? 答案是肯定的, 因为可以使用反向传播; 下文将详细讨论.

- 给定表示 (4.11)，需要寻找 θ ，使 PDE 以某种适当形式成立。将其与谱配置近似 (4.5) 比较，后者需要确定系数 u_n 。注意，式 (4.5) 对系数 u_n 的依赖是线性的，而式 (4.11) 对 θ 的依赖可能高度非线性。
- 接下来求该表示的导数。考虑图 4.3 所示的网络计算图，它由仿射变换与逐分量非线性变换交替组成。输出对输入的导数可通过反向传播求出。只需在图 2.8 所示计算图中令 $\Pi = \mathbf{x}^{(L+1)}$ ，即可得到图 4.3。又因为 $\partial \mathbf{x}^{(L+1)} / \partial \mathbf{x}^{(L+1)} = \mathbf{I}$ ，即单位矩阵，故可直接从图中读出

$$\frac{\partial \mathbf{x}^{(L+1)}}{\partial \mathbf{x}^{(0)}} = \mathbf{W}^{(L+1)} \mathbf{S}^{(L+1)} \mathbf{W}^{(L)} \mathbf{S}^{(L)} \dots \mathbf{W}^{(2)} \mathbf{S}^{(2)} \mathbf{W}^{(1)} \mathbf{S}^{(1)}.$$

因而，为计算 du/dx ，必须在原图上扩展出一条反向分支，用于计算向量 $\xi^{(l)}$ 各分量的激活函数导数（见图 4.3）。对扩展后的图执行反向传播即可计算二阶导数 d^2u/dx^2 。若要计算更高阶导数，还需以类似方式进一步扩展计算图。在 PyTorch 中调用 “autograd” 时，幕后执行的正是这些操作。

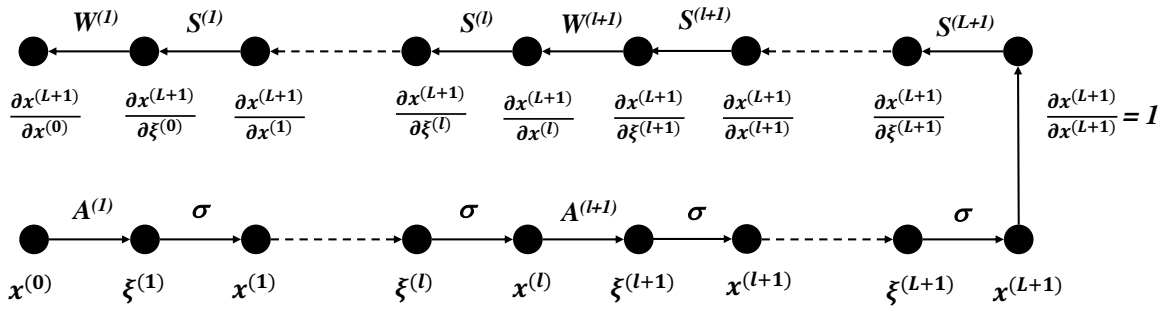


图 4.3: 用于计算网络输出对输入导数的扩展计算图。

- 将解的函数表示 (4.11) 代入 PDE，以求参数 θ 。首先定义用于训练网络的一组点 $\mathcal{S} = \{x_i : 1 \leq i \leq N_{\text{train}}\}$ ，它们类似于谱配置法中的配置点。然后，针对示例问题 (4.1) 定义损失函数

$$\begin{aligned} \Pi(\theta) &= \Pi_{\text{int}}(\theta) + \lambda_b \Pi_b(\theta), \\ \Pi_b(\theta) &= (\mathcal{F}(0; \theta) - 0)^2 + (\mathcal{F}(1; \theta) - 1)^2, \\ \Pi_{\text{int}}(\theta) &= \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (a\mathcal{F}'(x_i; \theta) - \kappa\mathcal{F}''(x_i; \theta)_n - f(x_i))^2. \end{aligned} \quad (4.12)$$

这里按源 PDF 保留 $\mathcal{F}''$ 后的下标 n 。训练网络就是求解极小化问题

$$\theta^* = \arg \min_{\theta} \Pi(\theta),$$

随后把解写成 $u^*(x) = \mathcal{F}(x; \theta^*)$ 。这与谱配置法的最小二乘变体完全一致，后者通过最小化类似损失求解系数 u_n 。

下面作几点说明：

- 如果能够找到满足 $\Pi(\boldsymbol{\theta}^*) = 0$ 的 $\boldsymbol{\theta}^*$ ，则 $\Pi_{\text{int}}(\boldsymbol{\theta}^*) = 0$ 且 $\Pi_b(\boldsymbol{\theta}^*) = 0$ 。换言之，PDE 残差在配置点处为零。只要配置点很好地覆盖整个区域，这就会给出良好解。
- 改善 PINN 精度的方法有很多，例如：
 - 增加配置点数目。
 - 改变加权边界损失的超参数 λ_b 。
 - 增大网络，即增大 N_θ 。
- 微分方程的边界条件（BC）承载着所描述现象的基本物理性质，数值解满足这些条件至关重要。在 PINN 框架中，边界条件通过惩罚项 $\Pi_b(\boldsymbol{\theta})$ 作为软约束（soft constraint）施加。因而超参数 λ_b 在网络训练中至关重要，因为它在极小化过程中平衡两个损失项。如果不同损失项的梯度没有得到适当缩放，同时满足边界条件和 PDE 本身的解可能收敛得极慢；对刚性 PDE 而言，这一问题尤其严重。为处理这一问题，人们提出了多种自适应技术，在训练过程中调节 λ_b [29, 16, 3]。

4.4 将 PINN 推广到更一般的 PDE

考虑一般 PDE：求解 $\mathbf{u} : \Omega \subset \mathbb{R}^d \rightarrow \mathbb{R}^D$ ，使得

$$\begin{aligned} L(\mathbf{u}(\mathbf{x})) &= \mathbf{f}(\mathbf{x}), & \mathbf{x} \in \Omega, \\ B(\mathbf{u}(\mathbf{x})) &= \mathbf{g}(\mathbf{x}), & \mathbf{x} \in \partial\Omega. \end{aligned} \quad (4.13)$$

其中， L 是微分算子， $\mathbf{f}$ 是已知强迫项， B 是边界算子， $\mathbf{g}$ 是边界条件中给定的非齐次部分。

作为例子，考虑三维不可压缩 Navier–Stokes 方程，在 $\Omega = \Omega_S \times [0, T]$ 上求速度场 $\mathbf{v} = [v_1, v_2, v_3]$ 和压力 p 。这里 Ω_S 是三维空间区域， $[0, T]$ 是关注的时间区间。方程为

$$\begin{aligned} \frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} + \nabla p - \mu \Delta \mathbf{v} &= \mathbf{f}, & (\mathbf{s}, t) \in \Omega, \\ \nabla \cdot \mathbf{u} &= 0, & (\mathbf{s}, t) \in \Omega, \\ \mathbf{v} &= \mathbf{0}, & (\mathbf{s}, t) \in \partial\Omega_S \times [0, T], \\ \mathbf{v}(\mathbf{s}, 0) &= \mathbf{v}_0(\mathbf{s}), & \mathbf{s} \in \Omega_S. \end{aligned} \quad (4.14)$$

第二式中的 $\mathbf{u}$ 按源 PDF 保留。第一式表示线性动量平衡，第二式施加质量守恒，第三式是边界刚性且固定时使用的无滑移边界条件，第四式给定初始速度场。

为式 (4.13) 设计 PINN，网络输入应为自变量 $\mathbf{x}$ ，输出应为解向量 $\mathbf{u}$ 。对 Navier–Stokes 系统 (4.14)，网络输入为 $[s_1, s_2, s_3, t] \in \mathbb{R}^4$ ，输出为 $\mathbf{u} = [v_1, v_2, v_3, p] \in \mathbb{R}^4$ 。步骤如下：

1. 构造损失函数：

- 定义内部残差 $R(\mathbf{u}) = L(\mathbf{u}) - \mathbf{f}$ 。

- 定义边界残差 $R_b(\mathbf{u}) = B(\mathbf{u}) - \mathbf{g}$.
- 在区域内部选择适当的 N_v 个配置点, 在区域边界上选择 N_b 个点, 用于计算残差. 这些点可按 Gaussian、Lobatto、均匀或随机等求积规则选择.

损失函数为

$$\begin{aligned}\Pi(\boldsymbol{\theta}) &= \Pi_{\text{int}}(\boldsymbol{\theta}) + \lambda_b \Pi_b(\boldsymbol{\theta}), \\ \Pi_{\text{int}}(\boldsymbol{\theta}) &= \frac{1}{N_v} \sum_{i=1}^{N_v} |R(\mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta}))|^2, \\ \Pi_b(\boldsymbol{\theta}) &= \frac{1}{N_b} \sum_{i=1}^{N_b} |R_b(\mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta}))|^2.\end{aligned}$$

2. 训练网络: 求 $\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \Pi(\boldsymbol{\theta})$, 并令 $\mathbf{u}^* = \mathcal{F}(\mathbf{x}; \boldsymbol{\theta}^*)$.

这里再作几点说明:

- 默认损失 $\Pi(\boldsymbol{\theta})$ 中还加入了权重正则化项.
- $\mathbf{u}^*(\mathbf{x})$ 是 PDE 的精确解吗? 答案是否定的:
 - 首先, $\Pi(\boldsymbol{\theta}^*)$ 可能不为零.
 - 即使 $\Pi(\boldsymbol{\theta}^*)$ 恒为零, 也只意味着残差在配置点处消失, 并不保证残差在整个区域中处处消失; 为此需要 $N_v, N_b \rightarrow \infty$.
 - 固定网络 (N_{θ} 固定) 也不能表示所有函数; 为此需要 $N_{\theta} \rightarrow \infty$.
- 实践中只能计算 $\Pi(\boldsymbol{\theta}^*)$. 解误差 $\|\mathbf{e}\| = \|\mathbf{u}^* - \mathbf{u}\|$ 是否与该损失值有关? 若有关, 损失小时能否断言误差也小? 下一节将尝试回答这些问题.

4.5 PINN 的误差分析

为了计算误差 $\mathbf{e} = \mathbf{u}^* - \mathbf{u}$, 需要知道精确解 $\mathbf{u}$, 但它通常不可得. 下面把讨论限制在线性 PDE, 即 L 和 B 都是线性算子, 以此绕开该困难.

若 $\mathbf{u}$ 是精确解, 则

$$L(\mathbf{e}) = L(\mathbf{u}^* - \mathbf{u}) = L(\mathbf{u}^*) - L(\mathbf{u}) = L(\mathbf{u}^*) - \mathbf{f} = R(\mathbf{u}^*), \quad (4.15)$$

以及

$$B(\mathbf{e}) = B(\mathbf{u}^* - \mathbf{u}) = B(\mathbf{u}^*) - B(\mathbf{u}) = B(\mathbf{u}^*) - \mathbf{g} = R_b(\mathbf{u}^*). \quad (4.16)$$

因此, 式 (4.15) 与式 (4.16) 给出一个由 MLP 解残差驱动的误差 PDE:

$$\begin{aligned}L(\mathbf{e}) &= R(\mathbf{u}^*), & \text{在 } \Omega \text{ 中,} \\ B(\mathbf{e}) &= R_b(\mathbf{u}^*), & \text{在 } \Omega \text{ 上.}\end{aligned} \quad (4.17)$$

第二行的区域按源 PDF 保留. 如果 $\mathbf{u}^*$ 的残差为零, 则 $\mathbf{e} = 0$. 遗憾的是, 这些残差并不为零; 至多只能说它们在配置点处很小. 但根据适定 PDE 的稳定性理论, 有

$$\|\mathbf{e}\|_{L^2(\Omega)} \leq C_1 (\|R(\mathbf{u}^*)\|_{L^2(\Omega)} + \|R_b(\mathbf{u}^*)\|_{L^2(\partial\Omega)}). \quad (4.18)$$

其中 C_1 是依赖于 PDE、区域 Ω 等的稳定性常数. 源文称该条件对所有适定 PDE 都成立; 它表示若驱动 PDE 的各项很小, PDE 解也会很小. 因而, 只要能控制 MLP 解的残差, 就能控制误差. 然而, 实践中已知并控制的是 Π_{int} 和 Π_b , 而不是 $\|R(\mathbf{u}^*)\|_{L^2(\Omega)}^2$ 和 $\|R_b(\mathbf{u}^*)\|_{L^2(\partial\Omega)}^2$. 这些量之间是否相关, 可由以下分析回答:

$$\begin{aligned} \|R(\mathbf{u}^*)\|_{L^2(\Omega)} &= |m_\Omega \Pi_{\text{int}}(\boldsymbol{\theta}^*)^{1/2} + \|R(\mathbf{u}^*)\|_{L^2(\Omega)} - m_\Omega \Pi_{\text{int}}(\boldsymbol{\theta}^*)^{1/2}| \\ &\leq m_\Omega \Pi_{\text{int}}(\boldsymbol{\theta}^*)^{1/2} + \left| \|R(\mathbf{u}^*)\|_{L^2(\Omega)} - m_\Omega \Pi_{\text{int}}(\boldsymbol{\theta}^*)^{1/2} \right| \\ &\leq m_\Omega \Pi_{\text{int}}(\boldsymbol{\theta}^*)^{1/2} + C_2(N_v)^{-\alpha}. \end{aligned} \quad (4.19)$$

其中 m_Ω 是区域 Ω 的测度, C_2 和 $\alpha > 0$ 依赖于所选求积点的类型. 上式第一行通过加上再减去 $m_\Omega \Pi_{\text{int}}(\boldsymbol{\theta}^*)^{1/2}$ 得到; 第二行使用三角不等式; 第三行则陈述了用求积点上的有限个函数值之和近似积分时的误差.

类似地, 对边界残差有

$$\|R_b(\mathbf{u}^*)\|_{L^2(\partial\Omega)} \leq m_{\partial\Omega} \Pi_b(\boldsymbol{\theta}^*)^{1/2} + C_3(N_b)^{-\beta}. \quad (4.20)$$

其中 $m_{\partial\Omega}$ 是 $\partial\Omega$ 的测度, C_3 和 $\beta > 0$ 依赖于边界求积点的类型.

结合式 (4.18)、(4.19) 和 (4.20), 得到

$$\|\mathbf{e}\|_{L^2(\Omega)} \leq C_1 \left(\underbrace{m_\Omega \Pi_{\text{int}}(\boldsymbol{\theta}^*)^{1/2} + m_{\partial\Omega} \Pi_b(\boldsymbol{\theta}^*)^{1/2}}_{\text{通过增大 } N_\theta \text{ 减小}} + \underbrace{C_2(N_v)^{-\alpha} + C_3(N_b)^{-\beta}}_{\text{通过增大 } N_v, N_b \text{ 减小}} \right). \quad (4.21)$$

该式说明, 可以通过减小损失函数 (增大 N_θ) 并增加内部与边界配置点的数目, 来控制 PINN 解的误差. 更详细的分析见文献 [18].

4.6 使用 PINN 进行数据同化

数据同化 (data assimilation) 问题常见于科学与工程领域. 在这类问题中, 只能对某个量作少数稀疏测量, 希望利用这些测量在细网格上处处估计该量. 同时还给定该变量必须满足的物理原理, 通常以 PDE 形式表示.

假设给定区域 Ω 上某个量 u 的一组稀疏测量:

$$u_i = u(\mathbf{x}_i), \quad \mathbf{x}_i \in \Omega, \text{quad}1 \leq i \leq M.$$

进一步给定 u 在 Ω 上满足约束 $R(u) = 0$. 数据同化就是利用这些信息求任意 $\mathbf{x} \in \Omega$ 处的 u 值.

可以使用 PINN 求解该问题. 首先, 以神经网络 $\mathcal{F}(\mathbf{x}, \boldsymbol{\theta})$ 表示 u . 然后定义如下损失:

$$\begin{aligned} \Pi(\boldsymbol{\theta}) = & \underbrace{\frac{\lambda_I}{M} \sum_{i=1}^M (u_i - \mathcal{F}(\mathbf{x}_i, \boldsymbol{\theta}))^2}_{\text{数据匹配}} + \underbrace{\frac{1}{N_v} \sum_{i=M+1}^{M+N_v} |R(\mathcal{F}(\mathbf{x}_i, \boldsymbol{\theta}))|^2}_{\text{物理约束}} \\ & + \underbrace{\lambda \|\boldsymbol{\theta}\|^2}_{\text{光滑性}}. \end{aligned}$$

其中, $\mathbf{x}_i$ ($M+1 \leq i \leq M+N_v$) 是用于计算残差的一些配置点, λ_I 和 λ 是超参数. 最后训练网络, 求 $\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \Pi(\boldsymbol{\theta})$, 并按源文记号令 PINN 解为 $\mathbf{u}^* = \mathcal{F}(\mathbf{x}; \boldsymbol{\theta}^*)$.

卷积神经网络

前几章已经介绍了如何使用全连接层构造神经网络. 下面考察另一类称为卷积层 (convolution layer) 的网络层, 它们在处理图像输入时非常有用. 相关任务包括图像分类、图像语义分割, 以及把一种图像变换为另一种图像.

5.1 函数与图像

考虑定义在 $\mathbf{x} \in [a, b] \times [c, d] \subset \mathbb{R}^2$ 上的函数 $u(\mathbf{x})$. 该函数的离散版本可以可视化为图像 $\mathbf{U} \in \mathbb{R}^{N_1 \times N_2}$, 其中

$$U[i, j] = u(ih, jh), \quad 1 \leq i \leq N_1, \quad 1 \leq j \leq N_2, \quad h = \text{像素尺寸}. \quad (5.1)$$

式 (5.1) 定义了一幅灰度图像, 每个像素处的 u 值就是灰度强度. 对彩色图像, 则得到三维张量, 其第三维对应源文所列的红、蓝、绿三个通道, 即 $\mathbf{U} \in \mathbb{R}^{N_1 \times N_2 \times 3}$.

如果用全连接神经网络 (MLP) 处理尺寸为 100×100 的二维彩色图像, 将整幅图像展开成单个向量后, 输入维数为 3×10^4 , 非常庞大; 这又会产生计算上不可行的巨大全连接层. 其次, 展开图像会丢失原图的全部空间语境. 最后, 边缘检测等局部运算理应在图像各区域保持相同. 考虑全连接层的权重矩阵 W_{ij} , 其中指标 i 表示线性变换的输出, 指标 j 表示输入. 如果每个输出指标都采用相同运算, 就会对每个 i 重复同一操作, 因而不需要一般的矩阵. 为解决这些问题, 可以在函数上使用卷积算子.

5.2 函数的卷积

卷积算子 (convolution operator) 把函数映射为函数. 考虑 $\mathbf{x} \in \mathbb{R}^d$ 上的函数 $u(\mathbf{x})$, 以及充分光滑且在 $|\mathbf{x}| \rightarrow \infty$ 时衰减的核函数 (kernel function) $g(\mathbf{x})$. 卷积算子定义为

$$\bar{u}(\mathbf{x}) = \int_{\mathbb{R}^d} g(\mathbf{y} - \mathbf{x}) u(\mathbf{y}) d\mathbf{y}. \quad (5.2)$$

可以把卷积算子解释为通过改变 $\mathbf{x}$ 对 u 采样. 例如, 在一维情况下, 令 $u(x)$ 和 $g(x)$ 如图 5.1 所示, 并令

$$\bar{u}(x) = \int_{\mathbb{R}} g(y - x) u(y) dy.$$

考虑一点 x_0 . 函数 $g(y - x_0)$ 把核平移到 x_0 , 从而在橙色阴影区域对函数 u 采样. 类似地, 对另一点 x_1 , $g(y - x_1)$ 把核平移到 x_1 , 在绿色阴影区域采样. 因此, 核移动时会在不同窗口中采样 u . 无论 x 取何值, 施加的都是相同运算. 下面考察几个典型核函数.

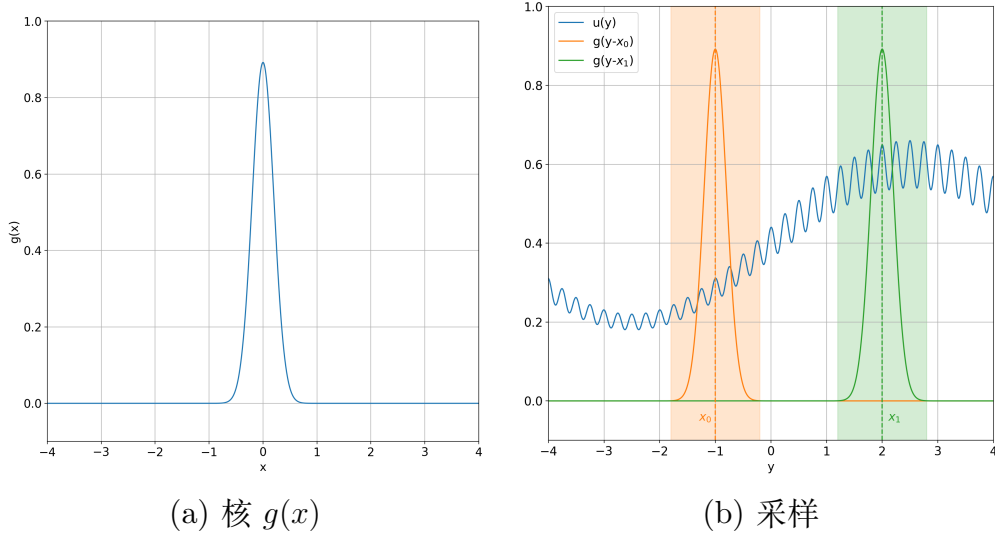


图 5.1: 一维情况下使用平移核采样.

5.2.1 例 1: Gaussian 核

一种常用选择是 Gaussian 核

$$g(\boldsymbol{\xi}) = \rho(|\boldsymbol{\xi}|), \quad \rho(r) = \frac{\exp(-r^2/(2\sigma^2))}{\sqrt{(2\pi\sigma^2)^d}},$$

其中 r 为任意标量. 该核用作平滑或模糊滤波器; d 是维数, σ 表示展宽. 图 5.1(a) 给出了 $\sigma = 0.2$ 时的一维核. 它具有以下重要性质:

- 它是各向同性的 (isotropic), 因为只依赖于 $\boldsymbol{\xi}$ 的模.
- 它在整个区域上的积分为 1.
- 它由 σ 参数化; 源文把 σ 解释为频率截断, 因为比 σ 更细的尺度会被卷积滤除. 图 5.2 展示了这一平滑效应.

5.2.2 例 2: 导数核

再考虑一种生成 u 的光滑版本之导数的核. 在二维中, 希望得到

$$\begin{aligned}
 \bar{u}(\mathbf{x}) &= \frac{\partial}{\partial x_1} \left(\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \rho(|\mathbf{y} - \mathbf{x}|) u(\mathbf{y}) \, dy_1 \, dy_2 \right) \\
 &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \frac{\partial \rho(|\mathbf{y} - \mathbf{x}|)}{\partial x_1} u(\mathbf{y}) \, dy_1 \, dy_2 \\
 &= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \underbrace{\left(-\frac{\partial \rho(|\mathbf{y} - \mathbf{x}|)}{\partial y_1} \right)}_{\text{所需的核}} u(\mathbf{y}) \, dy_1 \, dy_2.
 \end{aligned} \tag{5.3}$$

图 5.3 给出该核的一维和二维形式. 它的作用类似于光滑化的有限差分运算: 核中心左侧区域被赋予负权, 右侧区域被赋予正权.

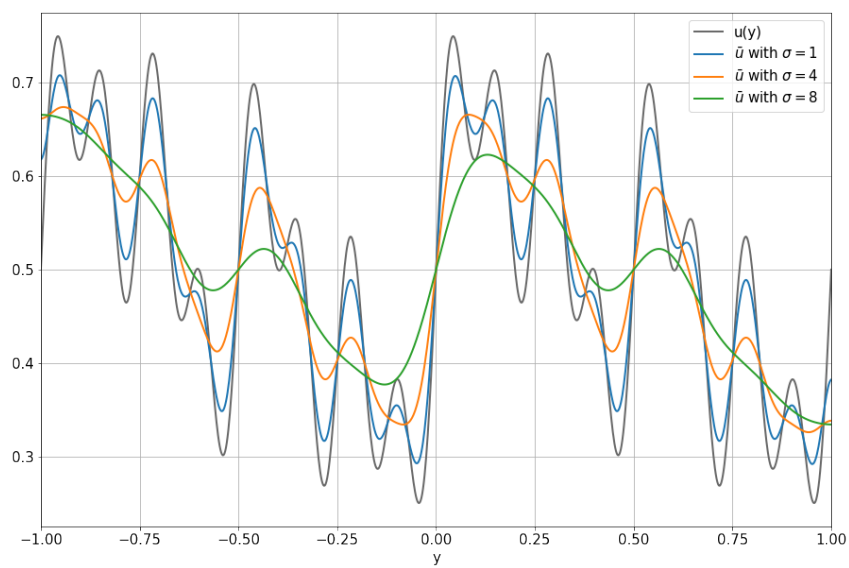
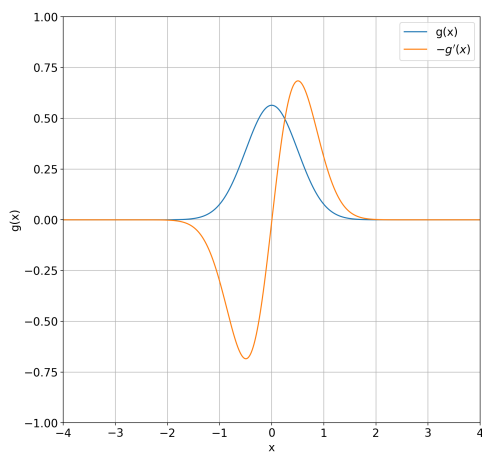
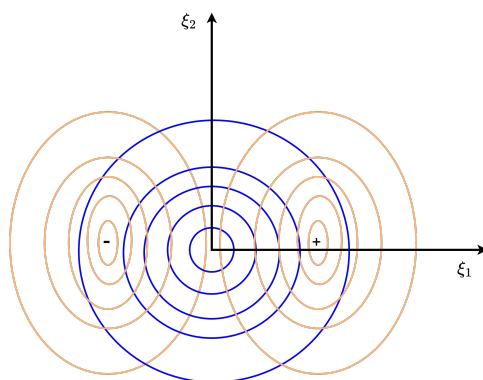


图 5.2: 使用 Gaussian 核的一维卷积.



(a) 一维



(b) 二维, 沿第 1 方向求导

图 5.3: 导数核. 蓝线表示 Gaussian 核, 橙线表示其导数.

5.3 离散卷积

为计算二维离散卷积，考虑式 (5.2) 并用求积法将其离散化，得到

$$\bar{U}[i, j] = \sum_{m=1}^{N_1} \sum_{n=1}^{N_2} g[m-i, n-j] U[m, n]. \quad (5.4)$$

这里已把测度因子 h^2 吸收到核的定义中. 与连续情形一样，假设 g 在一定距离之外消失：

$$g[p, q] = 0, \text{ 当 } |p|, |q| > \bar{N} \quad (\text{核宽度的度量}).$$

排除卷积为零的所有像素后，求和范围缩小为

$$\bar{U}[i, j] = \sum_{m=i-\bar{N}}^{i+\bar{N}} \sum_{n=j-\bar{N}}^{j+\bar{N}} g[m-i, n-j] U[m, n]. \quad (5.5)$$

令 $m' = m - i$ 、 $n' = n - j$ ，则

$$\bar{U}[i, j] = \sum_{m'=-\bar{N}}^{\bar{N}} \sum_{n'=-\bar{N}}^{\bar{N}} g[m', n'] U[i+m', j+n']. \quad (5.6)$$

深度学习中的卷积正是以这种方式施加. 因而，卷积完全由

$$g[m, n], \quad |m|, |n| \leq \bar{N}$$

决定；这些数成为卷积层的权重. 源文在此给出的权重数为 $(\bar{N} + 1)^2$.

下面看几个例子：

- 一种平滑核为

$$\frac{1}{8} \begin{bmatrix} \frac{1}{4} & 1 & \frac{1}{4} \\ 1 & 3 & 1 \\ \frac{1}{4} & 1 & \frac{1}{4} \end{bmatrix} \approx \text{具有某个 } \sigma \text{ 的 Gaussian 核.}$$

- 沿 x 方向和 y 方向求一阶导数的核分别为

$$\begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}.$$

- 类似地，沿 x 方向和 y 方向求二阶导数的核分别具有形式

$$\begin{bmatrix} 0 & 0 & 0 \\ -1 & 2 & -1 \\ 0 & 0 & 0 \end{bmatrix}, \quad \begin{bmatrix} 0 & -1 & 0 \\ 0 & 2 & 0 \\ 0 & -1 & 0 \end{bmatrix}.$$

- Laplace 算子由如下类型的核给出：

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}.$$

注 5.3.1: 不同方向可以采用不同的 $\overline{N}$ ，即核在各方向可以具有不同宽度. $\diamond$

5.4 与有限差分的联系

卷积概念与有限差分格式的模板之间存在很强的联系. 设函数 $u(x_1, x_2)$ 表示在有限网格上，网格点以 (i, j) 编号，网格尺寸为 h . 记 $U[i, j] = u(x_1^i, x_2^j)$. 在 (i, j) 附近作 Taylor 展开，有

$$\begin{aligned} \frac{U[i+1, j] - U[i-1, j]}{2h} &= \frac{u(x_1^{i+1}, x_2^j) - u(x_1^{i-1}, x_2^j)}{2h} \\ &= \frac{\partial u}{\partial x_1}(x_1^i, x_2^j) + \mathcal{O}(h^2). \end{aligned}$$

上述运算与使用权重

$$\frac{1}{2h} \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \approx \frac{\partial u}{\partial x_1}$$

的离散卷积完全相同. 因而可以说，该卷积运算近似第 1 方向的导数.

类似地，可以证明

$$\frac{U[i+1, j] - 2U[i, j] + U[i-1, j]}{h^2} = \frac{\partial^2 u}{\partial x_1^2}(x_1^i, x_2^j) + \mathcal{O}(h^2),$$

所以核

$$\frac{1}{h^2} \begin{bmatrix} 0 & 0 & 0 \\ 1 & -2 & 1 \\ 0 & 0 & 0 \end{bmatrix} \approx \frac{\partial^2 u}{\partial x_1^2}$$

对应的卷积近似第 1 方向的二阶导数.

5.5 卷积层

需要牢记以下两点：

- 每个卷积层包含多个离散卷积，每个卷积都有自己的核.
- 决定核作用方式（平滑、一阶导数、二阶导数等）的核权重是可学习参数，在训练网络时确定. 因而可以把学习过程理解为：网络学习适合当前任务的运算，即卷积；例如，任务可以是分类问题.

假设输入为一幅 $N_1 \times N_2$ 图像. 一个卷积层包含多个卷积, 每个卷积生成一幅不同图像, 如图 5.4(a) 所示. 该层的可训练参数是各卷积核的权重. 若核在每个方向的半宽为 $\bar{N}$, 且共有 P 个核, 则可训练权重数为 $P(2\bar{N} + 1)^2$.

再考虑施加单个核后输出图像的尺寸. 因为图像边界之外没有像素值, 所以无法把核施加到边界像素上 (见图 5.4(b)). 因而, 施加核时必须跳过各边界的 $\bar{N}$ 个像素; 源文给出的输出尺寸为 $(N_1 - \bar{N} + 1) \times (N_2 - \bar{N} + 1)$.

一种解决办法是在图像每条边上用 $\bar{N}$ 个零值像素作填充 (padding). 这样便可把核施加到边界像素, 输出图像尺寸与输入相同, 如图 5.4(c) 所示.

卷积的另一特征称为步幅 (stride), 它决定核在图像上移动时每次平移的像素数. 以上例子的两个方向上步幅均为 1. 实践中可以选择大于 1 的步幅, 这会进一步缩小输出图像. 例如, 若两个方向的步幅均为 S , 并采用零填充, 则输出图像的每个方向都缩小为原来的 $1/S$ (见图 5.4(d)).

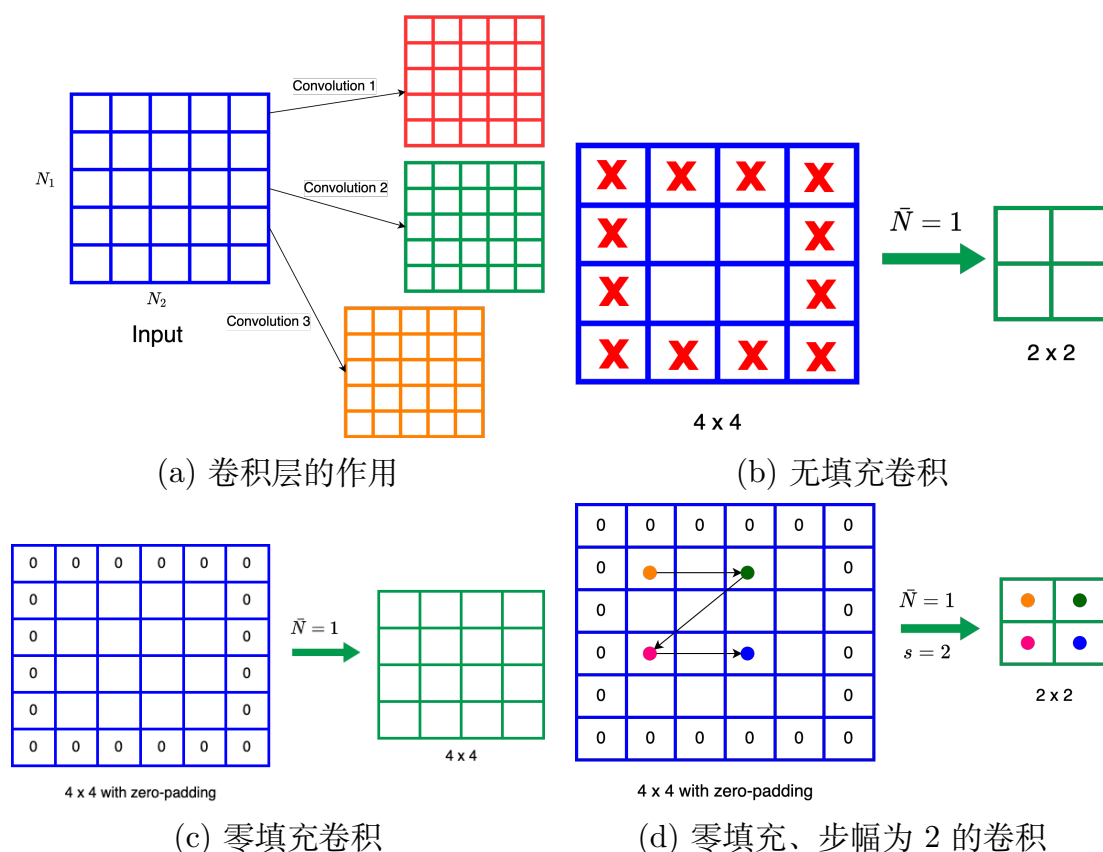


图 5.4: 卷积层或卷积核的作用.

5.5.1 平均池化与最大池化

池化 (pooling) 运算通常用于缩小图像, 并使我们能够逐级考察图像的不同尺度. 若在尺寸为 $N \times N$ 的图像上, 对 $S \times S$ 图像块施加池化, 新图像尺寸为 $(N/S) \times (N/S)$, 其中 S 是池化步幅. 图 5.5 展示了 $S = 2$ 的情形. 通常选择计算最大值或平均值的像素块大小为 $S \times S$. 图 5.5(b) 满足这一惯例, 而 (a) 不满足.

图 5.6 还展示了池化如何在图像的各个尺度之间移动：施加的池化操作越多，图像越粗糙。池化运算没有任何可训练参数。它与设计用于求解线性代数方程组的多重网格预条件器时采用的类似算子有很强的对应关系。

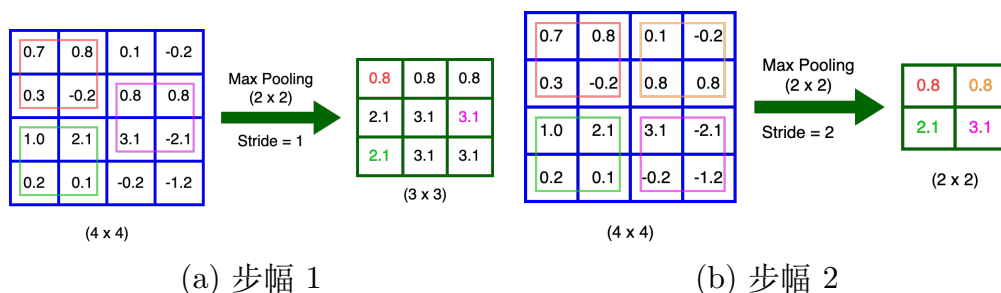


图 5.5: 在 2×2 图像块上施加最大池化。

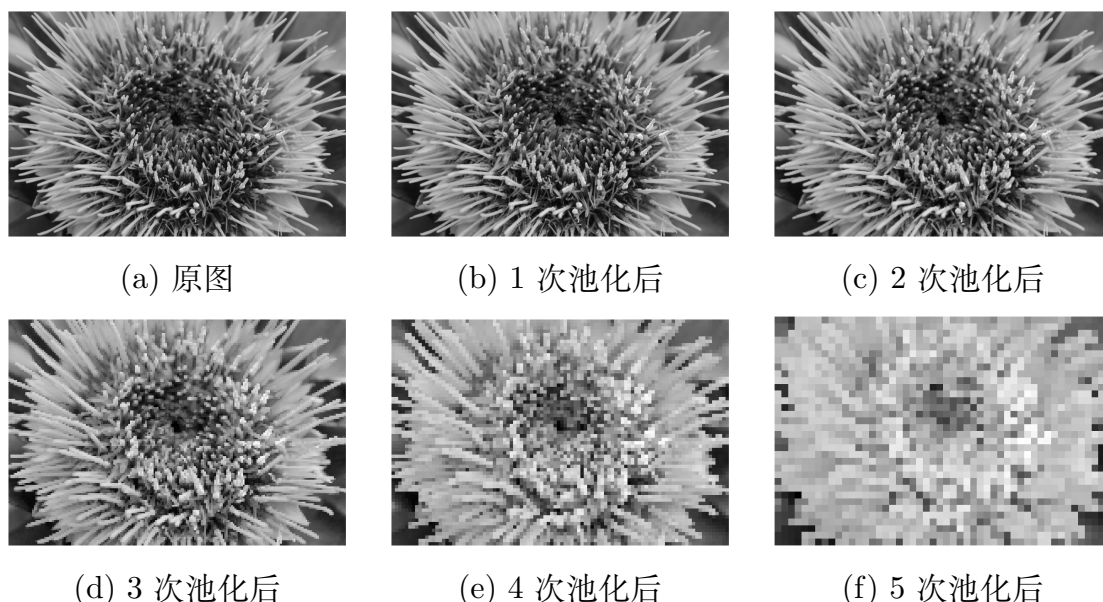


图 5.6: 在 2×2 图像块上以步幅 2 重复施加最大池化。

5.5.2 多通道输入的卷积

假设卷积层输入尺寸为 $N_1 \times N_2 \times C$ ，其中 C 是输入图像的通道数。卷积层对该输入施加 P 个卷积，得到尺寸为 $M_1 \times M_2 \times P$ 的输出。输出图像的空间分辨率和通道数都可能与输入不同。若该层单个卷积所用核的宽度为 $k = 2\bar{N} + 1$ ，则核形状为 $k \times k \times C$ ，即对输入图像的每个通道都有 $k \times k$ 个权重。每个卷积作用于输入后，得到形状为 $M_1 \times M_2 \times 1$ 的输出图像；把各卷积的输出堆叠起来，形成卷积层的最终输出。这可以写成

$$\bar{U}[i, j, k] = \sum_{m=-\bar{N}}^{\bar{N}} \sum_{n=-\bar{N}}^{\bar{N}} \sum_{c=1}^C g_k[m, n, c] U[i + m, j + n, c],$$

其中 $1 \leq i \leq M_1$ 、 $1 \leq j \leq M_2$ 、 $1 \leq k \leq P$ ，而 g_k 是该层第 k 个卷积的核。可训练参数总数为 $(2\bar{N} + 1)^2 PC$ 。这是卷积神经网络中最常见的一类卷积层，下一节将加以介绍。

5.6 卷积神经网络 (CNN)

现在把以上元素组合成完整网络。考虑一个图像分类问题，CNN 写作 $\mathbf{y} = \mathcal{F}(\mathbf{x}; \boldsymbol{\theta})$ ，其中 $\mathbf{x} \in \mathbb{R}^{N_1 \times N_2 \times N_3}$ 是具有 N_3 个通道的输入图像，而 $\mathbf{y} \in \mathbb{R}^C$ 是概率向量；其第 i 个分量表示输入图像属于总计 C 个类别中第 i 类的概率。源文随后称 $\mathbf{y}$ 通常采用独热编码。

一种可能的网络架构见图 5.7。它包含若干卷积层及随后的池化层，在降低输入图像空间分辨率的同时增加通道数。最后一个池化层的输出被展平 (flattening) 为向量，再送入若干采用某种激活函数 (例如 ReLU) 的全连接层。最后一个全连接层把向量大小降至 C (图中取 $C = 10$)，然后通过 softmax 函数生成预测概率向量 $\mathbf{y}$ 。因为求解的是分类问题，损失函数取交叉熵：

$$\Pi(\boldsymbol{\theta}) = - \sum_{i=1}^{N_{\text{train}}} \sum_{c=1}^C y_c^{(i)} \log(\mathcal{F}_c(\mathbf{x}^{(i)}; \boldsymbol{\theta})) .$$

通过寻找 $\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \Pi(\boldsymbol{\theta})$ 来训练 CNN，最终网络为 $\mathbf{y} = \mathcal{F}(\mathbf{x}; \boldsymbol{\theta}^*)$ 。

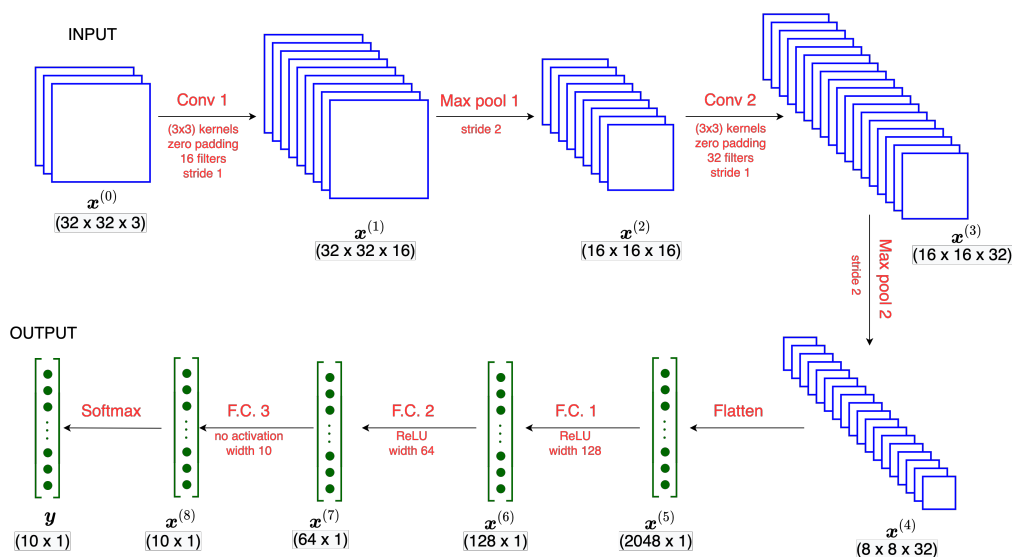


图 5.7: 用于十分类图像分类问题的一种 CNN 架构。

下面作几点重要说明：

1. 与全连接层一样，卷积运算也是输入上的线性运算。唯一区别在于：全连接层的权重把输出的每个像素连接到输入的每个像素，而卷积层的权重只把输出的一个像素连接到输入的一小块像素；此外，同一组权重会施加到输入的所有图像块上。
2. 在图 5.7 所示 CNN 中，可以把卷积层理解为编码输入图像的信息，把全连接层理解为使用这些信息求解分类问题。因此，文献中常说卷积层执行特征选择 (feature selection)。到“展平”向量为止的网络部分有时称为编码器 (encoder)。

3. 有时也会对卷积层输出加上偏置并施加激活函数：

$$x^{(l+1)}[i, j, k] = \sigma \left(\sum_{m, n, c} g_k[m, n, c] x^{(l)}[i + m, j + n, c] + b_k \right),$$

其中给定输出通道 k 使用单个偏置 b_k .

4. 上例考虑的是图像分类，即网络把图像变换为类别标签. 还可以考虑类似情形，例如把图像变换为实数，这在计算物理中有多种用途. 假设要构造一个涡量平方积分 (enstrophy) 计算器：网络输入为网格上定义的流体速度各分量图像，输出为整个区域上涡量平方的积分（称为 enstrophy）. 另一个例子是：网络输入弹性固体的位移分量，输出固体中储存的总应变能.
5. 以上架构把图像变换为向量，适用于图像分类和图像分析等问题. 另一类架构执行相反操作，即把向量映射为图像，适用于图像合成. 把两种架构组合起来，还可以先把图像变成向量，再变回另一幅图像. 这种图像到图像的变换适用于图像语义分割等任务. 后续各节将介绍这些思想.
6. 值得分析卷积层如何作用于图像以及它们为何如此有用. 在深度学习中处理图像输入时，一种朴素做法是把图像展平后送入普通全连接 MLP，但这会产生若干问题. 对一般图像，展平输入的维数极大；此时定义网络架构有两种选择：
- (a) 令网络最初几层的宽度与输入的巨大维数相当.
 - (b) 令第二个隐藏层的宽度急剧减小.

两种策略都会带来问题. 第一种使网络过大，可训练参数过多，需要不现实的数据量才能训练. 第二种使前两层之间的信息压缩过于激进，网络很难学会如何从不同图像中提取正确特征. 此外，展平图像会丢失像素之间的重要空间关系，例如边缘和形状. 理想情况下应尽量利用这些承载重要空间信息的关系.

卷积层可以同时解决这两个问题. 它允许输入保持为二维图像，并大幅减少特征提取任务所需的可学习参数. 与经典全连接层相比，卷积核引入的参数数目有限. 同一卷积核会用于图像中的不同像素位置，即参数共享 (parameter sharing)，因而能够高效而合理地利用计算资源.

5.7 转置卷积层

前面看到，卷积层和池化层可以缩小图像. 下面考虑执行相反操作、放大图像的层. 通过几个例子理解这种运算.

1. 考虑长度为 4 的一维图像：

$$\text{输入} = \begin{bmatrix} u_1 & u_2 & u_3 & u_4 \end{bmatrix}.$$

取尺寸为 3×1 的核

$$\mathbf{k} = \begin{bmatrix} x & y & z \end{bmatrix}.$$

考虑使用核 $\mathbf{k}$ 、步幅为 1、零填充层大小为 1 的卷积层. 该层作用于输入后得到

$$\text{输出} = \begin{bmatrix} yu_1 + zu_2 & xu_1 + yu_2 + zu_3 & xu_2 + yu_3 + zu_4 & xu_3 + yu_4 \end{bmatrix}.$$

卷积算子的步骤是填充、点积、按步幅移动. 填充和步幅 1 保证了输出与输入具有相同大小.

2. 再考虑使用同一核 $\mathbf{k}$ 、零填充层大小为 1、但步幅为 2 的卷积. 它作用于同一输入后得到

$$\text{输出} = \begin{bmatrix} yu_1 + zu_2 & xu_2 + yu_3 + zu_4 \end{bmatrix}.$$

输出大小缩小为原来的一半; 换言之, 增大步幅实现了输入的下采样 (downsampling). 我们自然会问, 能否以类似方式执行上采样 (upsampling)? 答案是肯定的, 只需把卷积层中的每种运算转置:

- 用外积代替点积 (内积).
- 不再跳过输入像素 (步幅), 而是跳过输出像素.
- 不再填充, 而是裁剪输出.

3. 下面看一个转置卷积层 (transpose convolution layer) 的例子. 考虑大小为 2×1 的一维输入图像

$$\text{输入} = \begin{bmatrix} u_1 & u_2 \end{bmatrix},$$

以及大小为 3×1 的核

$$\mathbf{k} = \begin{bmatrix} x & y & z \end{bmatrix}.$$

输入与 $\mathbf{k}$ 的外积为

$$\text{外积} = \begin{bmatrix} u_1x & u_1y & u_1z \\ u_2x & u_2y & u_2z \end{bmatrix}.$$

若步幅为 2, 需要把外积各行平移 2 格:

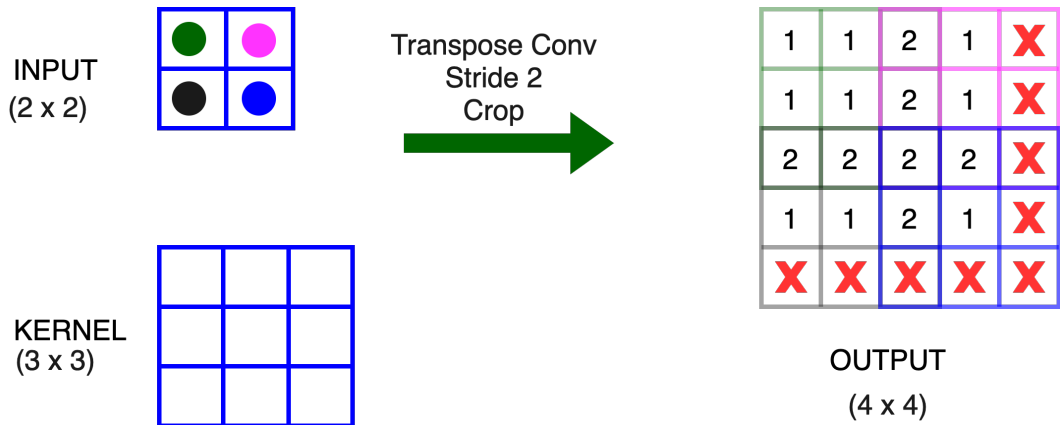
$$\text{按步幅平移} = \begin{bmatrix} u_1x & u_1y & u_1z & 0 & 0 \\ 0 & 0 & u_2x & u_2y & u_2z \end{bmatrix}.$$

平移后, 把各列条目相加并裁剪向量, 得到

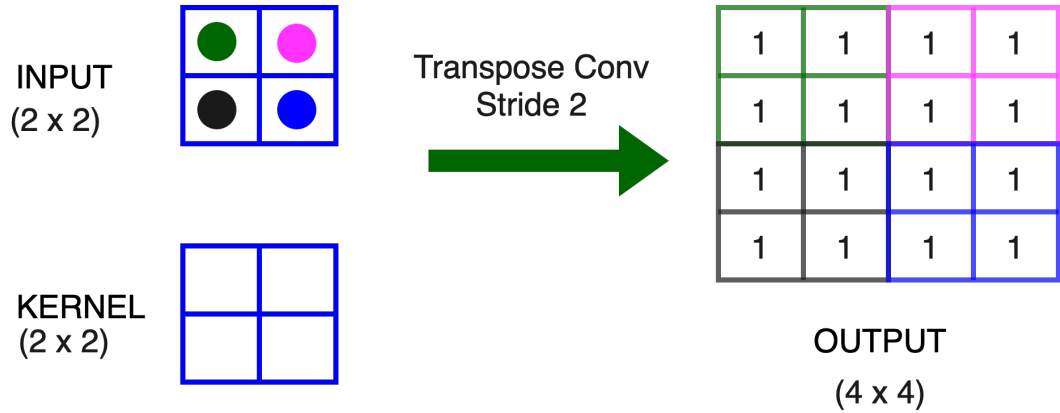
$$\begin{aligned} \text{输出} &= \text{Crop} \left(\begin{bmatrix} u_1x & u_1y & u_1z + u_2x & u_2y & u_2z \end{bmatrix} \right) \\ &= \begin{bmatrix} u_1x & u_1y & u_1z + u_2x & u_2y \end{bmatrix}. \end{aligned}$$

这里按照约定裁掉末尾元素, 使输出大小为输入的两倍.

- 考虑把二维转置卷积施加到大小为 2×2 的二维图像. 核形状为 3×3 , 步幅为 2, 并采用填充 (裁剪). 图 5.8(a) 展示了这一操作: 先得到大小为 5×5 的图像, 再裁剪为 4×4 的输出. 输出像素从各图像块得到的贡献数不相等 (图中数字所示), 导致不希望出现的棋盘格效应 (checker-boarding), 即输出图像的值在逐像素尺度上起伏.
- 避免棋盘格效应的一种方法, 是保证滤波器尺寸为步幅的整数倍. 用 2×2 核重复上例, 运算见图 5.8(b). 此时无需填充 (裁剪), 且每个输出像素得到相同数目的贡献.



(a) 核尺寸 3×3 , 出现棋盘格效应



(b) 核尺寸 2×2 , 无棋盘格效应

图 5.8: 转置卷积示例. 输出中标有红色叉号的单元被裁掉; 数字表示对相应输出像素值作出贡献的图像块数.

再作几点说明:

- 转置卷积层也称为分数步幅层 (fractionally-strided layer), 因为输入每移动一步, 输出会移动多于一步. 这与卷积层恰好相反: 卷积层在输出图像移动一个单位步时, 会在输入图像中移动大于一步.
- 转置卷积是通过可学习参数执行上采样的常用工具.
- 上采样通常伴随通道数减少, 这也与卷积层中的做法相反.

5.8 图像到图像的变换

图像到图像的变换可以类比函数到函数的变换. 这类网络通常用于计算机视觉、超分辨率和风格迁移, 也用于计算物理, 例如把源场 (PDE 右端) 映射到 PDE 解.

下面讨论一种用于此类变换的特定网络, 即 U-Net [26]. U-Net (见图 5.9) 具有一个向下分支, 它接收输入图像, 使用若干卷积层和池化运算逐步降低图像尺度; 沿该分支向下时, 通道数通常增加. 到达最粗尺度后, 向上分支使用转置卷积类运算放大图像并减少通道数, 最终给出输出图像. 除这两个分支外, U-Net 还使用跳跃连接, 把向下分支某一尺度的信息与向上分支同一尺度的信息组合起来. 这些连接与 ResNet 中的连接相似. 在 U-Net 的上采样分支中, 任一点的激活来自两个来源: 一部分来自 U-Net 下采样分支的同一空间尺度, 另一部分来自 U-Net 上采样分支的更粗尺度. 从总体结构看, 它属于编码器-解码器网络 (encoder-decoder network).

注 5.8.1: U-Net 架构与多重网格预条件器通常使用的 V 循环具有许多共同特征. ◇

注 5.8.2: 也可以把 U-Net 看成加入了额外跳跃连接的编码器-解码器网络. ◇

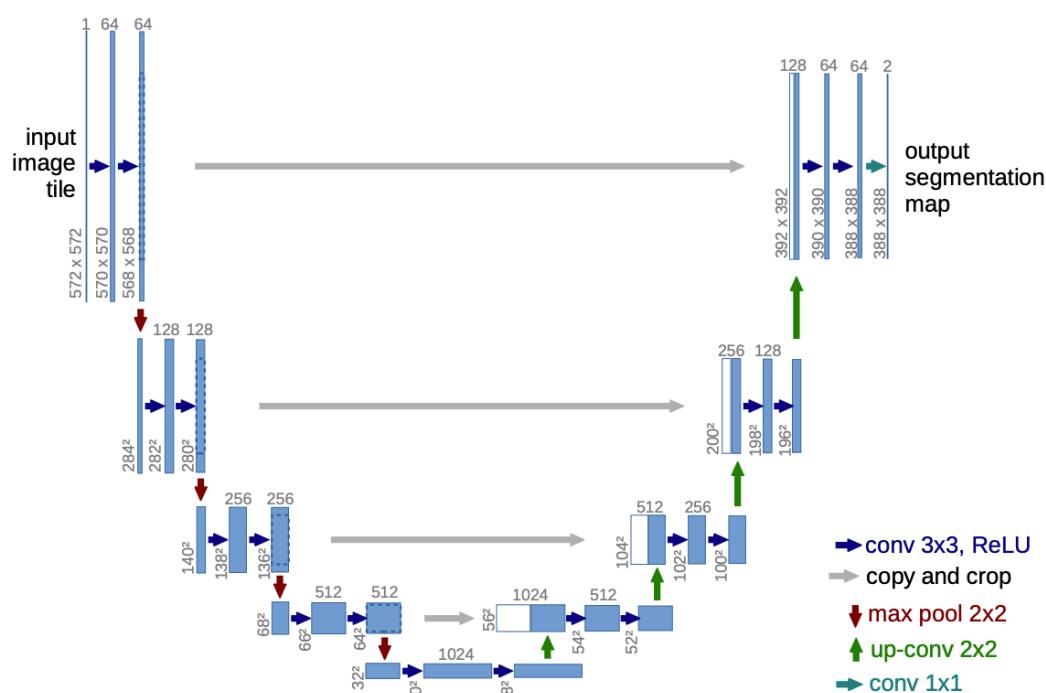


图 5.9: U-Net 示例, 引自文献 [26].

算子网络

6.1 PINN 的局限

典型 MLP $\mathbf{y} = \mathcal{F}(\mathbf{x}; \boldsymbol{\theta})$ 以 $\mathbf{x} \in \mathbb{R}^d$ 为输入并给出输出 $\mathbf{y} \in \mathbb{R}^d$, 其中 $\boldsymbol{\theta}$ 是可训练权重. 如第 4 章所述, PINN $u(\mathbf{x}) = \mathcal{F}(\mathbf{x}; \boldsymbol{\theta})$ 以 PDE 的自变量为输入, 以 PDE 的解为输出, 并通过最小化 PDE 残差与边界残差的加权和来训练. 然而, 这只给出某一组边界条件和源项下的一个解实例. 例如, 考虑

$$\begin{aligned} \nabla \cdot (\kappa \nabla u) &= f(\mathbf{x}), & \mathbf{x} \in \Omega &= [0, 1] \times [0, 1], \\ u(\mathbf{x}) &= g(\mathbf{x}), & \mathbf{x} \in \partial\Omega, \end{aligned} \quad (6.1)$$

并训练 $\mathcal{F}$ 以最小化

$$\Pi(\boldsymbol{\theta}) = \frac{1}{N_v} \sum_{i=1}^{N_v} |\nabla \cdot (\kappa \nabla \mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta})) - f(\mathbf{x}_i)|^2 + \frac{\lambda_b}{N_b} \sum_{i=1}^{N_b} |\mathcal{F}(\mathbf{x}_i; \boldsymbol{\theta}) - g(\mathbf{x}_i)|^2.$$

若 $\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \Pi(\boldsymbol{\theta})$, 则 $u(\mathbf{x}) = \mathcal{F}(\mathbf{x}; \boldsymbol{\theta}^*)$. 但若改变式 (6.1) 中的 f 或 g , 同一已训练网络并无理由仍然有效, 通常必须重新训练. 本章介绍避免这种繁琐做法的方法.

6.2 参数化 PDE

假设源项是参数函数 $f(\mathbf{x}; \alpha)$, 例如

$$f(x_1, x_2; \alpha) = 4\alpha x_1(1 - x_1)x_2(1 - x_2).$$

可采用同时输入 $\mathbf{x}$ 和 α 的网络 $\mathcal{F}(\mathbf{x}, \alpha; \boldsymbol{\theta})$, 如图 6.1, 并最小化

$$\begin{aligned} \Pi(\boldsymbol{\theta}) = \frac{1}{N_a} \sum_{j=1}^{N_a} \left[\frac{1}{N_v} \sum_{i=1}^{N_v} |\nabla \cdot (\kappa \nabla \mathcal{F}(\mathbf{x}_i, \alpha_j; \boldsymbol{\theta})) - f(\mathbf{x}_i, \alpha_j)|^2 \right. \\ \left. + \frac{\lambda_b}{N_b} \sum_{i=1}^{N_b} |\mathcal{F}(\mathbf{x}_i, \alpha_j; \boldsymbol{\theta}) - g(\mathbf{x}_i)|^2 \right]. \end{aligned}$$

构造损失时还须为 α 选取配置点. 若 $\boldsymbol{\theta}^*$ 使损失最小, 则 $u(\mathbf{x}, \alpha) = \mathcal{F}(\mathbf{x}, \alpha; \boldsymbol{\theta}^*)$; 对新值 $\hat{\alpha}$, 只需计算 $\mathcal{F}(\mathbf{x}, \hat{\alpha}; \boldsymbol{\theta}^*)$. 若能参数化 $\kappa(\mathbf{x})$ 和 $g(\mathbf{x})$, 也可采用同样方法. 若希望处理任意非参数化的 f , 则需要近似函数到函数的算子 (operator mapping functions to functions).

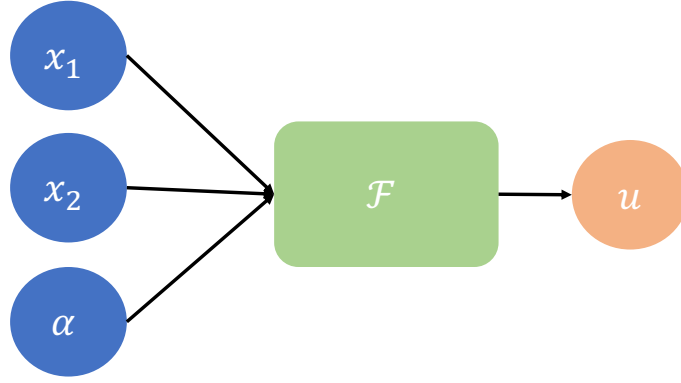


图 6.1: 具有参数化输入的 PINN 示意图.

6.3 算子

考虑函数类 $\mathbf{a}(\mathbf{y}) \in A$, 其中 $\mathbf{a} : \Omega_Y \rightarrow \mathbb{R}^D$, 并可能满足 $\mathbf{a} \in C(\Omega_Y)$ 或 $L^2(\Omega_Y)$. 再考虑 $\mathcal{N} : A \mapsto C(\Omega_X)$, 满足 $\mathbf{u}(\mathbf{x}) = \mathcal{N}(\mathbf{a})(\mathbf{x})$. 例子如下.

1. 对

$$\begin{aligned} \nabla \cdot (\kappa \nabla u) &= f, & \mathbf{x} \in \Omega, \\ u &= g, & \mathbf{x} \in \partial\Omega, \end{aligned} \quad (6.2)$$

$\Omega_X = \Omega_Y = \Omega$, $\mathcal{N}$ 把右端项 f 映射为解 (温度) $u = \mathcal{N}(f)(\mathbf{x})$, 其中 κ, g 固定.

2. 对同一 PDE, 若改变电导率场而固定 f, g , 则

$$\begin{aligned} \nabla \cdot (\kappa \nabla u) &= f, & \mathbf{x} \in \Omega, \\ u &= g, & \mathbf{x} \in \partial\Omega, \end{aligned} \quad (6.3)$$

且 $u = \mathcal{N}(\kappa)(\mathbf{x})$, $\Omega_X = \Omega_Y = \Omega$.

3. 若电导率和边界条件均可改变,

$$\begin{aligned} \nabla \cdot (\kappa \nabla u) &= f(\mathbf{x}), & \mathbf{x} \in \Omega, \\ u(\mathbf{x}) &= g(\mathbf{x}), & \mathbf{x} \in \partial\Omega, \end{aligned} \quad (6.4)$$

则 $u = \mathcal{N}(\kappa, g)(\mathbf{x})$. 输入是两个函数, 输出是一个函数, 故 $\Omega_X = \Omega$, $\Omega_Y = \Omega \times \partial\Omega$, 其中 f 固定.

4. 三维区域 $\Omega \subset \mathbb{R}^3$ 上的线性各向同性弹性方程为

$$\begin{aligned} \nabla \cdot (\lambda(\nabla \cdot \mathbf{u})\mathbf{I} + 2\mu\nabla^S \mathbf{u}) &= \mathbf{f}(\mathbf{x}), & \mathbf{x} \in \Omega, \\ \mathbf{u}(\mathbf{x}) &= \mathbf{g}(\mathbf{x}), & \mathbf{x} \in \partial\Omega. \end{aligned} \quad (6.5)$$

算子 $\mathbf{u}(\mathbf{x}) = \mathcal{N}(\mathbf{f})(\mathbf{x})$ 把 $\mathbf{f} : \Omega \rightarrow \mathbb{R}^3$ 映射为 $\mathbf{u} : \Omega \rightarrow \mathbb{R}^3$, 其中 $\lambda, \mu, \mathbf{g}$ 固定.

5. 对流-扩散-反应方程为

$$\begin{aligned}\frac{\partial u}{\partial t} + \mathbf{a} \cdot \nabla u - \kappa \nabla^2 u + u(1 - u) &= f, & (\mathbf{x}, t) \in \Omega \times (0, T], \\ u(\mathbf{x}, t) &= g(\mathbf{x}, t), & (\mathbf{x}, t) \in \partial\Omega \times (0, T], \\ u(\mathbf{x}, 0) &= u_0(\mathbf{x}), & \mathbf{x} \in \Omega.\end{aligned}\quad (6.6)$$

这里 $\mathcal{N}$ 把初始条件映射为终止时刻的解: $u(\mathbf{x}, T) = \mathcal{N}(u_0)(\mathbf{x})$, 且 $\Omega_X = \Omega_Y = \Omega$; $\mathbf{a}, \kappa, f, g$ 固定.

注 6.3.1: 判断算子是否线性通常很有用, 因为线性算子可由另一线性算子很好地近似. 上述例 1、4 的算子为线性, 例 2、3、5 为非线性. $\diamond$

近似 $\mathcal{N}$ 的网络通常称为算子网络 (operator network). 两种常见版本是深度算子网络 (Deep Operator Network, DeepONet) 和 Fourier 神经算子 (Fourier Neural Operator, FNO).

6.4 深度算子网络 (DeepONet) 架构

Chen 和 Chen [5] 最早提出单隐层算子网络; 文献 [14] 将其扩展到深层架构并称为 DeepONet. 考虑 $\mathcal{N}: A \rightarrow U$, 其中 $a: \Omega_Y \subset \mathbb{R}^d \rightarrow \mathbb{R}$, $u: \Omega_X \subset \mathbb{R}^D \rightarrow \mathbb{R}$, 并假设二者可逐点求值. 图 6.2 所示构造如下.

- 在 Ω_Y 固定 M 个互异传感器点 $\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(M)}$.
- 采样 a 得 $\mathbf{a} = [a(\mathbf{y}^{(1)}), \dots, a(\mathbf{y}^{(M)})]^\top \in \mathbb{R}^M$.
- 输入分支网络 (branch net) $\mathcal{B}: \mathbb{R}^M \rightarrow \mathbb{R}^p$, 得到 $\boldsymbol{\beta} = [\beta_1(\mathbf{a}), \dots, \beta_p(\mathbf{a})]^\top$; 其参数为 $\boldsymbol{\theta}_B$, 通常 $p \approx 100$.
- 把 $\mathbf{x}$ 输入主干网络 (trunk net) $\mathcal{T}: \mathbb{R}^D \rightarrow \mathbb{R}^p$, 得到 $\boldsymbol{\tau} = [\tau_1(\mathbf{x}), \dots, \tau_p(\mathbf{x})]^\top$; 其参数为 $\boldsymbol{\theta}_T$.
- 对两输出取点积:

$$u(\mathbf{x}) \approx \tilde{\mathcal{N}}(\mathbf{x}, \mathbf{a}; \boldsymbol{\theta}) = \sum_{k=1}^p \beta_k(\mathbf{a}) \tau_k(\mathbf{x}). \quad (6.7)$$

源 PDF 将合并参数写为 $\boldsymbol{\theta} = [\boldsymbol{\theta}_T, \boldsymbol{\theta}_B]$.

训练后, DeepONet 近似底层算子, 从而可对任意 $a \in A$ 、 $\mathbf{x} \in \Omega_X$ 近似 $\mathcal{N}(a)(\mathbf{x})$. M 个传感器点必须预先固定, 训练和求值时不能改变.

1. 式 (6.7) 像谱方法或有限元方法一样把解表示为系数与函数乘积之和; 但传统方法的基函数由用户预定, 而 DeepONet 的函数由主干网络和训练数据确定.
2. 随机采样输入函数时, 分支网络宜采用全连接层. 输入维数可达 $N_1 \approx 10^4$, 输出约 $p \approx 10^2$, 故可视为编码器. 均匀网格采样时宜采用卷积层, 把高维图像映射为低维潜向量.
3. 提高表达能力的两种办法是增加分支与主干网络的参数, 以及增加潜向量维数 p .

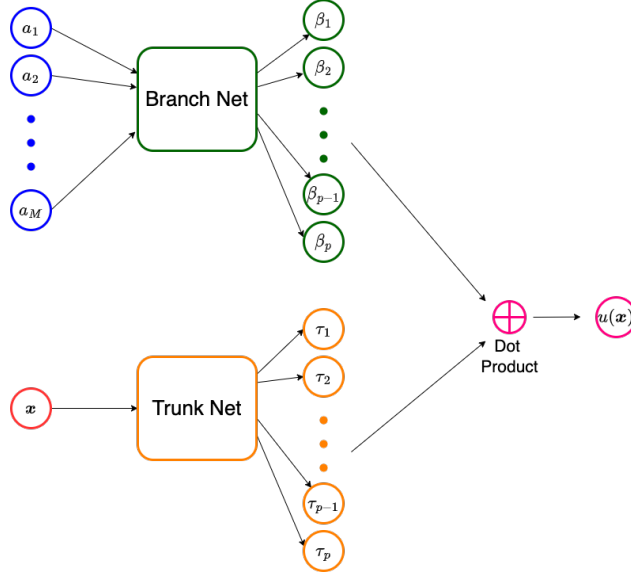


图 6.2: DeepONet 示意图.

6.5 训练 DeepONet

DeepONet 通常采用需要成对数据的监督训练.

1. 从 A 选取 N_1 个代表函数 $a^{(i)}$, 在 M 个传感器点求值, 得到

$$\mathbf{a}^{(i)} = [a^{(i)}(\mathbf{y}^{(1)}), \dots, a^{(i)}(\mathbf{y}^{(M)})]^\top.$$

2. 对每个 $a^{(i)}$, 以数值或解析方法确定 $u^{(i)} = \mathcal{N}(a^{(i)})$.

3. 在 Ω_X 的 N_2 个点取 $u^{(i)}(\mathbf{x}^{(k)})$.

4. 构造含 $N_1 N_2$ 个样本的训练集

$$\mathcal{S} = \{(\mathbf{a}^{(i)}, \mathbf{x}^{(k)}, u^{(i)}(\mathbf{x}^{(k)})) : 1 \leq i \leq N_1, 1 \leq k \leq N_2\}.$$

5. 定义

$$\Pi(\boldsymbol{\theta}) = \frac{1}{N_1 N_2} \sum_{i=1}^{N_1} \sum_{k=1}^{N_2} |\tilde{\mathcal{N}}(\mathbf{x}^{(k)}, \mathbf{a}^{(i)}; \boldsymbol{\theta}) - u^{(i)}(\mathbf{x}^{(k)})|^2.$$

6. 求 $\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \Pi(\boldsymbol{\theta})$.

7. 对新函数 a 和新点 $\mathbf{x}$, 计算 $u^*(\mathbf{x}) = \tilde{\mathcal{N}}(\mathbf{x}, \mathbf{a}; \boldsymbol{\theta}^*)$.

注 6.5.1: 不同 i 不必使用同一组 N_2 个输出点; 随机选点可得到更多样的数据集. $\diamond$

注 6.5.2: 输入可包含多个函数. 主干网络不变, 分支网络接收多个采样向量; 两个输入函数的情形见图 6.3. $\diamond$

注 6.5.3: 输出也可包含 D 个函数. 分支和主干网络各输出 D 个 p 维向量, 对各对向量取点积. 两输出情形见图 6.4; 源 PDF 正文误称图 6.3. $\diamond$

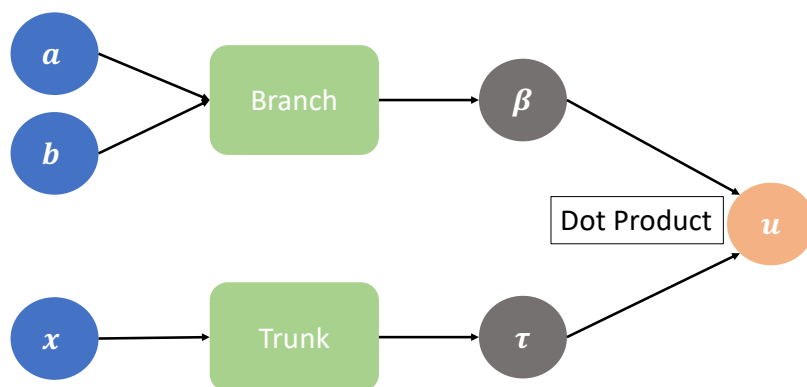


图 6.3: 具有两个输入函数的 DeepONet.

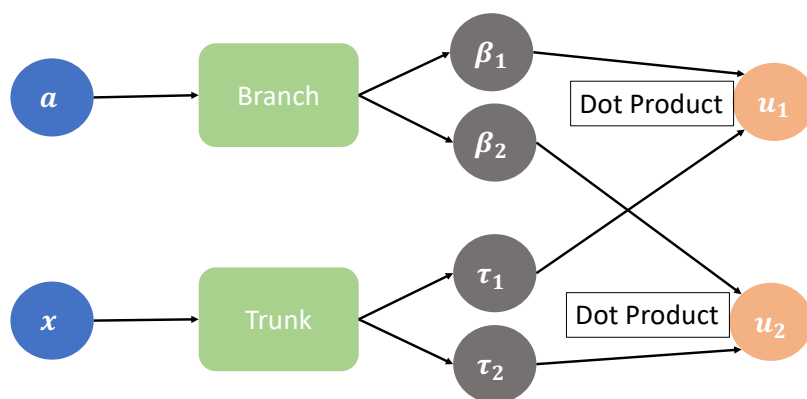


图 6.4: 具有两个输出函数的 DeepONet.

6.6 DeepONet 的误差分析

Chen 和 Chen [5] 给出了浅层 DeepONet 的通用逼近定理.

定理 6.6.1: 设 Ω_X 和 Ω_Y 分别是 $\mathbb{R}^D$ (或更一般的 Banach 空间) 和 $\mathbb{R}^d$ 中的紧集. 设 $\mathcal{N}$ 是把 $V \subset C(\Omega_Y)$ 映射到 $C(\Omega_X)$ 的非线性连续算子. 给定 $\epsilon > 0$, 存在一个分支、主干网络均有单隐层且宽度为 P 、具有 M 个传感器的 DeepONet, 使

$$\max_{\substack{\mathbf{x} \in \Omega_X \\ a \in A}} |\tilde{\mathcal{N}}(\mathbf{x}, \mathbf{a}; \boldsymbol{\theta}) - \mathcal{N}(a)(\mathbf{x})| < \epsilon,$$

只要 P, M 足够大.

该结果已推广到深层网络 [14], 又进一步去除了空间紧性假设 [12]. 文献 [20] 给出的误差估计为

$$\max_{a \in A} \|\tilde{\mathcal{N}}(\mathbf{x}, \mathbf{a}; \boldsymbol{\theta}) - \mathcal{N}(a)(\mathbf{x})\|_{L^2(\Omega)} \leq C(\epsilon_h + \sqrt{\epsilon_t} + \epsilon_s + M^{-\alpha_1} + N_2^{-\alpha_2}). \quad (6.8)$$

其中 ϵ_h 是生成近似目标解的数值求解误差, ϵ_t 是最终训练误差, ϵ_s 衡量训练函数 $\{a^{(i)}\}_{i=1}^{N_1}$ 覆盖输入空间 A 的程度; 最后两项是有限输入传感器与输出节点产生的离散化或求积误差. 这类似于式 (4.21) 的 PINN 误差估计.

6.7 物理信息 DeepONet

假设输入 a 与输出 u 满足 PDE, 例如

$$\begin{aligned} \nabla \cdot (\kappa u) &= f && \text{于 } \Omega, \\ u &= g && \text{于 } \partial\Omega. \end{aligned} \quad (6.9)$$

其中 κ, g 已知. 除了在外部求解 PDE 以生成数据, 还可加入 PINN 型损失; 这就是物理信息 DeepONet (physics-informed DeepONet) [30]. 物理损失为

$$\Pi_p(\boldsymbol{\theta}) = \frac{1}{\bar{N}_1 \bar{N}_2} \sum_{i=1}^{\bar{N}_1} \sum_{k=1}^{\bar{N}_2} |\nabla_{\mathbf{x}} \cdot (\kappa \nabla_{\mathbf{x}} \tilde{\mathcal{N}}(\mathbf{x}^{(k)}, \mathbf{f}^{(i)}; \boldsymbol{\theta})) - f^{(i)}(\mathbf{x}^{(k)})|^2. \quad (6.10)$$

标准数据驱动损失为

$$\Pi_d(\boldsymbol{\theta}) = \frac{1}{N_1 N_2} \sum_{i=1}^{N_1} \sum_{k=1}^{N_2} |\tilde{\mathcal{N}}(\mathbf{x}^{(k)}, \mathbf{f}^{(i)}; \boldsymbol{\theta}) - u^{(i)}(\mathbf{x}^{(k)})|^2, \quad (6.11)$$

总损失为

$$\Pi(\boldsymbol{\theta}) = \Pi_d(\boldsymbol{\theta}) + \lambda \Pi_p(\boldsymbol{\theta}), \quad (6.12)$$

其中 λ 是超参数.

1. 物理损失和数据损失的输出点通常不同: 前者是最小化 PDE 残差的位置, 共 $\bar{N}_2$ 个; 后者是外部可得解的位置, 共 N_2 个.

2. 两种损失的输入函数集通常也不同：物理部分有 $\bar{N}_1$ 个，数据部分的函数总数在源 PDF 中写为 $N - 1$ 。

训练仍求 $\theta^* = \arg \min \Pi$. 物理损失可减少生成 PDE 解所需的数据量，并提高网络对训练数据未包含的输入函数类型的稳健性。

6.8 Fourier 神经算子——架构

下面介绍 Fourier 神经算子 (FNO) [13]. 我们从把标量映射为标量的典型前馈 MLP 出发，逐步把它扩展为从标量值函数到标量值函数的映射. 图 6.5 只展示前向传

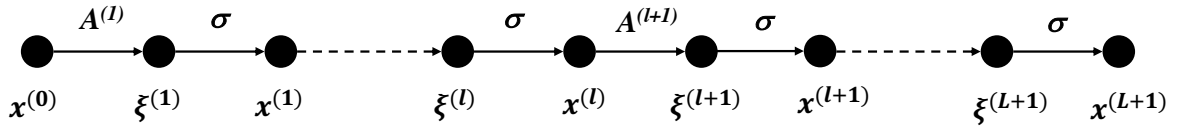


图 6.5: 前馈 MLP 的计算图.

播. 为简单起见，输入 $x^{(0)} = x$ 与输出 $x^{(L)} = y$ 均为标量；除 $\xi^{(L+1)}$ 外，所有隐藏变量都是 H 维向量，即每层宽度为 H 。

第一步把输入和输出替换为函数： $a(x) : \Omega \mapsto \mathbb{R}^1$ 与 $u(x) : \Omega \mapsto \mathbb{R}^1$ ，得到图 6.6. MLP 中的 H 维隐藏向量在 FNO 中成为 H 分量函数：

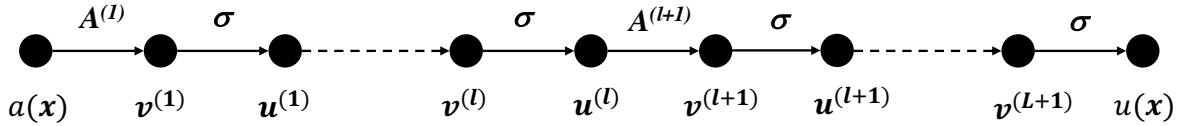


图 6.6: 前馈 Fourier 神经算子网络的计算图.

$$v^{(1)}, \dots, v^{(L)}, u^{(1)}, \dots, u^{(L)} : \Omega \mapsto \mathbb{R}^H. \quad (6.13)$$

图 6.6 中 $v^{(n)}$ 、 $u^{(n)}$ 分别对应 $\xi^{(n)}$ 、 $x^{(n)}$ ；因 $\xi^{(L+1)}$ 为标量，令 $v^{(L+1)}$ 为标量值函数. 接着把 MLP 中的向量算子扩展为函数算子.

$A^{(1)}$ 从单分量向量仿射映射到 H 分量向量，其函数版本为

$$v^{(1)}(x) = A^{(1)}(a)(x), \quad (6.14)$$

其中

$$v_i^{(1)}(x) = W_i^{(1)} a(x) + b_i^{(1)}, \quad i = 1, \dots, H. \quad (6.15)$$

$W_i^{(1)}, b_i^{(1)}$ 是该层的权重与偏置. 类似地， $A^{(L+1)}$ 的扩展为

$$v^{(L+1)}(x) = A^{(L+1)}(u^{(L)})(x), \quad (6.16)$$

其中

$$v^{(L+1)}(\mathbf{x}) = W_i^{(L+1)} u_i^{(L)}(\mathbf{x}) + b^{(L+1)}, \quad i = 1, \dots, H. \quad (6.17)$$

激活函数逐点作用：

$$\mathbf{u}^{(n)}(\mathbf{x}) = \sigma(\mathbf{v}^{(n)})(\mathbf{x}), \quad (6.18)$$

即

$$u_i^{(n)}(\mathbf{x}) = \sigma(v_i^{(n)}(\mathbf{x})), \quad i = 1, \dots, H. \quad (6.19)$$

最后，对 $n = 2, \dots, L$ 定义

$$\mathbf{v}^{(n+1)}(\mathbf{x}) = A^{(n+1)}(\mathbf{u}^{(n)})(\mathbf{x}), \quad (6.20)$$

其中

$$v_i^{(n+1)}(\mathbf{x}) = W_{ij}^{(n+1)} u_j^{(n)}(\mathbf{x}) + b_i^{(n+1)}, \quad (6.21)$$

$$+ \int_{\Omega} \kappa_{ij}^{(n+1)}(\mathbf{y} - \mathbf{x}) u_j^{(n)}(\mathbf{y}) d\mathbf{y}, \quad i = 1, \dots, H. \quad (6.22)$$

上式隐含对哑指标 $j = 1, \dots, H$ 求和. 新项是卷积，其动机是许多线性算子都能表示成卷积，例如把线性 PDE 的右端项（强迫函数）映射为解的 Green 算子. $\kappa_{ij}^{(n+1)}(\mathbf{z})$ 是卷积核，每层有 H^2 个核函数.

令 $\Omega = [0, L_1] \times [0, L_2]$ ，二维卷积可写成

$$v_i(x_1, x_2) = \int_0^{L_1} \int_0^{L_2} \kappa_{ij}(y_1 - x_1, y_2 - x_2) u_j(y_1, y_2) dy_2 dy_1, \quad i = 1, \dots, H. \quad (6.23)$$

这里省略了与讨论无关的上标.

注 6.8.1: FNO 可理解为仿射变换和卷积之后接逐点非线性激活的序列. 线性与非线性运算的组合使该架构能近似非线性算子. $\diamond$

注 6.8.2: FNO 的可训练参数为

$$W_i^{(1)}, W_{ij}^{(2)}, \dots, W_{ij}^{(L)}, W_i^{(L+1)}; \quad b_i^{(1)}, b_i^{(2)}, \dots, b_i^{(L)}, b^{(L+1)}, \quad (6.24)$$

可训练核函数为

$$\kappa_{ij}^{(n)}(\mathbf{z}), \quad n = 2, \dots, L. \quad (6.25)$$

$\diamond$

该神经算子直接把函数变换为函数；在计算机上实现时则必须离散表示函数.

6.9 Fourier 神经算子的离散化

上一节出现的函数为

$$a, \mathbf{v}^{(1)}, \mathbf{u}^{(1)}, \dots, \mathbf{v}^{(L)}, \mathbf{u}^{(L)}, v^{(L+1)}, u. \quad (6.26)$$

它们都定义在 Ω 上. 用 N 个均匀点离散该区域, 并用函数在这些点上的值表示函数. 二维区域 $\Omega = [0, L_1] \times [0, L_2]$ 上,

$$a[m, n] = a(x_{1m}, x_{2n}), \quad m = 1, \dots, N_1, \quad n = 1, \dots, N_2, \quad (6.27)$$

其中

$$x_{1m} = (m-1) \frac{L_1}{N_1-1}, \quad (6.28)$$

$$x_{2n} = (n-1) \frac{L_2}{N_2-1}. \quad (6.29)$$

第二式的 x_{1n} 按源 PDF 保留. 其他函数采用同样表示. $A^{(1)}$ 的离散形式为

$$\mathbf{v}^{(1)}[m, n] = A^{(1)}(a)[m, n], \quad (6.30)$$

其中

$$v_i^{(1)}[m, n] = W_i^{(1)} a[m, n] + b_i^{(1)}, \quad i = 1, \dots, H. \quad (6.31)$$

$A^{(L+1)}$ 的离散形式为

$$v^{(L+1)}[m, n] = A^{(L+1)}(\mathbf{u}^{(L)})[m, n], \quad (6.32)$$

其中

$$v^{(L+1)}[m, n] = W_i^{(L+1)} u_i^{(L)}[m, n] + b^{(L+1)}, \quad i = 1, \dots, H. \quad (6.33)$$

离散函数上的激活为

$$\mathbf{u}^{(n)}[m, n] = \sigma(\mathbf{v}^{(n)})[m, n], \quad (6.34)$$

其中

$$u_i^{(n)}[m, n] = \sigma(v_i^{(n)}[m, n]), \quad i = 1, \dots, H. \quad (6.35)$$

最后, 对 $n = 2, \dots, L$,

$$\mathbf{v}^{(p+1)}[m, n] = A^{(p+1)}(\mathbf{u}^{(p)})[m, n], \quad (6.36)$$

其中

$$v_i^{(p+1)}[m, n] = W_{ij}^{(p+1)} u_j^{(p)}[m, n] + b_i^{(p+1)}, \quad (6.37)$$

$$+ \sum_{r=1}^{N_1} \sum_{s=1}^{N_2} \kappa_{ij}^{(p+1)} [r-m, s-n] u_j^{(p)}[r, s] h_1 h_2, \quad i = 1, \dots, H. \quad (6.38)$$

其中 $h_1 = L_1/(N_1-1)$ 、 $h_2 = L_2/(N_2-1)$. 积分已变为全部网格点上的求和. 对每个 i, m, n 计算它需 $O(N_1 N_2 H)$ 次浮点运算. 源 PDF 随后称, 对 H 个 i 、 N_1 个 M 值和 N_2 个 j 值, 总成本为 $O(N_1^2 N_2^2 H) = O(N^2 H^2)$, 其中 $N = N_1 N_2$. N^2 因子使直接实现不实用; 正、逆 Fourier 变换可克服这一瓶颈, 故称 Fourier 神经算子.

6.10 Fourier 变换的使用

考虑定义于 $\Omega = [0, L_1] \times [0, L_2]$ 的周期函数 $u(x_1, x_2)$. 若足够光滑, 可用截断 Fourier 级数近似:

$$u(x_1, x_2) \approx \sum_{m=-N_1/2}^{N_1/2} \sum_{n=-N_2/2}^{N_2/2} \hat{u}[m, n] \exp \left[2\pi i \left(\frac{mx_1}{L_1} + \frac{nx_2}{L_2} \right) \right]. \quad (6.39)$$

N_1, N_2 为偶数, $\hat{u}[m, n]$ 为 Fourier 系数, $i = \sqrt{-1}$. 虽然 u 为实值, 系数却为复数, 并满足 $\hat{u}[-m, -n] = \hat{u}^*[m, n]$. 增大 N_1, N_2 可提高精度, 二者趋于无穷时恢复等式. 上式称为逆 Fourier 变换, 因为它把系数映射到物理空间函数.

正 Fourier 变换可由以下步骤得到.

1. 两边乘 $\exp[-2\pi i(rx_1/L_1 + sx_2/L_2)]$, 其中 r, s 为整数.
2. 两边在 Ω 上积分.
3. 利用 $\int_{\Omega} \exp[2\pi i((m-r)x_1/L_1 + (n-s)x_2/L_2)] dx_1 dx_2$ 仅在 $m=r, n=s$ 时非零, 且此时等于 $L_1 L_2$.

得到

$$\hat{u}[r, s] = \frac{1}{L_1 L_2} \int_0^{L_1} \int_0^{L_2} u(x_1, x_2) \exp \left[-2\pi i \left(\frac{rx_1}{L_1} + \frac{sx_2}{L_2} \right) \right] dx_1 dx_2. \quad (6.40)$$

为高效计算式 (6.23), 代入 u_j 的 Fourier 级数, 得

$$\begin{aligned} v_i(x_1, x_2) &= \int_0^{L_1} \int_0^{L_2} \kappa_{ij}(y_1 - x_1, y_2 - x_2) \sum_{m,n} \hat{u}_j[m, n] e^{2\pi i(my_1/L_1 + ny_2/L_2)} dy_2 dy_1 \\ &= \sum_{m,n} \hat{u}_j[m, n] \int_0^{L_1} \int_0^{L_2} \kappa_{ij}(y_1 - x_1, y_2 - x_2) e^{2\pi i(my_1/L_1 + ny_2/L_2)} dy_2 dy_1 \\ &= \sum_{m,n} \hat{u}_j[m, n] \int_{-x_1}^{L_1-x_1} \int_{-x_2}^{L_2-x_2} \kappa_{ij}(z_1, z_2) e^{2\pi i(m(z_1+x_1)/L_1 + n(z_2+x_2)/L_2)} dz_2 dz_1 \\ &= \sum_{m,n} \hat{u}_j[m, n] e^{2\pi i(mx_1/L_1 + nx_2/L_2)} \int_0^{L_1} \int_0^{L_2} \kappa_{ij}(z_1, z_2) e^{2\pi i(mz_1/L_1 + nz_2/L_2)} dz_2 dz_1 \\ &= L_1 L_2 \sum_{m,n} \hat{u}_j[m, n] \hat{\kappa}_{ij}[-m, -n] e^{2\pi i(mx_1/L_1 + nx_2/L_2)}. \end{aligned} \quad (6.41)$$

第一步把与 y_1, y_2 无关的系数和求和移到积分外; 第二步令 $z_1 = y_1 - x_1, z_2 = y_2 - x_2$; 第三步利用核的周期性; 最后使用式 (6.40). 因而卷积可按以下步骤计算.

1. 计算 u_j 的 Fourier 变换.
2. 计算 κ_{ij} 的 Fourier 变换.
3. 将两者的变换系数相乘.

4. 计算乘积的逆 Fourier 变换.

对离散函数, 在有限网格点计算式 (6.39), 并近似式 (6.40) 的积分:

$$\hat{u}[r, s] = \frac{h_1 h_2}{L_1 L_2} \sum_{m=1}^{N_1} \sum_{n=1}^{N_2} u[m, n] \exp \left[-2\pi i \left(\frac{r x_{1m}}{L_1} + \frac{s x_{2n}}{L_2} \right) \right]. \quad (6.42)$$

这里 $h_1 = L_1/N_1$ 、 $h_2 = L_2/N_2$, $x_{1m} = (m-1)h_1$ 、 $x_{2n} = (n-1)h_2$. 直接计算式 (6.39) 和 (6.42) 需要 $O(N^2)$ 次运算, 只有 N 很小时才实用. 快速 Fourier 变换 (FFT) 把成本降到 $O(N \log N)$, 从而使卷积成本成为 $O(N \log N H^2)$, 令 FNO 的实现切实可行.

概率深度学习

此前我们考虑的是回归与分类问题：给定输入 x ，需要计算单个输出 y 。但许多问题并非如此；给定一个 x ，可能存在多个 y 。例如：

1. y 和 x 的测量可能带有随机噪声。
2. y 和 x 可能本质上具有随机性。例如， y 可以是湍流中空间点 x 处测得的压力。
3. 反问题可能有多个解。例如，正问题是在已知热导率时确定温度场，而反问题是在已知可能含噪的温度场时确定热导率场。

因此，需要建立概率框架，以用深度学习算法求解这类问题。确定性模型为 $y = \mathcal{F}(x; \theta)$ ；在概率设置中， y 、 x 和 θ 都视为随机变量。在使用随机变量之前，先介绍定义随机变量所需的概率论基本要素。

7.1 概率论的基本要素

随机试验 (random experiment) 由一个过程和一项或多项观察或测量描述。例如：

1. 观察开关并判断它是开还是关。
2. 抛硬币 3 次，记录正面 H 或反面 T 的序列。
3. 抛硬币 3 次，计数 H 出现的次数。试验与上一项相同，但测量不同。
4. 转动转盘，以弧度测量最终角度。

结果 (outcome) 是试验中不能再分解为更小部分的结果。样本空间 (sample space) 记作 $\mathcal{S}$ ，是试验所有可能结果的集合。上述四个试验分别有

1. $\mathcal{S} = \{\text{开}, \text{关}\}$ 。
2. $\mathcal{S} = \{TTT, TTH, THT, HTT, THH, HTH, HHT, HHH\}$ 。
3. $\mathcal{S} = \{0, 1, 2, 3\}$ 。
4. $\mathcal{S} = \{\psi : \psi \in (0, 2\pi]\}$ 。

前三个试验的 $\mathcal{S}$ 离散且可数，而最后一个试验的 $\mathcal{S}$ 不可数，这是一个根本差别。

事件 (event) 是若干结果的集合，即 $\mathcal{S}$ 的子集；事件中的结果通常满足某个条件。对上述试验，例如：

1. $A = \{\text{开}\}$, 或 $A = \{\text{开}, \text{关}\} = \mathbb{S}$.
2. A 是至少出现 2 次 H 的结果, 即 $A = \{THH, HTH, HHT, HHH\}$.
3. A 是至少出现 2 次 H 的结果, 即 $A = \{2, 3\}$. 若 B 表示出现 4 次 H , 则没有结果满足条件, 故 $B = \emptyset$ 是空集.
4. A 是 $\psi > \pi/4$ 的结果, 即 $A = \{\psi : \psi \in (\pi/4, 2\pi]\}$.

事件类 (event class) $\mathcal{E}$ 是所有可定义概率的事件集合. 当 $\mathbb{S}$ 可数时, $\mathcal{E}$ 是 $\mathbb{S}$ 的全部子集; 当 $\mathbb{S}$ 不可数时, 源文称 $\mathcal{E}$ 为 Borel 域 (或 Borel 代数), 即 $\mathbb{S}$ 中所有开集与闭集的集合.

概率律 (probability law) 是给事件类 $\mathcal{E}$ 中每个集合赋予概率的规则. 概率律必须满足以下概率公理. 给定试验的样本空间 $\mathbb{S}$ 和事件类 $\mathcal{E}$, 令 $P : \mathcal{E} \mapsto [0, 1]$ 满足

1. 对所有 $A \in \mathcal{E}$, $P[A] \geq 0$.
2. $P[\mathbb{S}] = 1$.
3. 若 $A_1, A_2, \dots$ 两两互斥, 即对 $i \neq j$ 有 $A_i \cap A_j = \emptyset$, 则

$$P\left[\bigcup_{i=1}^{\infty} A_i\right] = \sum_{i=1}^{\infty} P[A_i].$$

满足这些条件的赋值 P 称为有效概率律. 概率类似质量: 它非负 (公理 1), 守恒 (总质量恒为 1, 公理 2), 且不同点的总质量是各自质量之和 (公理 3).

若 $\mathbb{S}$ 可数, 只需为每个基本结果定义非负且总和为 1 的概率. 对前三个例子, 可定义:

1. 对某个 $p \in [0, 1]$, $P[\text{开}] = p$, $P[\text{关}] = 1 - p$.
2. 对无记忆的公平骰子, 源文写 $P[a_i] = 1/8$, 其中 $a_i \in \mathbb{S}$, $i = 1, \dots, 8$.
3. 对无记忆的公平骰子, 源文写 $P[0] = 1/8$, $P[1] = 3/8$, $P[2] = 3/8$, $P[3] = 1/8$.

注 7.1.1: 作为练习, 请验证这些情形分别满足概率公理. ◇

对于连续样本空间, 只需在保证公理 1、2 的同时, 为所有开、闭区间定义概率律. 考虑第四个例子. 若转盘完全无偏, 则概率均匀分布; 对 $b \geq a$, 定义 $P[(a, b]] = (b-a)/(2\pi)$. 连续样本空间中单点集的概率为零, 对任何分布均如此.

注 7.1.2: 此后提到样本空间 $\mathbb{S}$ 时, 默认指三元组 $(\mathbb{S}, \mathcal{E}, P)$. 该三元组也称为“测度空间”. ◇

7.2 随机变量

随机变量 (random variable) X 是从 $\mathbb{S}$ 到实数轴的函数, 并要求对每个 $b \in \mathbb{R}$, 集合 $A_b = \{\xi \in \mathbb{S} : X(\xi) \leq b\}$ 都属于 $\mathcal{E}$. 用测度论语言说, 这要求 X 是可测函数; 按照 A_b 的定义, 还要求对所有 b 都能计算 $P[A_b]$.

为上述例子定义随机变量 (RV):

1. 对 $\mathbb{S} = \{\text{开}, \text{关}\}$, 且 $P[\text{开}] = p$, $P[\text{关}] = 1 - p$, 定义

$$X(\xi) = \begin{cases} 0, & \xi = \text{关}, \\ 1, & \xi = \text{开}. \end{cases} \quad (7.1)$$

这也称为 Bernoulli 随机变量 (Bernoulli random variable) .

2. 对三次抛硬币的八个等概率结果, 定义

$$X(\xi) = \xi \text{ 中正面的个数}. \quad (7.2)$$

这正是试验 3 所描述的随机事件.

3. 该随机事件本身已经是随机变量.

4. 对转盘试验, 定义

$$X(\psi) = \frac{\psi}{2\pi}. \quad (7.3)$$

定义在离散样本空间上的 X 称为离散随机变量, 定义在连续样本空间上的称为连续随机变量. 随机变量从定义它的测度空间继承概率解释, 下面定义这种解释.

7.2.1 累积分布函数

随机变量 X 的累积分布函数 (cumulative distribution function, cdf) 为

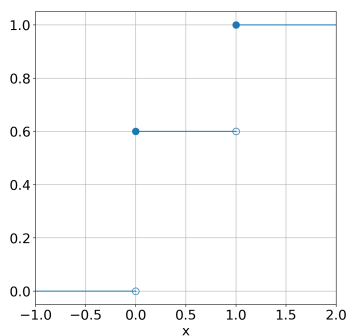
$$F_X(x) = P[\xi : X(\xi) \leq x],$$

它定义了 X 取值于 $(-\infty, x]$ 的概率. 对上述例子:

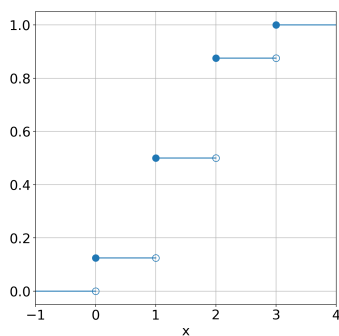
1. 对式 (7.1) 的 Bernoulli 随机变量, $x < 0$ 时 $F_X(x) = 0$; $0 \leq x < 1$ 时 $F_X(x) = 1 - p$; $x \geq 1$ 时 $F_X(x) = 1$. 见图 7.1(a).
2. 对式 (7.2), $x < 0$ 时为 0; $0 \leq x < 1$ 时为 $1/8$; $1 \leq x < 2$ 时为 $4/8$; 源文写 $3 \leq x < 3$ 时为 $7/8$; $x \geq 3$ 时为 1. 见图 7.1(b).
3. 该随机变量与例 2 相同.
4. 对式 (7.3),

$$F_X(x) = P[\psi : X(\psi) \leq x] = P[\{\psi : \psi \leq 2\pi x\}].$$

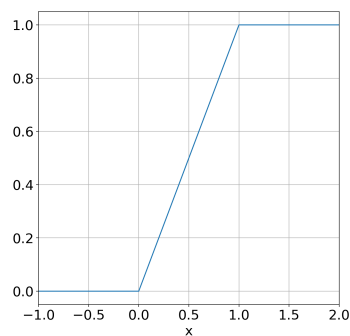
$x < 0$ 时为 0; $0 \leq x < 1$ 时为 x ; $x \geq 1$ 时为 1. 见图 7.1(c).



(a) Bernoulli



(b) 三次抛硬币



(c) 转盘

图 7.1: 累积分布函数示例.

F_X 具有以下性质:

1. $0 \leq F_X(x) \leq 1$.
2. $\lim_{x \rightarrow \infty} F_X(x) = 1$.
3. $\lim_{x \rightarrow -\infty} F_X(x) = 0$.
4. F_X 单调递增.
5. cdf 总是右连续:

$$F_X(x) = \lim_{h \rightarrow 0+} F_X(x+h).$$

离散随机变量的 cdf 在有限多个 x 处不连续, 可写成

$$F_X(x) = \sum_{k=1}^K p_k H(x - x_k), \quad \sum_{k=1}^K p_k = 1,$$

其中 p_k 是概率质量, H 是 Heaviside 函数:

$$H(x) = \begin{cases} 1, & x > 0, \\ 0, & x \leq 0. \end{cases}$$

注 7.2.1: 由 F_X 可计算 X 落在任意区间中的概率. 因互斥事件概率可加,

$$\begin{aligned} F_X(b) &= P[X \leq b] \\ &= P[X \leq a] + P[a < X \leq b] \\ &= F_X(a) + P[a < X \leq b], \end{aligned}$$

故 $P[a < X \leq b] = F_X(b) - F_X(a)$.

◇

7.2.2 概率密度函数

对连续的 F_X , 概率密度函数 (probability density function, pdf) 定义为

$$f_X(x) = \frac{d}{dx} F_X(x). \quad (7.4)$$

它继承 cdf 的以下性质:

1. 对所有 $x \in \mathbb{R}$, $f_X(x) \geq 0$.
2. $\lim_{x \rightarrow -\infty} f_X(x) = \lim_{x \rightarrow \infty} f_X(x) = 0$.
3. $\int_{-\infty}^x f_X(y) dy = F_X(x)$.
4. $P[a < X \leq b] = \int_a^b f_X(y) dy$. pdf 在区间上的积分给出随机变量落在其中的概率质量, 这正是它称为“密度”的原因.
5. $\int_{-\infty}^{\infty} f_X(y) dy = 1$.
6. 对很小的 $h > 0$, $P[a < X \leq a + h] \approx h f_X(a)$. 当 $h \rightarrow 0^+$ 时该概率趋于零, 即连续随机变量取单个值的概率为零.

7.2.3 重要随机变量示例

以下是几种重要随机变量及其 cdf、pdf (见图 7.2) .

1. 均匀随机变量 (uniform random variable): 在 $(a, b]$ 上,

$$f_X(x) = \begin{cases} (b-a)^{-1}, & x \in (a, b], \\ 0, & \text{其他,} \end{cases}$$
$$F_X(x) = \begin{cases} 0, & x < a, \\ (x-a)/(b-a), & x \in (a, b], \\ 1, & x > b. \end{cases}$$

2. 指数随机变量 (exponential random variable) 用于刻画临界事件后设备或人的寿命. X 表示失效时间, $P[X > x] = e^{-\lambda x}$, 其中 $\lambda > 0$ 是失效率, 因此

$$F_X(x) = 1 - e^{-\lambda x}, \quad f_X(x) = \lambda e^{-\lambda x}.$$

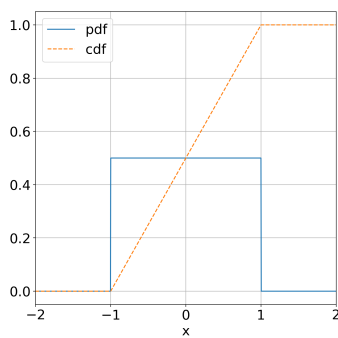
3. Gaussian 随机变量 (Gaussian random variable) 用于刻画身高、体重等自然量; 由中心极限定理, 许多随机变量的聚合也服从这种分布. 其 pdf 为

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp \left[-\frac{1}{2} \left(\frac{x-\mu}{\sigma} \right)^2 \right],$$

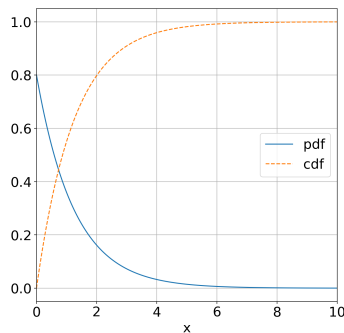
其中均值 μ 表示分布中心, 方差 σ^2 表示离散程度. 相应 cdf 为

$$F_X(x) = \frac{1}{2} \left[1 + \operatorname{erf} \left(\frac{x-\mu}{\sigma\sqrt{2}} \right) \right], \quad \operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt.$$

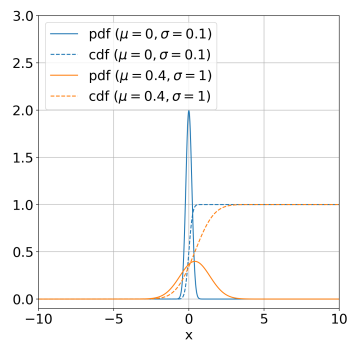
概率机器学习大量使用均匀随机变量和 Gaussian 随机变量.



(a) 均匀随机变量
($a = -1, b = 1$)



(b) 指数随机变量 ($\lambda = .8$)



(c) Gaussian 随机变量

图 7.2: 连续随机变量.

7.2.4 随机变量的期望与方差

给定 pdf 为 f_X 的随机变量 X , 其期望值 (expected value, expectation, mean) 定义为

$$\mu_X := \mathbb{E}[X] = \int_{-\infty}^{\infty} x f_X(x) dx.$$

期望具有以下性质:

- 若 pdf 关于 $x = m$ 对称, 则 $\mathbb{E}[X] = m$. 因为 $(m - x)f_X(x)$ 关于 m 反对称,

$$\begin{aligned} 0 &= \int_{-\infty}^{\infty} (m - x) f_X(x) dx \\ &= m \int_{-\infty}^{\infty} f_X(x) dx - \int_{-\infty}^{\infty} x f_X(x) dx, \end{aligned}$$

故 $\int x f_X(x) dx = m$. 因此, 均匀随机变量的均值为 $(a + b)/2$, Gaussian 随机变量的均值为 μ .

- 对常数 c , $\mathbb{E}[c] = c$.
- 随机变量函数的期望为

$$\mathbb{E}[g(X)] = \int_{-\infty}^{\infty} g(x) f_X(x) dx.$$

- 期望是线性的: $\mathbb{E}[g(X) + ch(X)] = \mathbb{E}[g(X)] + c\mathbb{E}[h(X)]$.

随机变量的方差 (variance) 度量它相对于均值的变化:

$$\text{VAR}[X] = \int_{-\infty}^{\infty} (x - \mu_X)^2 f_X(x) dx.$$

标准差 (standard deviation) 记为

$$\sigma_X := \text{STD}[X] = \sqrt{\text{VAR}[X]}.$$

均匀随机变量满足

$$\text{VAR}[X] = \int_a^b \left(x - \frac{b+a}{2}\right)^2 \frac{dx}{b-a} = \frac{(b-a)^2}{12}.$$

对 Gaussian 随机变量, 先用

$$\int_{-\infty}^{\infty} \exp \left[-\frac{1}{2} \left(\frac{x-\mu}{\sigma} \right)^2 \right] dx = \sqrt{2\pi} \sigma.$$

两边对 σ 求导, 得

$$\int_{-\infty}^{\infty} \exp \left[-\frac{1}{2} \left(\frac{x-\mu}{\sigma} \right)^2 \right] (x-\mu)^2 \sigma^{-3} dx = \sqrt{2\pi},$$

整理后得到

$$\text{VAR}[X] = \int_{-\infty}^{\infty} (x-\mu)^2 \frac{1}{\sqrt{2\pi}\sigma} \exp \left[-\frac{1}{2} \left(\frac{x-\mu}{\sigma} \right)^2 \right] dx = \sigma^2.$$

7.2.5 随机变量对

概率机器学习常涉及多个随机变量, 例如输入、输出与权重都可能是随机变量. 因而需把单个随机变量的概念推广到随机变量向量. 先考虑随机变量对; 这里的大多数概念都可直接推广到随机变量向量.

随机变量对是从事件类为 $\mathcal{E}$ 的测度空间出发的映射 (源文把定义域写作 $\mathcal{E}$)

$$\mathbf{X} : \mathcal{E} \rightarrow \mathbb{R}^2,$$

它可以是离散的或连续的, 有时写作 $\mathbf{X} = (X, Y)$. 例如, 转动转盘两次, 测得 $\psi_1, \psi_2 \in (0, 2\pi]$, 可定义

$$X(\psi_1) = \frac{\psi_1}{2\pi}, \quad Y(\psi_2) = \frac{\psi_2}{2\pi}.$$

$\mathbf{X}$ 的事件是 $\mathbb{R}^2$ 中的集合. 为计算事件概率, 定义联合累积分布函数 (joint cumulative distribution function) $F_{XY} : \mathbb{R}^2 \rightarrow \mathbb{R}$:

$$F_{XY}(x, y) = P[X \leq x, Y \leq y] = P[\xi \in \mathbb{S} : X(\xi) \leq x, Y(\xi) \leq y].$$

与单个随机变量类似:

- 源文称联合 cdf 是 x, y 的非增函数, 但随即给出: 若 $x \geq x', y \geq y'$, 则 $F_{XY}(x, y) \geq F_{XY}(x', y')$.

- $\lim_{x \rightarrow -\infty} F(x, y) = 0, \lim_{y \rightarrow -\infty} F(x, y) = 0, \lim_{x, y \rightarrow \infty} F(x, y) = 1.$

•

$$P[x_1 < X \leq x_2, y_1 < Y \leq y_2] = F_{XY}(x_2, y_2) + F_{XY}(x_1, y_1) - F_{XY}(x_1, y_2) - F_{XY}(x_2, y_1).$$

若 X, Y 联合连续, 联合概率密度函数 (joint probability density function) 定义为

$$f_{XY}(x, y) = \frac{\partial^2 F_{XY}(x, y)}{\partial x \partial y}.$$

其性质为:

- 当 $x \rightarrow \pm\infty$ 或 $y \rightarrow \pm\infty$ 时, $f_{XY}(x, y) = 0.$
- $F_{XY}(x, y) = \int_{-\infty}^x \int_{-\infty}^y f_{XY}(r, s) dr ds.$
- $\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f_{XY}(r, s) dr ds = 1.$
- $P[x_1 < X \leq x_2, y_1 < Y \leq y_2] = \int_{x_1}^{x_2} \int_{y_1}^{y_2} f_{XY}(x, y) dx dy.$
- $P[\mathbf{X} \in B] = \iint_B f_{XY}(x, y) dx dy.$

两个重要例子如下.

1. 联合均匀随机变量 (joint uniform random variable): 在区域 $(a, b] \times (c, d]$ 上,

$$f_{XY}(x, y) = \begin{cases} \frac{1}{(b-a)(d-c)}, & (x, y) \in (a, b] \times (c, d], \\ 0, & \text{其他.} \end{cases} \quad (7.5)$$

2. 联合 Gaussian 随机变量 (joint Gaussian random variable):

$$f_{XY}(x, y) = \frac{1}{\sqrt{(2\pi)^2 \det(\Sigma)}} \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right],$$

其中 $\mathbf{x} = (x, y)$, $\boldsymbol{\mu} = (\mu_x, \mu_y)$ 是均值, 协方差矩阵为

$$\Sigma = \begin{bmatrix} \sigma_x^2 & \rho\sigma_x\sigma_y \\ \rho\sigma_x\sigma_y & \sigma_y^2 \end{bmatrix}.$$

该矩阵对称正定.

X 的边缘概率密度函数 (marginal probability density function) 是让 Y 取遍所有可能值后得到的 X 的 pdf:

$$f_X(x) = \int_{-\infty}^{\infty} f_{XY}(x, y) dy, \quad f_Y(y) = \int_{-\infty}^{\infty} f_{XY}(x, y) dx.$$

若 $f_{XY}(x, y) = f_X(x)f_Y(y)$, 则 X, Y 相互独立 (independent) .

问题 7.2.1: 证明式 (7.5) 的联合均匀随机变量相互独立.

若 $g(\mathbf{X})$ 为标量、向量或张量值函数, 只要积分有定义,

$$\mathbb{E}[g(\mathbf{X})] = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} g(\mathbf{x}) f_{XY}(x, y) dx dy.$$

例如:

- $g(\mathbf{X}) = X$ 时, $\mathbb{E}[g(\mathbf{X})] = \iint x f_{XY}(x, y) dx dy$.
- $g(\mathbf{X}) = \mathbf{X}$ 时, $\mathbb{E}[g(\mathbf{X})] = [\mathbb{E}[X], \mathbb{E}[Y]]$.
- $g(\mathbf{X}) = X + Y$ 时, $\mathbb{E}[g(\mathbf{X})] = \mathbb{E}[X] + \mathbb{E}[Y]$.

$\mathbf{X}$ 的协方差 (covariance) 为

$$\text{COV}[\mathbf{X}] = \mathbb{E}[(\mathbf{X} - \mathbb{E}[\mathbf{X}]) \otimes (\mathbf{X} - \mathbb{E}[\mathbf{X}])],$$

其中

$$\text{COV}[\mathbf{X}]_{11} = \mathbb{E}[(X - \mathbb{E}[X])^2] = \text{VAR}[X],$$

$$\text{COV}[\mathbf{X}]_{22} = \mathbb{E}[(Y - \mathbb{E}[Y])^2] = \text{VAR}[Y],$$

$$\text{COV}[\mathbf{X}]_{12} = \text{COV}[\mathbf{X}]_{21} = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])].$$

若 $\text{COV}[\mathbf{X}]_{12} = 0$, 称 X, Y 不相关. 独立随机变量满足该等式; 但必须注意, 协方差为零并不蕴含独立.

最后考虑已知 $X = \hat{x}$ 时 Y 的 pdf. $f_{XY}(\hat{x}, y)$ 不一定对 y 积分为 1, 因此定义条件概率密度函数 (conditional probability density function):

$$f_{Y|X}(y | \hat{x}) = \frac{f_{XY}(\hat{x}, y)}{\int_{-\infty}^{\infty} f_{XY}(\hat{x}, y) dy} = \frac{f_{XY}(\hat{x}, y)}{f_X(\hat{x})},$$

$$f_{X|Y}(x | \hat{y}) = \frac{f_{XY}(x, \hat{y})}{f_Y(\hat{y})}.$$

把回归问题推广到概率框架, 便需要在给定一个输入实例时确定输出的条件分布; 后文监督概率深度学习部分将讨论这一点.

7.3 无监督概率深度学习算法

先考虑具有 N_X 个分量、pdf 为 f_X 的随机向量 $\mathbf{X}$. 假设给定从密度 f_X 抽取的样本数据集 $\{\mathbf{x}_i\}$, 记作 $\mathbf{x}_i \sim f_X$. 例如, 这些样本可以是分辨率为 512×512 的汽车 RGB 图像. 此时样本所在空间的维数为 $N_X = 512 \times 512 \times 3$, 非常高. 可把每个图像像素视为一个随机变量, 其取值是三个通道上的像素强度. 因而, 这些图像可视为某个未知密度 f_X 所描述的 N_X 维随机变量的样本. 又由于图像中物体 (这里是汽车) 的内在结构, 多维随机变量的各分量预计高度相关, 使 f_X 具有非平凡形式. 这种相关性也说明, 也许可以把 $\mathbf{X}$ 的维数从 N_X 降至较小数目, 从而简化表示.

我们的目标是用有限样本集 $\{\mathbf{x}_i\}$ 学习数据密度 f_X , 并从学得分布生成新样本. 这类方法称为生成算法 (generative algorithm). 生成算法有很多种; 这里聚焦一类称为生成对抗网络 (Generative Adversarial Network, GAN) 的深度学习算法.

7.3.1 生成对抗网络

GAN 最早由 Goodfellow 等人于 2014 年提出 [6]. 此后出现了许多 GAN 变体, 它们采用不同的网络架构与训练目标函数. 下面先描述抽象问题, 再介绍 GAN 的架构与训练过程.

考虑数据集

$$\mathcal{S} = \{\mathbf{x}_i \in \Omega_X \subset \mathbb{R}^{N_X} : 1 \leq i \leq N_{\text{train}}\}.$$

假设这些样本是密度为 f_X 的随机变量 $\mathbf{X}$ 的实现, 即 $\mathbf{x}_i \sim f_X$. 我们希望训练 GAN 学得 f_X 并从中生成新样本.

GAN 通常由两个子网络组成: 生成器 (generator) 和判别器 (或评论器) (discriminator (or critic)). 生成器是如下网络:

$$g(\cdot; \boldsymbol{\theta}_g) : \Omega_Z \rightarrow \Omega_X, \quad g : \mathbf{z} \mapsto \mathbf{x}. \quad (7.6)$$

其中 $\boldsymbol{\theta}_g$ 是可训练参数, $\mathbf{z} \in \Omega_Z \subset \mathbb{R}^{N_Z}$ 是服从简单分布的随机变量 $\mathbf{Z}$ 的实现, 例如密度为

$$f_Z(\mathbf{z}) = \frac{1}{\sqrt{(2\pi)^2 \det(\boldsymbol{\Sigma})}} \exp \left[-\frac{1}{2}(\mathbf{z} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{z} - \boldsymbol{\mu}) \right], \quad \boldsymbol{\mu} = \mathbf{0}, \quad \boldsymbol{\Sigma} = \mathbf{I}$$

的不相关多元 Gaussian 分布. 通常 $N_Z \ll N_X$, $\mathbf{Z}$ 称为 GAN 的潜变量 (latent variable).

生成器架构取决于 $\mathbf{x}$ 的大小与形状. 若 $\mathbf{x}$ 是向量, g 可取输入维数为 N_Z 、输出维数为 N_X 的 MLP. 若 $\mathbf{x}$ 是形状为 $H \times W \times 3$ 的图像, 则生成器先经过若干全连接层, 再重塑成通道数很多的粗糙图像; 随后通过若干转置卷积层, 逐步提高空间分辨率并压缩通道数, 最终放大至 $H \times W \times 3$. 这也称为解码器架构, 与 U-Net 的上升支路相似 (见图 5.9).

无论哪种情形, 固定 $\boldsymbol{\theta}_g$ 后, 生成器都把随机变量 $\mathbf{Z}$ 变换为另一随机变量

$$\mathbf{X}^g = g(\mathbf{Z}; \boldsymbol{\theta}_g),$$

其密度记为 f_X^g , 对应于潜在密度 f_Z 在 g 下的前推 (push-forward). 我们希望选择 $\boldsymbol{\theta}_g$, 使 f_X^g 接近未知目标分布 f_X .

评论器网络为

$$d(\cdot; \boldsymbol{\theta}_d) : \Omega_X \rightarrow \mathbb{R}, \quad (7.7)$$

其中 $\boldsymbol{\theta}_d$ 是可训练参数. 评论器架构同样取决于 $\mathbf{x}$ 的形状. 若 $\mathbf{x}$ 是向量, d 可取输入维数为 N_X 、输出维数为 1 的 MLP. 若 $\mathbf{x}$ 是图像, 评论器可由若干卷积层、一个展平层和若干全连接层组成. 它类似于图 5.7 的 CNN 架构, 但输出为标量且没有输出函数.

图 7.3 展示了 GAN 及其子网络的输入和输出. 生成器与评论器扮演对抗角色: 评论器接受训练, 以区分来自 f_X 的真实样本与生成器 g 所生成、密度为 f_X^g 的伪样本; 生成器则试图生成类似于 f_X 样本的逼真样本, 以欺骗评论器.

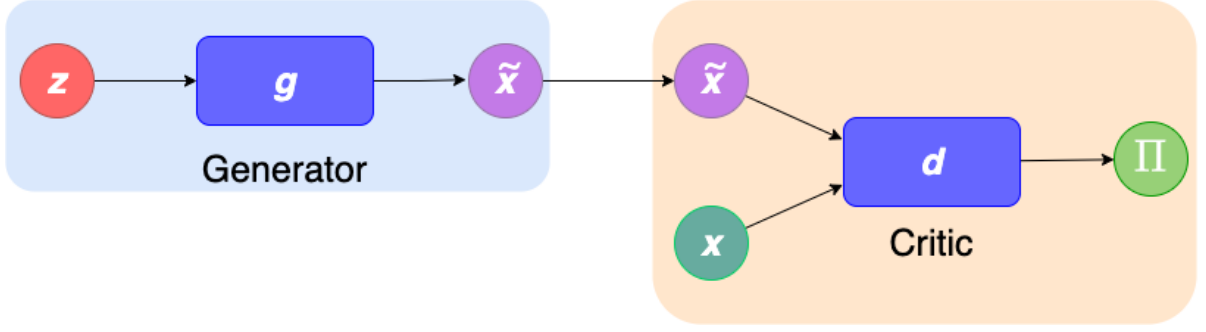


图 7.3: GAN 示意图.

下面定义 Wasserstein 生成对抗网络 (Wasserstein GAN, WGAN) 的目标函数 [2]. 与原始 GAN 相比, WGAN 具有更好的稳健性与收敛性质. 其目标函数为

$$\Pi(\theta_g, \theta_d) = \underbrace{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} d(\mathbf{x}_i; \theta_d)}_{\text{评论器在真实样本上的取值}} - \underbrace{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} d(g(\mathbf{z}_i; \theta_g); \theta_d)}_{\text{评论器在伪样本上的取值}}. \quad (7.8)$$

其中 $\mathbf{x}_i \in \mathcal{S}$ 来自真实目标分布 f_X , 而 $\mathbf{z}_i \sim f_Z$ 经 g 生成伪样本. 为区分真实样本与伪样本, 评论器在真实样本上取较大正值, 在生成的伪样本上取较大负值. 因而评论器通过最大化目标函数来训练, 即对任意 θ_g 求解

$$\theta_d^*(\theta_g) = \arg \max_{\theta_d} \Pi(\theta_g, \theta_d). \quad (7.9)$$

评论器的最优参数依赖于 θ_g . 为欺骗评论器, 生成器 g 尝试最小化目标函数:

$$\theta_g^* = \arg \min_{\theta_g} \Pi(\theta_g, \theta_d^*). \quad (7.10)$$

因此, 训练 WGAN 就是求解一个极小极大优化问题 (minmax optimization problem). 评论器试图最大化目标函数, 生成器则试图最小化它, 二者以对抗方式工作; 这正是“生成对抗网络”名称的由来.

实际计算中需向评论器损失加入稳定项, 因此评论器最大化

$$\Pi_c(\theta_g, \theta_d) = \Pi(\theta_g, \theta_d) - \frac{\lambda}{N} \sum_{i=1}^N \left(\left\| \frac{\partial d}{\partial \hat{\mathbf{x}}}(\hat{\mathbf{x}}_i; \theta_d) \right\| - 1 \right)^2, \quad (7.11)$$

其中 $\hat{\mathbf{x}}_i = \alpha \mathbf{x}_i + (1 - \alpha)g(\mathbf{z}_i; \theta_g)$, α 从 $(0, 1)$ 上的均匀随机变量抽取. 式 (7.11) 的附加项称为梯度惩罚项 (gradient penalty term), 用于约束评论器 d 关于输入的梯度范数接近 1, 从而使其成为 1-Lipschitz 函数. 详情参见文献 [7].

算法 1 同时训练 g 与 d , 该迭代方法也称为交替最速下降 (alternating steepest descent); η_d 、 η_g 分别是评论器与生成器的学习率. 评论器先执行 $K > 1$ 个优化步, 生成器再执行一个优化步. 这是为了先求解内部最大化问题, 使评论器能够区分真实样本与伪样本. 很大的 K 虽会更精确地求解极小极大问题, 却会使中等规模网络的训练在计算上不可行; 实践中通常取 $4 \leq K \leq 6$.

算法 1 (GAN 训练算法): 输入: $\theta_d^0, \theta_g^0, K, N_{\text{epochs}}, \eta_d, \eta_g$.

1. 对 $n = 1, \dots, N_{\text{epochs}}$, 执行:

(a) 令 $\hat{\theta}_d \leftarrow \theta_d^{(n-1)}$.

(b) 对 $k = 1, \dots, K$, 执行最大化更新

$$\hat{\theta}_d \leftarrow \hat{\theta}_d + \eta_d \frac{\partial \Pi_c}{\partial \theta_d}(\theta_g^{(n-1)}, \hat{\theta}_d).$$

(c) 令 $\theta_d^{(n)} \leftarrow \hat{\theta}_d$.

(d) 执行最小化更新

$$\theta_g^n \leftarrow \theta_g^{(n-1)} - \eta_g \frac{\partial \Pi}{\partial \theta_g}(\theta_g^{(n-1)}, \theta_d^{(n)}).$$

极小极大问题难以求解, 通常需训练很多轮次才能收敛. 也可在训练数据的小批量上执行评论器优化, 每个轮次使用许多小批量, 从而用较少轮次获得近似数量的优化步. 随迭代进行, d 越来越善于发现伪样本, g 则越来越善于生成可欺骗评论器的样本.

在容量无限 ($N_{\theta_g}, N_{\theta_d} \rightarrow \infty$)、数据无限 ($N_{\text{train}} \rightarrow \infty$) 且优化器完美的假设下, 可以证明生成分布 f_X^g 弱收敛 (converges weakly) 到目标分布 f_X [2]. 等价地, 对 Ω_X 上每个连续有界函数 $\ell \in C_b(\Omega_X)$,

$$\mathbb{E}_Z[\ell(g(\mathbf{Z}; \theta_g^*))] \longrightarrow \mathbb{E}_X[\ell(\mathbf{X})]. \quad (7.12)$$

GAN 训练完成后, 先抽取 $\mathbf{z} \sim f_Z$, 再通过生成器得到 $\mathbf{x} = g(\mathbf{z}; \theta_g^*)$, 即可用优化后的 g 从 $f_X^g \approx f_X$ 生成新样本. 由上述弱收敛, 生成样本的统计量 (均值、方差等) 也将收敛到 f_X 的真实统计量.

注 7.3.1: 这里有三点重要说明:

1. GAN 训练完成后, 通常只保留生成器, 不再需要评论器. 训练评论器的主要作用, 是得到能够生成逼真样本的适当 g .
2. WGAN 名称中的 “Wasserstein” 源于: 可以证明, 求解极小极大问题等价于最小化 f_X^g 与 f_X 之间的 Wasserstein-1 距离 [2, 28]. Wasserstein-1 距离是衡量两个概率测度差异的一种常用度量.
3. 潜变量维数 N_Z 通常远小于 Ω_X 中样本的维数 N_X , 故训练后的生成器还提供了高维数据的低维表示, 这对若干下游任务很有用 [21, 22].

◇

7.4 监督概率深度学习算法

回顾确定性问题：给定带标签的成对数据集

$$\mathcal{S} = \{(\mathbf{x}_i, \mathbf{y}_i) : \mathbf{x}_i \in \Omega_X \subset \mathbb{R}^{N_X}, \mathbf{y}_i \in \Omega_Y \subset \mathbb{R}^{N_Y}\}_{i=1}^{N_{\text{train}}}, \quad (7.13)$$

希望对不在 $\mathcal{S}$ 中的新 $\mathbf{x}$ 求出 $\mathbf{y}$. 前几章已经说明神经网络如何求解这种回归（或分类）问题. 注意，式 (7.13) 的成对数据写作 $(\mathbf{x}_i, \mathbf{y}_i)$ ，但源 PDF 在成员条件中写的是无下标的 $\mathbf{y}$.

现在考虑该问题的概率版本. 假设 $\mathbf{x}, \mathbf{y}$ 分别由随机变量 $\mathbf{X}, \mathbf{Y}$ 建模，并且式 (7.13) 中的成对样本来自未知联合分布 f_{XY} . 给定一个实现 $\mathbf{X} = \hat{\mathbf{x}}$ ，我们希望用 $\mathcal{S}$ 确定条件分布 $f_{Y|X}(\mathbf{y} | \hat{\mathbf{x}})$ ，并从中生成样本.

这类概率问题有多种常用解法，例如 Bayesian 神经网络、变分推断、dropout 或深度 Boltzmann 机. 这里聚焦一种也能处理此类问题的 GAN 扩展.

7.4.1 条件生成对抗网络

条件生成对抗网络（conditional GAN）最早由文献 [17] 提出，用于学习条件分布. 这里讨论一种特殊变体，即条件 Wasserstein 生成对抗网络（conditional Wasserstein GAN, cWGAN）；它由文献 [1] 提出，并在文献 [25] 中用于求解若干基于物理的（反）问题.

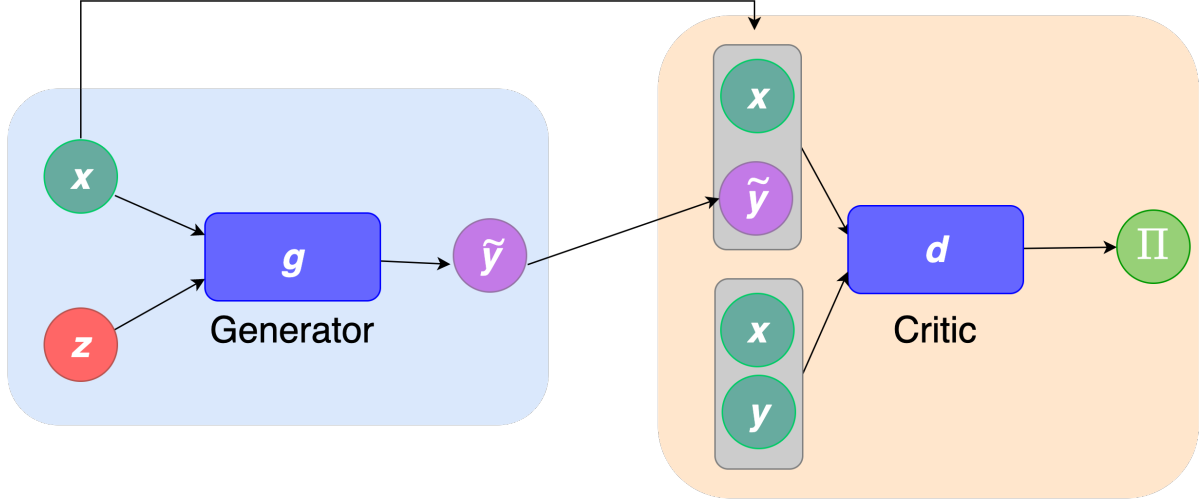


图 7.4: 条件 GAN 示意图.

图 7.4 展示了条件 GAN 的结构. 生成器是如下网络:

$$g(\cdot; \theta_g) : \Omega_Z \times \Omega_X \rightarrow \Omega_Y, \quad g : (z, x) \mapsto y. \quad (7.14)$$

其中 $z \sim f_Z$ 是潜变量. 与 GAN 不同，条件 GAN 的生成器还把 $\mathbf{x}$ 作为输入. 给定 $\mathbf{X} = \hat{\mathbf{x}}$ ，从 f_Z 抽取 z 会由某个诱导条件分布 $f_{Y|X}^g(\mathbf{y} | \hat{\mathbf{x}})$ 生成多个 $\mathbf{y}$ 样本. 目标是选定参数 θ_g ，使得对（几乎）每个 $\hat{\mathbf{x}}$ ， $f_{Y|X}^g(\mathbf{y} | \hat{\mathbf{x}})$ 都逼近真实条件分布 $f_{Y|X}(\mathbf{y} | \hat{\mathbf{x}})$.

评论器网络为

$$d(\cdot; \boldsymbol{\theta}_d) : \Omega_X \times \Omega_Y \rightarrow \mathbb{R}. \quad (7.15)$$

它接受训练，以区分来自真实联合分布 f_{XY} 的成对样本 $(\mathbf{x}, \mathbf{y})$ 与伪样本对 $(\mathbf{x}, \hat{\mathbf{y}})$ ；其中 $\hat{\mathbf{y}}$ 是生成器 g 在给定真实 $\mathbf{x}$ 后生成的。

cWGAN 的目标函数为

$$\Pi(\boldsymbol{\theta}_g, \boldsymbol{\theta}_d) = \underbrace{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} d(\mathbf{x}_i, \mathbf{y}_i; \boldsymbol{\theta}_d)}_{\text{评论器在真实样本对上的取值}} - \underbrace{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} d(\mathbf{x}_i, g(\mathbf{z}_i, \mathbf{x}_i; \boldsymbol{\theta}_g); \boldsymbol{\theta}_d)}_{\text{评论器在伪样本对上的取值}}. \quad (7.16)$$

与前文一样，评论器最大化式 (7.9) 所示的目标函数，生成器则按式 (7.10) 将其最小化。优化评论器时还需加入稳定化梯度惩罚项，参见文献 [25]。生成器与评论器使用 GAN 部分所述的交替最速下降算法训练。

在容量无限 ($N_{\boldsymbol{\theta}_g}, N_{\boldsymbol{\theta}_d} \rightarrow \infty$)、数据无限 ($N_{\text{train}} \rightarrow \infty$) 且优化器完美的假设下，可以证明 [1]：对给定的 $\mathbf{X} = \hat{\mathbf{x}}$ ，生成条件分布 $f_{Y|X}^g(\mathbf{y} | \hat{\mathbf{x}})$ 以弱意义（平均而言）收敛到目标条件分布 $f_{Y|X}(\mathbf{y} | \hat{\mathbf{x}})$ 。

参考文献

- [1] J. ADLER AND O. ÖKTEM, *Deep bayesian inversion*. <https://arxiv.org/abs/1811.05910>, 2018.
- [2] M. ARJOVSKY, S. CHINTALA, AND L. BOTTOU, *Wasserstein generative adversarial networks*, in Proceedings of the 34th International Conference on Machine Learning, D. Precup and Y. W. Teh, eds., vol. 70 of Proceedings of Machine Learning Research, International Convention Centre, Sydney, Australia, 06–11 Aug 2017, PMLR, pp. 214–223.
- [3] R. BISCHOF AND M. KRAUS, *Multi-objective loss balancing for physics-informed deep learning*. <http://rgdoi.net/10.13140/RG.2.2.20057.24169>, 2021.
- [4] R. T. Q. CHEN, Y. RUBANOVA, J. BETTENCOURT, AND D. DUVENAUD, *Neural ordinary differential equations*. <https://arxiv.org/abs/1806.07366>, 2018.
- [5] T. CHEN AND H. CHEN, *Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems*, IEEE Transactions on Neural Networks, 6 (1995), pp. 911–917.
- [6] I. GOODFELLOW, J. POUGET-ABADIE, M. MIRZA, B. XU, D. WARDE-FARLEY, S. OZAI, A. COURVILLE, AND Y. BENGIO, *Generative adversarial nets*, in Advances in neural information processing systems, 2014, pp. 2672–2680.
- [7] I. GULRAJANI, F. AHMED, M. ARJOVSKY, V. DUMOULIN, AND A. C. COURVILLE, *Improved training of wasserstein gans*, in Advances in neural information processing systems, 2017, pp. 5767–5777.
- [8] K. HE, X. ZHANG, S. REN, AND J. SUN, *Deep residual learning for image recognition*, in 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- [9] P. KIDGER AND T. LYONS, *Universal Approximation with Deep Narrow Networks*, in Proceedings of Thirty Third Conference on Learning Theory, J. Abernethy and S. Agarwal, eds., vol. 125 of Proceedings of Machine Learning Research, PMLR, 09–12 Jul 2020, pp. 2306–2327.
- [10] D. P. KINGMA AND J. BA, *Adam: A method for stochastic optimization*. <https://arxiv.org/abs/1412.6980v9>, 2017.
- [11] I. LAGARIS, A. LIKAS, AND D. PAPAGEORGIOU, *Neural-network methods for boundary value problems with irregular boundaries*, IEEE Transactions on Neural Networks, 11 (2000), pp. 1041–1049.

- [12] S. LANTHALER, S. MISHRA, AND G. E. KARNIADAKIS, *Error estimates for Deep-ONets: a deep learning framework in infinite dimensions*, Transactions of Mathematics and Its Applications, 6 (2022).
- [13] Z. LI, N. KOVACHKI, K. AZIZZADENESHELI, B. LIU, K. BHATTACHARYA, A. STUART, AND A. ANANDKUMAR, *Fourier neural operator for parametric partial differential equations*. <https://arxiv.org/abs/2010.08895>, 2020.
- [14] L. LU, P. JIN, G. PANG, Z. ZHANG, AND G. E. KARNIADAKIS, *Learning nonlinear operators via deeponet based on the universal approximation theorem of operators*, Nature Machine Intelligence, 3 (2021), pp. 218–229.
- [15] A. L. MAAS, A. Y. HANNUN, A. Y. NG, ET AL., *Rectifier nonlinearities improve neural network acoustic models*, in Proc. ICML, vol. 30, 2013.
- [16] L. MCCLENNY AND U. BRAGA-NETO, *Self-adaptive physics-informed neural networks using a soft attention mechanism*. <https://arxiv.org/abs/2009.04544>, 2020.
- [17] M. MIRZA AND S. OSINDERO, *Conditional generative adversarial nets*. <https://arxiv.org/abs/1411.1784>, 2014.
- [18] S. MISHRA AND R. MOLINARO, *Estimates on the generalization error of physics-informed neural networks for approximating PDEs*, IMA Journal of Numerical Analysis, (2022).
- [19] A. NEMIROVSKI, A. JUDITSKY, G. LAN, AND A. SHAPIRO, *Robust stochastic approximation approach to stochastic programming*, SIAM Journal on Optimization, 19 (2009), pp. 1574–1609.
- [20] D. PATEL, D. RAY, M. R. A. ABDELMALIK, T. J. R. HUGHES, AND A. A. OBERAI, *Variationally mimetic operator networks*. <https://arxiv.org/abs/2209.12871>, 2022.
- [21] D. V. PATEL AND A. A. OBERAI, *Gan-based priors for quantifying uncertainty in supervised learning*, SIAM/ASA Journal on Uncertainty Quantification, 9 (2021), pp. 1314–1343.
- [22] D. V. PATEL, D. RAY, AND A. A. OBERAI, *Solution of physics-based bayesian inverse problems with deep generative priors*, Computer Methods in Applied Mechanics and Engineering, 400 (2022), p. 115428.
- [23] A. PINKUS, *Approximation theory of the mlp model in neural networks*, Acta Numerica, 8 (1999), pp. 143–195.

- [24] M. RAISSI, P. PERDIKARIS, AND G. KARNIADAKIS, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, 378 (2019), pp. 686–707.
- [25] D. RAY, H. RAMASWAMY, D. V. PATEL, AND A. A. OBERAI, *The efficacy and generalizability of conditional gans for posterior inference in physics-based inverse problems*. <https://arxiv.org/abs/2202.07773>, 2022.
- [26] O. RONNEBERGER, P. FISCHER, AND T. BROX, *U-net: Convolutional networks for biomedical image segmentation*, in Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, eds., Cham, 2015, Springer International Publishing, pp. 234–241.
- [27] V. SITZMANN, J. N. P. MARTEL, A. W. BERGMAN, D. B. LINDELL, AND G. WETZSTEIN, *Implicit neural representations with periodic activation functions*. <https://arxiv.org/abs/2006.09661>, 2020.
- [28] C. VILLANI, *Optimal Transport: Old and New*, Grundlehren der mathematischen Wissenschaften, Springer Berlin Heidelberg, 2008.
- [29] S. WANG, Y. TENG, AND P. PERDIKARIS, *Understanding and mitigating gradient flow pathologies in physics-informed neural networks*, SIAM Journal on Scientific Computing, 43 (2021), pp. A3055–A3081.
- [30] S. WANG, H. WANG, AND P. PERDIKARIS, *Learning the solution operator of parametric partial differential equations with physics-informed deepnets*, Science Advances, 7 (2021).
- [31] L. WU, C. MA, AND W. E, *How sgd selects the global minima in over-parameterized learning: A dynamical stability perspective*, in Advances in Neural Information Processing Systems, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, eds., vol. 31, Curran Associates, Inc., 2018.
- [32] M. YANG, K. YU, C. ZHANG, Z. LI, AND K. YANG, *Denseaspp for semantic segmentation in street scenes*, in 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 3684–3692.
- [33] D. YAROTSKY AND A. ZHEVNERCHUK, *The phase diagram of approximation rates for deep neural networks*. <https://arxiv.org/abs/1906.09477>, 2019.

术语索引

中文术语	英文术语	页码
计算物理	computational physics	2
机器学习	machine learning	2
物理定律	physical law	2
分类	classification	3
人工智能	artificial intelligence	4
语义分割	semantic segmentation	4
深度学习	deep learning	5
前馈网络	feed-forward network	5
多层感知机	multilayer perceptron, MLP	7
人工神经元	artificial neuron	7
源层	source layer	7
输出层	output layer	7
隐藏层	hidden layer	7
权重	weight	7
偏置	bias	7
激活函数	activation function	7
深度	depth	8
神经元死亡	dying neuron	9
超参数	hyper-parameter	9
训练	training	13
验证	validation	13
测试	testing	13
训练集	training set	13
验证集	validation set	13
测试集	test set	13
过拟合	over-fitting	13
泛化	generalize	13
损失函数	loss function	13

中文术语	英文术语	页码
泛化误差	generalizing error	14
泛化能力	generalizability	14
正则化	regularization	14
过参数化	over-parametrized	15
梯度下降	gradient descent, GD	16
学习率	learning rate	16
动量方法	momentum method	18
Adam 优化算法	Adam optimization	18
随机梯度下降	stochastic gradient descent, SGD	19
小批量优化	mini-batch optimization	20
轮次	epoch	20
前向传播	forward pass	21
反向传播	back-propagation	22
softmax 函数	softmax function	23
独热编码	one-hot encoding	23
交叉熵	cross-entropy	23
残差神经网络 (ResNet)	residual neural network, ResNet	25
最大奇异值	largest singular value	25
梯度消失	vanishing gradient	25
恒等映射	identity mapping	26
零映射	null mapping	26
跳跃连接	skip connection	26
时间积分器	time integrator	28
向前欧拉法	forward Euler method	28
神经 ODE	neural ODE	28
时间推进格式	time-marching scheme	28
对流速度	advective velocity	31
Péclet 数 (Pe)	Péclet number, Pe	31
有限差分格式	finite difference scheme	32
中心差分	central difference	32

中文术语	英文术语	页码
迎风近似	upwind approximation	32
模板	stencil	33
配置点	collocation point	33
谱配置法	spectral collocation method	34
物理信息神经网络 (PINN)	physics-informed neural network, PINN	36
软约束	soft constraint	38
数据同化	data assimilation	40
卷积层	convolution layer	42
卷积算子	convolution operator	42
核函数	kernel function	42
各向同性的	isotropic	43
填充	padding	47
步幅	stride	47
池化	pooling	47
展平	flattening	49
特征选择	feature selection	49
编码器	encoder	49
涡量平方积分	enstrophy	50
参数共享	parameter sharing	50
下采样	downsampling	51
上采样	upsampling	51
转置卷积层	transpose convolution layer	51
棋盘格效应	checker-boarding	52
分数步幅层	fractionally-strided layer	52
编码器-解码器网络	encoder-decoder network	53
函数到函数的算子	operator mapping functions to functions	54
算子网络	operator network	56
深度算子网络	Deep Operator Network, DeepONet	56
Fourier 神经算子	Fourier Neural Operator, FNO	56

中文术语	英文术语	页码
分支网络	branch net	56
主干网络	trunk net	56
物理信息 DeepONet	physics-informed DeepONet	59
随机试验	random experiment	65
结果	outcome	65
样本空间	sample space	65
事件	event	65
事件类	event class	66
概率律	probability law	66
随机变量	random variable	67
Bernoulli 随机变量	Bernoulli random variable	67
累积分布函数	cumulative distribution function, cdf	67
概率密度函数	probability density function, pdf	69
均匀随机变量	uniform random variable	69
指数随机变量	exponential random variable	69
Gaussian 随机变量	Gaussian random variable	69
期望值	expected value, expectation, mean	70
方差	variance	70
标准差	standard deviation	71
联合累积分布函数	joint cumulative distribution function	71
联合概率密度函数	joint probability density function	72
联合均匀随机变量	joint uniform random variable	72
联合 Gaussian 随机变量	joint Gaussian random variable	72
边缘概率密度函数	marginal probability density function	72
相互独立	independent	72
协方差	covariance	73
条件概率密度函数	conditional probability density function	73
生成算法	generative algorithm	73
生成对抗网络	Generative Adversarial Network, GAN	73
生成器	generator	74

中文术语	英文术语	页码
判别器 (或评论器)	discriminator (or critic)	74
潜变量	latent variable	74
前推	push-forward	74
Wasserstein 生成对抗网络	Wasserstein GAN, WGAN	75
极小极大优化问题	minmax optimization problem	75
梯度惩罚项	gradient penalty term	75
交替最速下降	alternating steepest descent	75
弱收敛	converges weakly	76
条件生成对抗网络	conditional GAN	77
条件 Wasserstein 生成对抗网络	conditional Wasserstein GAN, cWGAN	77